

TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG
ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÀI GIẢNG HỌC PHẦN
ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

Nhóm tác giả:
TS. Bùi Thị Mai Anh
TS. Trần Nhật Hoá

Lời nói đầu

Trong biên soạn không thể tránh khỏi sai sót và nhầm lẫn mong bạn đọc cho ý kiến. Mọi góp ý gửi về địa chỉ:

Hà Nội, ngày 1 tháng 06 năm 2024

Nhóm tác giả

Những kí hiệu

Trong cuốn sách này ta dùng những kí hiệu với các ý nghĩa xác định trong bảng dưới đây:

$\mathbb{N}$	Tập hợp số tự nhiên
$\mathbb{R}$	Tập hợp số thực
$[k]$	$\{1, 2, \dots, k\}$, tập hợp các số nguyên dương không vượt quá k
$\emptyset$	Tập hợp rỗng
$\mathbb{R}^n$	Không gian các vectơ thực n chiều
$\mathbf{x}$	Một vectơ trong không gian nhiều chiều
$\ \mathbf{x}\ $	Chuẩn 2 của vectơ $\mathbf{x}$, tức là $\ \mathbf{x}\ = \sqrt{x_1^2 + \dots + x_n^2}$ khi cho trước vectơ $\mathbf{x} = (x_1, \dots, x_n)$
$\mathbf{A}^T$	Chuyển vị của ma trận $\mathbf{A}$
$\langle \mathbf{x}, \mathbf{w} \rangle$	Tích vô hướng của hai vectơ $\mathbf{x}$ và $\mathbf{w}$ có cùng số chiều. Đôi khi ta có dùng ký hiệu $\langle \mathbf{x} \cdot \mathbf{w} \rangle$ hoặc $\mathbf{x}^T \mathbf{w}$.
$\Pr(X = a)$	Xác suất để biến ngẫu nhiên X nhận giá trị cụ thể a
$\mathbb{E}_x[y(x)]$	Kỳ vọng của hàm y theo phân bố của x
n	Số chiều của mỗi vectơ đầu vào
m	Số lượng mẫu trong tập huấn luyện
$\mathbf{D}$	Tập huấn luyện (tập học)
ℓ	Hàm mất mát, hàm lỗi

Mục lục

Lời nói đầu	3
Những kí hiệu	4
Mục lục	5
1 Tổng quan về Đảm bảo chất lượng phần mềm	9
1.1 Giới thiệu	9
1.2 Các thuật ngữ cơ bản	11
1.3 Quan điểm về chất lượng sản phẩm	12
1.4 Định nghĩa về chất lượng sản phẩm phần mềm	12
1.5 Các mô hình chất lượng	13
1.5.1 Mô hình McCall	14
1.5.2 Mô hình chất lượng Boehm	21
1.5.3 Mô hình chất lượng Dromey	22
1.5.4 Mô hình chất lượng ISO/IEC 9126	22
1.5.5 So sánh 2 mô hình McCall và ISO 9126	24
1.5.6 Mô hình FURPS của HP	25
1.5.7 Mô hình GQM	25
1.6 Quy trình đảm bảo chất lượng phần mềm	27
1.6.1 Vai trò của QA trong quá trình quản lý dự án	27
1.6.2 Vai trò của QA trong quản lý yêu cầu phần mềm	28
1.6.3 QA đối với quy trình quản lý cấu hình	28
1.6.4 Vai trò của QA đối với quy trình phát triển mã nguồn	29
1.6.5 Quy trình kiểm thử là nền tảng của quy trình QA	29
2 Độ đo phần mềm	32

2.1	Giới thiệu chung về độ đo phần mềm	32
2.2	Độ đo độ phức tạp mã nguồn	34
2.2.1	Động cơ	34
2.2.2	Phân loại	34
2.2.3	Chỉ số bảo trì	41
2.3	Độ đo hướng đối tượng	43
2.3.1	Các độ đo hướng đối tượng HK	44
2.3.2	Các độ đo hướng đối tượng CK	46
2.4	Độ đo gói	51
2.4.1	Stable-Dependencies Principle	51
2.4.2	Stable Abstraction Principle	52
3	Giới thiệu về kiểm thử phần mềm	55
3.1	Định nghĩa kiểm thử phần mềm	55
3.2	Các thuật ngữ trong kiểm thử phần mềm	57
3.3	Các hoạt động kiểm thử	61
3.3.1	Nguồn dữ liệu sử dụng để thiết kế ca kiểm thử	62
3.3.2	Yêu cầu và đặc tả chức năng	63
3.3.3	Mã nguồn	64
3.3.4	Miền dữ liệu đầu vào và đầu ra	64
3.3.5	Hồ sơ vận hành	65
3.3.6	Mô hình lỗi	66
3.3.7	Các kĩ thuật thiết kế ca kiểm thử	67
3.3.8	Lập kế hoạch kiểm thử	67
3.3.9	Quản lý và đánh giá kiểm thử	68
3.4	Các mức độ kiểm thử	69
3.4.1	Kiểm thử đơn vị	69
3.4.2	Kiểm thử hệ thống	70
3.4.3	Kiểm thử hồi quy	71
3.4.4	Kiểm thử chấp nhận	72
4	Các kĩ thuật kiểm thử hộp đen	73

MỤC LỤC	7
4.1 Giới thiệu	73
4.2 Phương pháp phân lớp tương đương	74
4.2.1 Giới thiệu	74
4.2.2 Phương pháp thực hiện	74
4.2.3 Ưu và nhược điểm	77
4.3 Phương pháp phân tích giá trị biên	78
4.3.1 Giới thiệu	78
4.3.2 Các bước thực hiện	79
4.3.3 Ưu và nhược điểm	80
4.4 Phương pháp bảng quyết định	82
4.4.1 Giới thiệu	82
4.4.2 Thiết kế test case	83
4.4.3 Ưu và nhược điểm	84
4.5 Phương pháp biểu đồ trạng thái	85
4.5.1 Giới thiệu	85
4.5.2 Áp dụng phương pháp biểu đồ trạng thái	86
4.5.3 Các bước thiết kế kiểm thử trạng thái	86
5 Các kĩ thuật kiểm thử hộp trắng	87
5.1 Giới thiệu	87
5.2 Kiểm thử luồng điều khiển	89
5.2.1 Các thuật ngữ cơ bản	89
5.2.2 Kĩ thuật kiểm thử luồng điều khiển	90
5.2.3 Quy trình kiểm thử luồng điều khiển	92
5.2.4 Tiêu chí lựa chọn đường đi	92
5.2.5 Tiêu chí bao phủ điều kiện - Predicate Coverage	97
5.2.6 Mệnh đề chủ động	102
5.3 Kiểm thử luồng dữ liệu	113
5.3.1 Động lực	113
5.3.2 Các định nghĩa	114
5.3.3 Các tiêu chí kiểm thử bao phủ luồng dữ liệu	117

6	Độ tin cậy phần mềm	125
6.1	Định nghĩa độ tin cậy phần mềm - Software Reliability	125
6.2	Mô hình toán học cho độ tin cậy phần mềm	131
6.3	Các nhân tố ảnh hưởng đến độ tin cậy phần mềm	133
6.4	Hồ sơ vận hành - Operational Profile	138
7	Quy trình kiểm thử phần mềm	142
7.1	Vòng đời kiểm thử phần mềm (Software Testing Lifecycle – STLC) .	142
7.2	Lập kế hoạch và tổ chức kiểm thử	145
7.3	Kiểm thử hệ thống - System testing	149
7.4	Kiểm thử chấp nhận (Acceptance Testing)	157
	Danh mục từ khóa	161

Chương 1

Tổng quan về Đảm bảo chất lượng phần mềm

1.1	Giới thiệu	9
1.2	Các thuật ngữ cơ bản	11
1.3	Quan điểm về chất lượng sản phẩm	12
1.4	Định nghĩa về chất lượng sản phẩm phần mềm	12
1.5	Các mô hình chất lượng	13
1.6	Quy trình đảm bảo chất lượng phần mềm	27

1.1 Giới thiệu

Đảm bảo chất lượng phần mềm là một quy trình cho phép đánh giá và ghi lại chất lượng của các sản phẩm công việc trong mỗi giai đoạn phát triển của vòng đời phần mềm. Nội dung của môn học này sẽ tập trung vào một số vấn đề quan trọng sau:

- Chất lượng phần mềm – Các tiêu chí về chất lượng phần mềm - Chúng ta sẽ tìm hiểu về các tiêu chí tiêu chuẩn chung để đánh giá ước lượng chất lượng của phần mềm ví dụ: các tiêu chuẩn ISO/IEC 9126, 12207, các tiêu chuẩn của IEEE
- Các mô hình đánh giá chất lượng phần mềm ví dụ mô hình McCall, Boehm, Dromey, FURPS, GQM...

- Độ đo chất lượng phần mềm cho phép đánh giá chất lượng phần mềm bằng những phép đo cụ thể ví dụ: độ đo Halstead, McCabe, Henry & Kafura. . .
- Kiểm thử phần mềm – Các vấn đề về kiểm thử, pha kiểm thử, các loại kiểm thử khác nhau trong quy trình phát triển phần mềm như: kiểm thử đơn vị, kiểm thử tích hợp, kiểm thử hệ thống, kiểm thử chấp nhận.
- Các phương pháp và kĩ thuật kiểm thử phần mềm – tập trung vào hai lớp kĩ thuật kiểm thử hộp đen với các phương pháp phân tích giá trị biên, phân lớp tương đương, bang quyết định và kiểm thử hộp trắng với các phương pháp bao phủ luồng điều khiển, bao phủ luồng dữ liệu, độ phức tạp chương trình, phương pháp mutation testing
- Quy trình và vòng đời kiểm thử phần mềm – Quy trình kiểm thử theo mô hình chữ V, các phương pháp kiểm thử nhanh (Agile Testing Methods)
- Các công cụ kiểm thử phần mềm – Tìm hiểu về các công cụ kiểm thử tự động, kiểm thử hồi quy, kiểm thử ứng dụng web. . .

Môn học cung cấp cho sinh viên những kiến thức và kĩ năng cơ bản và nâng cao trong việc kiểm thử để phát triển các hệ thống phần mềm quy mô lớn. Môn học cũng cho phép sinh viên luyện tập với các kĩ thuật và phương pháp kiểm thử phần mềm thông qua các buổi làm bài tập (lab sessions) về kiểm thử hộp đen và kiểm thử hộp trắng. Sinh viên cũng sẽ tham gia làm việc nhóm để làm quen với các tài liệu đặc tả kiểm thử của phần mềm, xây dựng các kế hoạch kiểm thử và trường hợp kiểm thử cho một dự án phần mềm.

Yêu cầu đối với sinh viên về mục tiêu kiến thức và mức độ hiểu biết sau khoá học:

- Nắm được các khái niệm cơ bản và các kiến thức chung về kiểm thử phần mềm.
- Nắm được và sử dụng được các kĩ thuật phổ biến trong việc thiết kế các trường hợp kiểm thử ở các pha kiểm thử khác nhau của phần mềm.
- Hiểu được và mô tả được mối quan hệ giữa các quy trình kiểm thử phần mềm và các quy trình khác trong vòng đời phát triển phần mềm (ví dụ: phân tích yêu cầu, thiết kế, cài đặt, bảo trì. . .).
- Hiểu được và mô tả được mối quan hệ giữa kiểm chứng và các yêu cầu phần mềm về mặt chất lượng và chức năng.

- Có thể áp dụng các kiến thức về kiểm thử và đảm bảo chất lượng trong dự án thực tế (course project), giải thích được những chiến lược và lựa chọn cụ thể của mình trong dự án và xây dựng được tài liệu đặc tả kiểm thử phần mềm cho dự án.

Về mục tiêu kĩ năng và khả năng, sau khi kết thúc môn học, sinh viên cần có khả năng:

- Lựa chọn và giải thích được các chiến lược test được áp dụng đối với một hệ thống phần mềm cụ thể (sinh viên sẽ được thực hành với các bài toán cụ thể)
- Phân tích và thiết kế được các trường hợp kiểm thử một cách đầy đủ và tối ưu (các kịch bản test phải là tối thiểu) đối với kĩ thuật kiểm thử hộp đen và kiểm thử hộp trắng
- Hiểu được và triển khai được các kĩ thuật kiểm thử phi thực thi walkthrough, inspection
- Áp dụng một cách có hệ thống quy trình kiểm thử vào dự án thực tế

1.2 Các thuật ngữ cơ bản

Hai khái niệm đầu tiên luôn được nhắc đến trong kiểm thử và đảm bảo chất lượng phần mềm: Validation và Verification. Verification có thể được dịch ra tiếng Việt là xác minh, kiểm tra và Validation có thể dịch là kiểm chứng, cũng có thể dịch là xác minh. Trong tiếng Việt có nghĩa gần như giống nhau nhưng lại là hai khái niệm khác hẳn nhau trong quy trình phát triển phần mềm:

- Verification là hoạt động đánh giá hệ thống phần mềm bằng cách xác định xem hệ thống có thoả mãn các yêu cầu được nêu ra ban đầu hay không, được nhìn từ quan điểm của người phát triển. Sản phẩm ở đây có thể là một sản phẩm trung gian trong quá trình phát triển phần mềm như: tài liệu đặc tả, tài liệu thiết kế, mã nguồn chương trình,...
- Validation là hoạt động xác nhận xem sản phẩm phần mềm có thoả mãn tất cả các yêu cầu của người dung hay không? Nói một cách khác các hoạt động validation tập trung vào sản phẩm cuối cùng, được thực hiện kiểm chứng từ quan điểm của người dùng

Khái niệm về lỗi: Failure – Fault – Error – Defect

- Failure là một sự kiện xảy ra bất cứ khi nào hệ thống có một hành vi không tương ứng với các hành vi đặc tả của hệ thống
- Error là một trạng thái của hệ thống gây ra bởi một Fault.
- Fault – Defect hay Bug: đều là lỗi – đó là những bất thường trong phần mềm. Fault cũng có thể là các human error – Những lỗi gây ra bởi các lập trình viên ví dụ như lỗi logic, lỗi cú pháp ...

1.3 Quan điểm về chất lượng sản phẩm

- Quan điểm siêu việt về chất lượng: chất lượng là một thứ gì đó mà chúng ta có thể nhận ra nhưng lại không thể định nghĩa. Nói đơn giản thì chúng ta biết thế nào là một phần mềm tốt nhưng chúng ta không bao giờ có thể cài đặt hay xây dựng được hết các chức năng lí tưởng của nó.
- Quan điểm người dùng: Người dùng bao giờ cũng có một cái nhìn rất cụ thể về chất lượng của sản phẩm mà họ muốn. Một phần mềm có chất lượng là khi các mục đích của phần mềm đều đạt được. Khi đánh giá chất lượng theo quan điểm của người dùng, chúng ta cần phải trả lời câu hỏi: phần mềm có thoả mãn các yêu cầu và mong đợi của người dùng hay không?
- Quan điểm sản xuất: chất lượng được hiểu là những gì tương thích với đặc tả
- Quan điểm sản phẩm: chất lượng được xem là gắn liền với các đặc tính cố hữu của sản phẩm.
- Quan điểm về giá trị sản phẩm: chất lượng phụ thuộc vào số tiền mà khách hàng có thể bỏ ra để trả cho sản phẩm của mình.

1.4 Định nghĩa về chất lượng sản phẩm phần mềm

Một cách chung nhất thì chất lượng của một sản phẩm được hiểu là khi sản phẩm đạt được tất cả các yêu cầu đề ra trong đặc tả của nó. Theo định nghĩa của IEEE thì chất lượng phần mềm gồm hai ý:

- Mức độ một hệ thống hay một thành phần hay một tiến trình đạt được các yêu cầu đặt ra
- Mức độ một hệ thống hay một thành phần hay một tiến trình đạt được những mong đợi của khách hàng hay người dùng hệ thống đó

Như vậy chất lượng của phần mềm theo định nghĩa của IEEE được đánh giá từ hai quan điểm: quan điểm của người phát triển phần mềm (internal view) và quan điểm của người dùng (external view). Chất lượng phần mềm gắn liền với người phát triển và người dùng phần mềm. Vấn đề là luôn tồn tại căng thẳng giữa yêu cầu của người dùng (tính hiệu quả, độ tin cậy...) đối với sản phẩm phần mềm và yêu cầu về phát triển (dễ tái sử dụng, dễ bảo trì...). Người dùng quan tâm đến các tiêu chuẩn về sử dụng phần mềm phải đảm bảo tính an toàn, tin cậy,... còn người phát triển quan tâm đến các tiêu chí về chất lượng phát triển như làm thế nào để tái sử dụng các module qua các dự án khác nhau, làm thế nào để dễ dàng bảo trì phần mềm...

Vấn đề tiếp theo là đặc tả của sản phẩm phần mềm không bao giờ đầy đủ ngay từ đầu và thường không nhất quán. Do vậy để có được những tiêu chuẩn đánh giá chất lượng ngay từ đầu là rất khó mà việc đảm bảo chất lượng sẽ được hình thành và mở rộng dần dần trong suốt vòng đời phần mềm. Một vấn đề khác nữa là các yêu cầu về chất lượng của phần mềm thường rất khó để có thể đặc tả một cách rõ ràng, cụ thể.

1.5 Các mô hình chất lượng

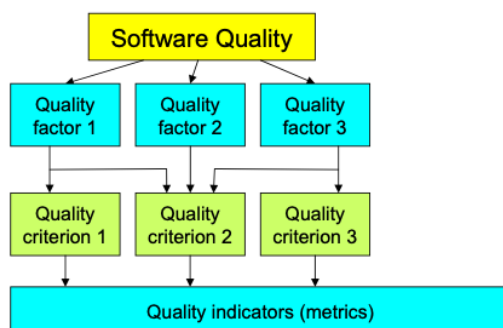
Những định nghĩa hình thức về chất lượng phần mềm như định nghĩa của IEEE thì thực sự không đủ để có thể áp dụng vào thực tế trong việc đánh giá chất lượng của các dự án phần mềm. Vì vậy chất lượng phần mềm cần được đặc tả bằng các mô hình chất lượng. Một mô hình chất lượng là một tập hợp các quy chuẩn về tiêu chuẩn, nhân tố, và các độ đo cần thiết để có thể đánh giá được chất lượng phần mềm.

Có hai cách tiếp cận chính:

- Các mô hình chuẩn: ví dụ như các mô hình của McCall, ISO/IEC 9126, Boehm và Dromey
- Các mô hình được áp dụng trong công nghiệp do các công ty phát triển phần mềm đề xuất dựa trên những thay đổi trên các mô hình chuẩn để phù hợp với quy mô và môi trường phát triển phần mềm của công ty mình. Ví dụ như mô hình FURPS của HP, GQM...

Một số khái niệm thường được sử dụng trong mô hình chất lượng:

- Nhân tố – Factors trong các mô hình chất lượng được sử dụng để mô tả góc nhìn từ phía bên ngoài đối với sản phẩm phần mềm hay góc nhìn từ quan



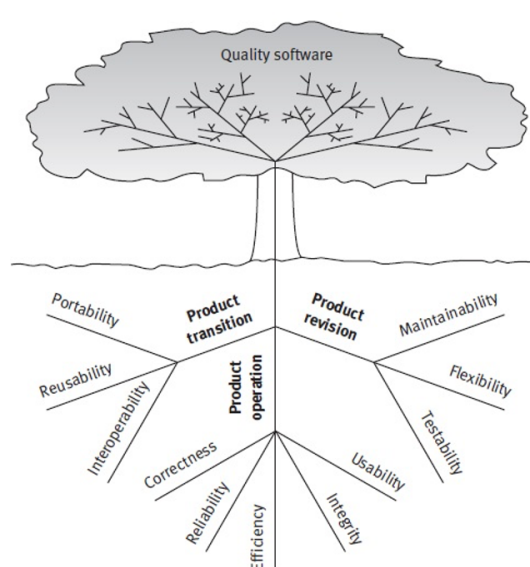
Hình 1.1: Mô hình chất lượng phần mềm

điểm người dùng sản phẩm. Các nhân tố này cho phép xác định các đặc tính chất lượng của phần mềm đứng từ quan điểm người dùng

- Tiêu chí – Criteria trong các mô hình chất lượng được sử dụng để mô tả góc nhìn từ bên trong của sản phẩm phần mềm, hay góc nhìn về quan điểm chất lượng phần mềm từ người phát triển sản phẩm. Các tiêu chí chất lượng cũng được coi như các đặc điểm được dùng để định nghĩa các nhân tố chất lượng.
- Độ đo – Metrics được định nghĩa và sử dụng để cung cấp một thước đo và phương pháp cụ thể để đo lường một cách định tính hoặc định lượng các tiêu chí chất lượng phần mềm

1.5.1 Mô hình McCall

Giữa những năm 70 thế kỷ trước, McCall, Richards và Walters là những người đầu tiên nghiên cứu về chất lượng phần mềm dưới dạng các nhân tố và tiêu chí chất lượng cụ thể. Theo McCall và các cộng sự, một nhân tố chất lượng (quality factor) diễn tả các đặc điểm về hành vi hoạt động của một hệ thống, ví dụ: tính đúng đắn - yêu cầu hệ thống phải chạy đúng như đặc tả về từng chức năng cụ thể, độ tin cậy yêu cầu phải có cùng một hành xử ứng với những thiết lập cụ thể về cấu hình, môi trường và trạng thái của hệ thống... Các nhân tố chất lượng do đó có thể được xem như các thuộc tính bên ngoài của sản phẩm. Các đối tượng khác nhau sẽ có các mối quan tâm khác nhau đối với các nhân tố chất lượng: ví dụ khách hàng thì quan tâm đặc biệt đến tính hiệu quả và độ tin cậy của hệ thống mà ít bận tâm đến vấn đề về tính khả chuyển – đưa hệ thống hoạt động được ở các môi trường khác nhau. Trong khi đó lập trình viên một mặt cố gắng đạt được các yêu cầu về độ tin cậy và tính hiệu quả của hệ thống mà khách hàng yêu cầu nhưng cũng phải đảm bảo được tính tái sử dụng, tính bảo trì được của sản phẩm để giảm thiểu chi phí phát triển



Hình 1.2: Mô hình McCall với 3 nhóm nhân tố chất lượng sản phẩm

phần mềm. Đội nhóm đảm bảo chất lượng thì quan tâm nhiều hơn đến tính kiểm thử được của hệ thống từ đó một vài nhân tố như tính chính xác, tính hiệu quả, độ tin cậy,... có thể dễ dàng xác minh được thông qua kiểm thử.

Mô hình McCall nhóm các nhân tố chất lượng thành 3 nhóm chính: Vận hành sản phẩm, Chuyển giao sản phẩm, Đánh giá sản phẩm. Các nhân tố chất lượng của mô hình McCall được định nghĩa như sau:

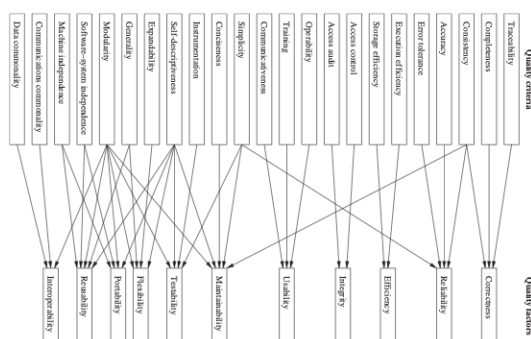
- **Correctness:** Tính đúng đắn – Yêu cầu một phần mềm phải đảm bảo được các yêu cầu chức năng được đặc tả một cách tường minh và các yêu cầu phi chức năng có thể được đặc tả một cách không tường minh. Nói cách khác nếu một phần mềm đảm bảo được hết các yêu cầu chức năng đặt ra, thì có thể gọi đó là một phần mềm xây dựng đúng. Tuy nhiên một phần mềm đúng chưa chắc đã đảm bảo được các yêu cầu phi chức năng khác của khách hàng như: độ tin cậy, hiệu năng... Thậm chí khách hàng có thể thông qua một sản phẩm chưa đúng nếu nó đảm bảo được hết các yêu cầu phi chức năng.
- **Reliability** – Độ tin cậy - Rất khó để có thể xây dựng một hệ thống phần mềm lớn một cách đúng đắn hoàn toàn (đảm bảo tất cả các yêu cầu chức năng đều đạt được). Một vài chức năng có thể không hoạt động trong một số kịch bản thực thi. Trong trường hợp đó phần mềm được coi là chưa đúng. Tuy nhiên khách hàng vẫn có thể thông qua các sản phẩm phần mềm chưa đúng bởi vì một số kịch bản thực thi làm một số chức năng của hệ thống bị fail không

thường xuyên xảy ra khi hệ thống được triển khai. Khi đó hệ thống đạt được một mức độ tin cậy nào đó trong một khoảng thời gian thực thi cụ thể. Khách hàng có thể dùng đó làm tiêu chí chất lượng để đánh giá độ tin cậy của hệ thống.

- Efficiency - Tính hiệu quả - liên quan đến một hệ thống phần mềm tiêu thụ bao nhiêu tài nguyên ví dụ: bộ nhớ trong, bộ nhớ ngoài, điện năng, băng thông ... Một hệ thống phần mềm cần phải sử dụng ít nhất có thể nguồn tài nguyên để thực hiện các chức năng của nó.
- Integrity – Tính toàn vẹn – đề cập đến khả năng chịu đựng được các cuộc tấn công vào tính bảo mật của hệ thống. Nói cách khác tính toàn vẹn đề cập đến khả năng có thể truy cập vào phần mềm hoặc dữ liệu của những người hoặc chương trình trái phép.
- Usability - Khả năng sử dụng được – Một hệ thống phần mềm được coi là sử dụng được nếu người dùng hệ thống thấy nó dễ sử dụng. Tính sử dụng được thường đề cập đến giao diện người dùng của hệ thống. Nếu không có một giao diện người dùng tốt, hệ thống phần mềm có thể bị xáo trộn ngay cả khi nó có những phẩm chất tốt mong muốn về tính an toàn, độ tin cậy, hiệu năng. Nhưng nếu chỉ có giao diện tốt mà không có những phẩm chất khác thì cũng không thể giữ được phần mềm trên thị trường.
- Maintainability – Khả năng bảo trì được – Bảo trì sản phẩm liên quan đến việc đáp ứng với những thay đổi của các thành phần phần mềm do quá trình phát triển liên tục của hệ thống. Khả năng bảo trì được của sản phẩm là một tiêu chí chất lượng đánh giá xem hệ thống phần mềm có thể dễ dàng bảo trì với chi phí thấp nhất có thể hay không. Đối với một sản phẩm phần mềm thì có 3 hoạt động bảo trì: khắc phục – thích nghi và hoàn thiện. Bảo trì khắc phục liên quan đến việc loại bỏ các lỗi các khiếm khuyết còn tồn tại trong phần mềm hoặc mới phát hiện trong quá trình triển khai sản phẩm. Bảo trì thích nghi liên quan đến các hoạt động điều chỉnh hệ thống phần mềm đáp ứng với các thay đổi trong môi trường thực thi. Cuối cùng các hoạt động bảo trì hoàn thiện nhằm thay đổi một số thành phần hoặc cả hệ thống để cải thiện một số phẩm chất của phần mềm.
- Testability – Khả năng kiểm thử được – Tất cả các yêu cầu của hệ thống cả những yêu cầu được định nghĩa một cách tường minh hoặc chỉ là những yêu cầu mong đợi, yêu cầu chức năng hoặc phi chức năng đều cần được xác minh. Khả năng kiểm thử được một hệ thống liên quan đến khả năng các yêu cầu

của hệ thống có thể được xác minh, kiểm chứng. Ở mọi giai đoạn của phát triển phần mềm, cần phải xem xét khía cạnh kiểm thử được của sản phẩm. Cụ thể đối với mỗi yêu cầu, chúng ta cần phải xác định ngay từ đầu quy trình nào cần thực hiện để kiểm chứng yêu cầu đó, và kiểm chứng như thế nào, kịch bản kiểm chứng ra sao...

- Flexibility – Tính linh hoạt – được phản ánh trong chi phí sửa đổi một hệ thống. Khi ngày càng có nhiều thay đổi cần phải thực hiện trong quá trình triển khai vận hành hệ thống, những thay đổi tiếp theo có thể tốn nhiều chi phí hơn nữa nếu như thiết kế ban đầu không linh hoạt, không dễ dàng thích ứng với các thay đổi. Để đo được tính linh hoạt của hệ thống người ta cần phải tìm cách trả lời được câu hỏi, làm thế nào để dễ dàng thêm một tính năng mới vào hệ thống.
- Portability – Tính khả chuyển của một hệ thống liên quan đến khả năng một hệ thống phần mềm có thể dễ dàng điều chỉnh để chạy được trong các môi trường thực thi khác nhau. Môi trường thực thi là một thuật ngữ rộng liên quan đến các nền tảng phần cứng, hệ điều hành, mạng, Tính khả chuyển rất quan trọng đối với các nhà phát triển vì một sự thích ứng nhỏ của hệ thống có thể làm tăng tiềm năng của nó đối với thị trường. Tính khả chuyển mang đến cho khách hàng nhiều lựa chọn để dễ dàng chuyển từ môi trường thực thi này sang môi trường thực thi khác để tận dụng tối đa các công nghệ mới trong việc thúc đẩy kinh doanh của họ. Các nguyên tắc thiết kế tốt như chia tách các mô-đun độc lập, thiết kế tương tác giữa các hệ thống độc lập môi trường thực thi... tạo điều kiện thuận lợi cho tính khả chuyển.
- Reusability – Tính tái sử dụng được – liên quan đến một thành phần hoặc một nhóm thành phần có thể được tái sử dụng lại với một số thay đổi nhỏ không đáng kể trong các hệ thống khác. Khả năng sử dụng lại của các thành phần giúp tiết kiệm chi phí và thời gian để phát triển và kiểm thử thành phần đó. Khả năng tái sử dụng không chỉ áp dụng cho các thành phần của sản phẩm mà cho cả các quy trình phát triển. Ví dụ hiện nay chúng ta quan tâm đến các quy trình tốt mà phần lớn các giai đoạn của nó có thể lặp lại trong suốt quá trình phát triển sản phẩm.
- Interoperability – Tính tương tác được – Trong thời đại mạng máy tính ngày nay, các phần mềm cô lập đang ngày càng trở nên hiếm hoi. Các hệ thống phần mềm ngày nay thường được kết hợp ở mức độ đầu vào – đầu ra đối với các hệ thống khác. Một cách trực quan, khả năng tương tác có nghĩa là có



Hình 1.3: 23 tiêu chí chất lượng McCall

hay không đầu ra của một hệ thống được chấp nhận là đầu vào của một hệ thống khác. Khi xem xét các ứng dụng chạy trên Internet hoặc mạng không dây, tính tương tác là rất quan trọng. Ví dụ: người dùng các gói xử lý tài liệu như Latex, Word muốn nhập nhiều hình ảnh tạo ra bởi các gói đồ hoạ khác nhau. Do đó các gói đồ hoạ và các gói xử lý dữ liệu phải tương thích với nhau.

Tiêu chí chất lượng là một thuộc tính của một nhân tố chất lượng liên quan đến việc phát triển phần mềm. Ví dụ: tính mô đun hoá (modularity) là một thuộc tính của kiến trúc của một hệ thống phần mềm. Một phần mềm modun hoá cao cho phép các nhà sản xuất gắn kết các thành phần vào mô đun làm tăng tính bảo trì của hệ thống. Tương tự khả năng theo dõi của một yêu cầu phần mềm cho phép nhà phát triển ánh xạ chính xác yêu cầu tới một tập con mô đun do đó làm tăng tính đúng đắn của hệ thống. Mc Call đưa ra 23 tiêu chí chất lượng phần mềm sẽ được trình bày cụ thể trong phần tiếp theo.

- Access audit: kiểm tra truy cập – Việc kiểm chứng dữ liệu và phần mềm có tuân thủ theo các yêu cầu hay các tiêu chuẩn đặt ra có dễ dàng hay không
- Access control: kiểm soát truy cập – Quy định về việc kiểm soát và bảo vệ phần mềm và dữ liệu
- Accuracy – tính chính xác – Độ chính xác của các tính toán và đầu ra
- Communication Commonality – Độ tương đồng giao tiếp – Mức độ các giao thức và giao diện chuẩn được sử dụng trong hệ thống
- Completeness – Tính đầy đủ – Mức độ các chức năng của phần mềm / hệ thống được cài đặt đầy đủ hoàn toàn

- Communicativeness – Sự sẵn sàng giao tiếp thể hiện dữ liệu đầu vào và đầu ra có dễ dàng được đồng hoá hay không
- Conciseness – Tính súc tích – Mức độ nhỏ gọn của mã nguồn tính theo các dòng mã
- Consistency – Tính nhất quán – Việc sử dụng cùng các kĩ thuật thiết kế và cài đặt giống nhau xuyên suốt toàn bộ dự án – Thể hiện qua việc sử dụng nhất quán một phong cách thiết kế và cài đặt xuyên suốt toàn bộ dự án
- Data commonality – Độ tương đồng dữ liệu – Sử dụng các chuẩn để biểu diễn dữ liệu
- Error Tolerance – Mức độ vận hành liên tục được đảm bảo dưới các điều kiện bất lợi
- Execution Efficiency – Hiệu quả thực thi tính bằng thời gian chạy của phần mềm
- Expandability – Khả năng mở rộng được tính bằng mức độ các yêu cầu của phần mềm và các chức năng của phần mềm có thể được mở rộng
- Generality – Mức độ khái quát hoá được đo bằng bề rộng của ứng dụng tiềm năng của các thành phần phần mềm
- Hardware Independence – Độc lập phần cứng đc đo bằng mức độ phần mềm lệ thuộc vào các phần cứng của hệ thống
- Instrumentation - Thiết bị đo đạc – Mức độ hệ thống phần mềm cung cấp các khả năng để đo đạc việc sử dụng của nó hoặc khả năng xác định lỗi
- Modularity – Tính mô đun hoá – Khả năng cung cấp các mô đun độc lập cao
- Operability – Khả năng vận hành – Mức độ dễ dàng vận hành của phần mềm
- Self-documentation – Tự đặc tả tài liệu – Khả năng cung cấp các tài liệu mô tả và giải thích việc cài đặt các thành phần
- Simplicity – Tính đơn giản – Mức độ dễ dàng phần mềm có thể được hiểu
- Software System Independence – Sự độc lập phần mềm hệ thống thể hiện qua mức độ độc lập của hệ thống với các phần mềm môi trường như: hệ điều hành, thư viện, hệ quản trị CSDL...

- Traceability – Khả năng theo vết thể hiện ở khả năng ánh xạ tương ứng từ các thành phần của phần mềm sang các yêu cầu của phần mềm
- Training – Đào tạo thể hiện ở khả năng một người dùng mới có thể dễ dàng tiếp cận và sử dụng hệ thống phần mềm

Cải thiện một nhân tố chất lượng có thể làm ảnh hưởng tiêu cực đến các nhân tố chất lượng khác. Ví dụ: Chúng ta muốn tăng khả năng kiểm thử được của một phần mềm thì tính hiệu quả của phần mềm có thể bị giảm. Bởi vì để làm cho code có thể kiểm thử được, lập trình viên không thể viết code một cách súc tích ngắn gọn do đó tính hiệu quả của code có thể bị giảm. Tương tự để tăng tính khả chuyển của code, lập trình viên cần viết code theo cách dễ hiểu do đó cũng ko thể viết theo cách súc tích ngắn gọn và sẽ làm giảm tính hiệu quả của code. Một số nhân tố chất lượng có thể ảnh hưởng tích cực đến các nhân tố chất lượng khác. Ví dụ: Tăng tính đúng đắn của hệ thống sẽ tăng độ tin cậy của hệ thống đó hoặc tăng tính kiểm thử được sẽ cải thiện khả năng bảo trì của hệ thống.

Các yếu tố chất lượng không thể đo đạc được trực tiếp ví dụ chúng ta không thể đo đạc được mức độ kiểm thử được của một hệ thống phần mềm và mức độ kiểm thử được của hệ thống phần mềm cũng không thể đơn giản trả lời được bằng câu trả lời “có” hoặc “không”. Thay vì thế, mức độ kiểm thử được sẽ được đo đạc bằng cách gắn với nó một số độ đo ví dụ: tính đơn giản simplicity, tính mô đun hoá modularity, tính tự mô tả được self-descriptiveness.

Một độ đo chất lượng là một đơn vị đo cho phép đặc tả một số khía cạnh của một tiêu chí chất lượng. Các độ đo có thể được hình thành bằng cách:

- Xây dựng một tập các câu hỏi liên quan đến các tiêu chí chất lượng tương ứng và đưa ra các câu trả lời “có” hoặc “không” cho mỗi câu hỏi.
- Tính % số câu trả lời có để đạt được một độ đo cụ thể cho tiêu chí chất lượng tương ứng. Ví dụ: Chúng ta có thể đặt ra một số câu hỏi tương ứng với tính tự mô tả self-descriptiveness:
 - Liệu tất cả các tài liệu có được viết một cách rõ ràng và đơn giản để tất cả các chức năng, thuật toán, thủ tục đều dễ dàng hiểu được.
 - Liệu thiết kế phía sau mỗi module có rõ ràng và dễ hiểu?
 - Mỗi một câu hỏi có thể có một mức độ ảnh hưởng khác nhau đến độ đo chất lượng do đó có thể gắn với một trọng số cụ thể khi tính toán với câu trả lời “có” tương ứng của câu hỏi đó.

Factors	Criteria
Portability	Self-contentedness, device independence
Reliability	Self-contentedness, accuracy, completeness, robustness, integrity, consistency
Efficiency	Accountability, device efficiency, accessibility
Usability	Completeness
Testability (Human Engineering)	Accountability, communicativeness, self descriptiveness, structured
Understandability	Consistency, structured, conciseness
Modifiability (Flexibility)	Structured, expandability

Hình 1.4: Các nhân tố chất lượng mô hình Boehm

1.5.2 Mô hình chất lượng Boehm

Mô hình Boehm đơn giản hoá mô hình của McCall để có thể đánh giá một cách tự động và định lượng chất lượng của phần mềm. Trong mô hình này các yếu tố chất lượng được xếp vào 3 đặc tính chất lượng mức cao ví dụ: tính dễ sử dụng, tính bảo trì được và tính khả chuyển. 7 yếu tố chất lượng khác được phân vào 3 nhóm này gọi là các đặc điểm chất lượng mức trung gian

- Mô hình Boehm đơn giản hoá mô hình của McCall để có thể đánh giá một cách tự động và định lượng chất lượng của phần mềm.
- Trong mô hình này các yếu tố chất lượng được xếp vào 3 đặc tính chất lượng mức cao ví dụ: tính dễ sử dụng, tính bảo trì được và tính khả chuyển
- 7 yếu tố chất lượng khác được phân vào 3 nhóm này gọi là các đặc điểm chất lượng mức trung gian

Hình 1.4 thể hiện 7 nhân tố chất lượng của mô hình Boehm. Định nghĩa của các nhân tố thì tương ứng như trong mô hình McCall. Tương ứng với mỗi nhân tố là một số tiêu chí chất lượng được coi như thuộc tính của từng nhân tố.

Mô hình Boehm nhìn chung thu hẹp lại mô hình McCall để đánh giá chất lượng của phần mềm tự động và định lượng hơn

Factors	Criteria
Correctness	Functionality, Reliability
Internal	Maintainability, Efficiency, Reliability
Contextual	Maintainability, Reusability, Portability, Reliability
Descriptive	Maintainability, Efficiency, Reliability, Usability

Hình 1.5: Các nhân tố chất lượng trong mô hình Dromey

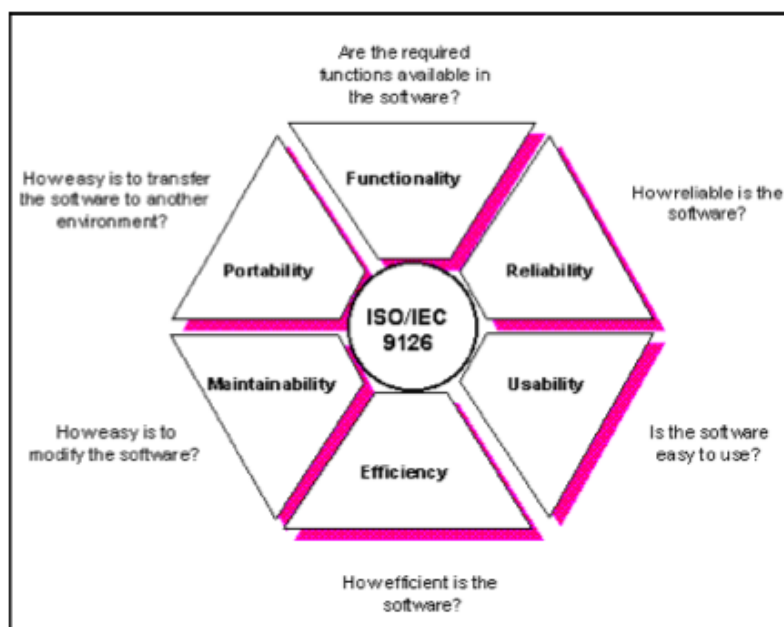
1.5.3 Mô hình chất lượng Dromey

Mô hình chất lượng của Dromey được đề nghị để đánh giá các pha phân tích yêu cầu, thiết kế và cài đặt của phát triển phần mềm. Mô hình này quan tâm đến các nhân tố chất lượng: Tính đúng đắn, Cấu trúc bên trong, Thuộc về ngữ cảnh, và mức độ đặc tả.

- Mô hình chất lượng của Dromey được đề nghị để đánh giá các pha phân tích yêu cầu, thiết kế và cài đặt của phát triển phần mềm.
- Các nhân tố chất lượng có liên quan đến việc cài đặt hệ thống phần mềm bao gồm:
 - Tính đúng đắn - Correctness
 - Cấu trúc bên trong - Internal
 - Tính ngữ cảnh - Contextual
 - Mức độ đặc tả - Descriptive

1.5.4 Mô hình chất lượng ISO/IEC 9126

Mô hình chất lượng ISO/IEC 9126 là một tập hợp các quy chuẩn quốc tế để đánh giá chất lượng của phần mềm. Mô hình này chia thành 6 nhóm độc lập, mỗi nhóm là một tập hợp các nhân tố chất lượng để đánh giá chất lượng phần mềm. Trong mô hình ISO/IEC 9126, chúng ta không gọi là quality factor, mà gọi

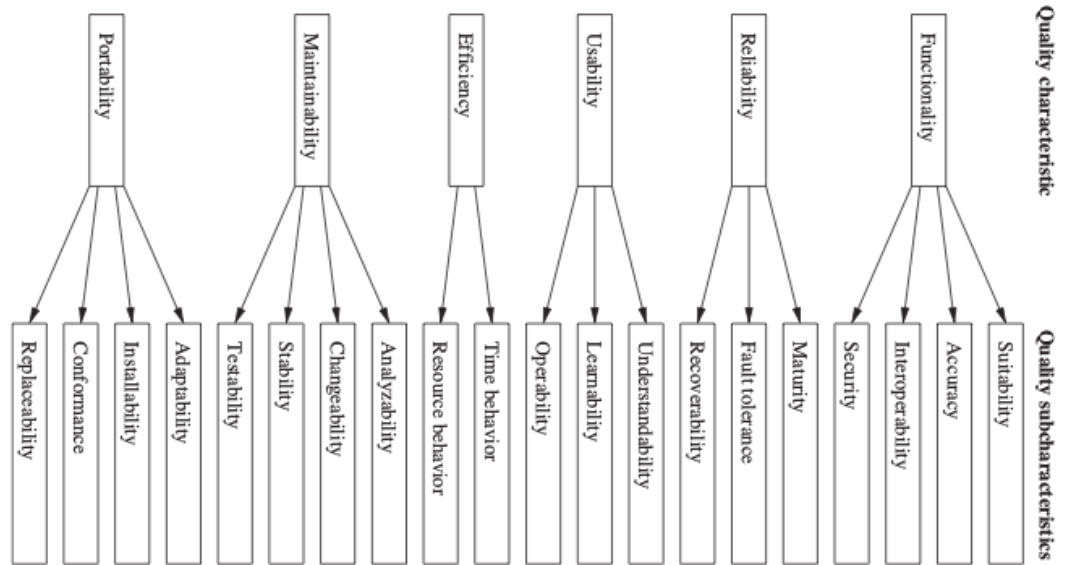


Hình 1.6: Mô hình ISO 9126

là characteristics – các đặc tính chất lượng. các tiêu chí chất lượng là các đặc tính con – sub-characteristics.

6 nhóm đặc tính này gồm có:

- **Functionality** – Tập hợp các chức năng và các thuộc tính tương ứng của các chức năng được đặc tả sẵn dựa trên yêu cầu của người dùng
- **Reliability** – Tập hợp các thuộc tính về khả năng của phần mềm để duy trì hiệu năng của nó dưới những điều kiện cụ thể trong một khoảng thời gian cụ thể
- **Usability** – Tập hợp các thuộc tính được xây dựng dựa trên các nỗ lực cần thiết của người dùng để có thể sử dụng được phần mềm
- **Efficiency** – Tính hiệu quả là một tập hợp các thuộc tính được xây dựng từ mối quan hệ giữa hiệu năng của phần mềm và lượng tài nguyên phần mềm sử dụng dưới những điều kiện cụ thể
- **Maintainability** – Tính bảo trì được của hệ thống bao gồm một tập hợp các thuộc tính được xây dựng từ những nỗ lực cần thiết phải bỏ ra để thực hiện những thay đổi (có thể là sửa lỗi, cải tiến hoặc đáp ứng của phần mềm với



Hình 1.7: Các thuộc tính con trong mô hình ISO9126

những thay đổi từ môi trường và yêu cầu hoặc đặc tả chức năng của người dùng)

- Portability – Tính khả chuyển là một tập hợp các thuộc tính xây dựng từ khả năng phần mềm có thể chuyển từ môi trường này sang môi trường khác (bao gồm phần cứng, hoặc phần mềm môi trường và hệ thống)

Từ nhóm các đặc tính chất lượng này, mô hình chất lượng của ISO/IEC 9126 cũng đề nghị 20 đặc tính chất lượng con tương ứng.

Mỗi một tổ chức, công ty khi phát triển dự án phần mềm cần phải định nghĩa những thuộc tính chất lượng và các thuộc tính chất lượng con phù hợp với nhu cầu của công ty hoặc dự án phần mềm. Nói một cách khác, các tổ chức phải xác định mức độ các đặc tính chất lượng khác nhau mà họ cần để đáp ứng được trong bối cảnh phát triển phần mềm của họ. Việc đạt đến một mức chất lượng lý tưởng nhất từ cấp độ hiện tại phải là một quá trình dần dần, do đó điều quan trọng là phải hiểu những yêu cầu cần thiết để chuyển sang bước tiếp theo và hướng tới mức chất lượng lý tưởng là gì.

1.5.5 So sánh 2 mô hình McCall và ISO 9126

Mô hình ISO 9126 nhấn mạnh các thuộc tính chất lượng có ý nghĩa đối với người dùng trong khi McCall model còn coi trọng các thuộc tính bên trong của sản phẩm.

Ví dụ, reusability, người phát triển cố gắng thiết kế một phần mềm sao cho nhiều thành phần có thể tái sử dụng được, nhưng đặc tính chất lượng đó thì không hề được người dùng nhận biết được

Trong McCall model, một tiêu chí chất lượng có thể ảnh hưởng đến nhiều nhân tố chất lượng trong khi trong mô hình ISO 9126, một thuộc tính chất lượng con chỉ ảnh hưởng duy nhất một thuộc tính chất lượng mức cao.

Trong mô hình McCall, tính kiểm thử được là một nhân tố chất lượng trong khi nó chỉ là một thuộc tính chất lượng con thuộc về thuộc tính bảo trì được trong mô hình ISO. Nhìn chung lại, không có một quy định cụ thể gì về việc đưa ra thuộc tính nào là thuộc tính mức cao và thuộc tính nào là thuộc tính mức thấp. Như trong mô hình McCall có tới 11 nhân tố chất lượng trong khi mô hình ISO chỉ xem xét có 6 thuộc tính chất lượng. Một vài thuộc tính chất lượng mức cao trong mô hình McCall thì chỉ quan trọng đối với người phát triển trong khi mô hình ISO 9126 lại chỉ đặc biệt quan tâm đến các thuộc tính gắn với sản phẩm và người dùng

1.5.6 Mô hình FURPS của HP

Một số mô hình khác mang tính ứng dụng cao hơn và được áp dụng trong công nghiệp, tùy thuộc quy mô của công ty và dự án phát triển phần mềm. Mô hình FURPS của HP được ra đời từ các thống kê về dự án của HP năm 1987 được dùng để cải thiện chất lượng sản phẩm của HP.

Các nhân tố chất lượng đưa ra gồm 5 nhân tố:

- Functionality – các chức năng của hệ thống
- Usability – Khả năng sử dụng liên quan đến tính nhất quán và tài liệu
- Reliability – Tính tin cậy liên quan đến tần suất xuất hiện failure và độ chính xác của đầu ra
- Performance – Hiệu năng liên quan đến thời gian đáp ứng và mức độ tiêu tốn tài nguyên
- Supportability – Khả năng hỗ trợ – liên quan đến khả năng mở rộng, đáp ứng và sửa chữa của hệ thống

1.5.7 Mô hình GQM

Mô hình GQM được đưa ra hướng tới tính áp dụng cao vì nó được xây dựng dựa trên mục đích của việc kiểm tra giám sát chất lượng. Mô hình này gồm có 3 bước:

Factors	Criteria
Functionality	Capability, security
Usability	Consistency, user documentation, training materials
Reliability	Frequency and security of failure, recoverability, predictability, accuracy, mean time between failure
Performance	Speed efficiency, availability, accuracy, throughput, response time, recovery time, resource usage
Supportability	Testability, extensibility, adaptability, maintainability, compatibility, configurability, serviceability, install ability, localizability

Hình 1.8: Caption

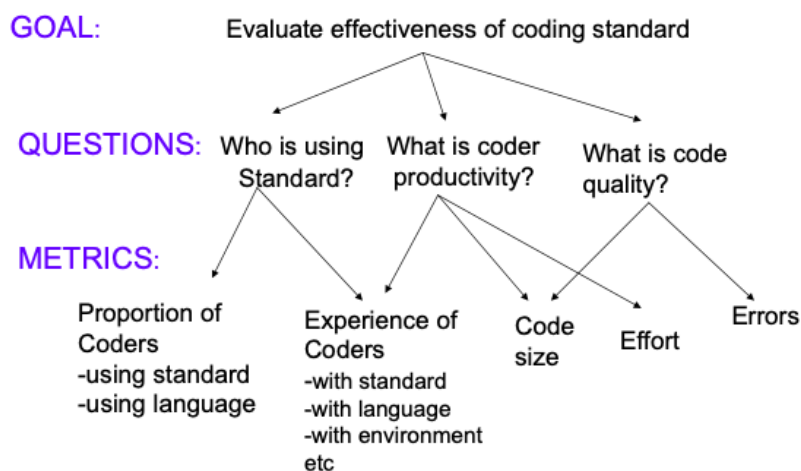
- Liệt kê tất cả các mục tiêu chính của dự án phát triển hoặc bảo trì
- Từ mỗi mục tiêu, cần đưa ra các danh sách câu hỏi có liên quan mà việc trả lời câu hỏi này sẽ xác định mục tiêu đó có đạt được hay không
- Quyết định cái gì cần được đo đạc để trả lời các câu hỏi tương ứng với mục tiêu

Mô hình GQM có các lợi điểm như sau:

- Vì mô hình này chỉ tập trung vào việc tạo ra các phép đo cần thiết để đánh giá được mục tiêu đề ra nên nó tương đối hiệu quả và ngắn gọn
- Nhiều phép đo khác nhau có thể cần thiết để trả lời được một câu hỏi đề ra
- Cùng một phép đo có thể được sử dụng để trả lời cho các câu hỏi khác nhau
- Mục tiêu chất lượng đề ra các mục đích cần thiết cho việc thu thập dữ liệu

Mô hình GQM thì có những vấn đề sau:

- Mô hình này cần kết hợp các phép đo theo một cách nào đó để có thể trả lời cho các câu hỏi do đó, đôi khi việc áp dụng là không dễ dàng và đòi hỏi kinh nghiệm của người phát triển và người quản lý dự án



Hình 1.9: Caption

- Nhiều khi cần đến nhiều model khác nhau để có thể diễn tả mối quan hệ giữa các phép đo.
- Mô hình này cần nhiều nỗ lực để có thể đưa ra được mục tiêu và các phép đo cụ thể
- So sánh với các mô hình chuẩn thì mô hình này có khả năng gây ra lỗi cao hơn

1.6 Quy trình đảm bảo chất lượng phần mềm

Quy trình đảm bảo phần mềm cần được tích hợp với tất cả các quy trình khác trong toàn bộ vòng đời phát triển phần mềm. Có 4 quy trình là: quản lý dự án – quản lý yêu cầu – quản lý cấu hình và quản lý mã nguồn

1.6.1 Vai trò của QA trong quá trình quản lý dự án

Ngay tại bước lập kế hoạch cho dự án, quy trình đảm bảo chất lượng cần được khởi động:

- Xác định các mục tiêu và tiêu chí chất lượng cần quan tâm và kiểm chứng trong dự án
- Xác định các thủ tục kiểm thử và đảm bảo chất lượng cần được áp dụng

- Chỉ định trước các nguồn lực cần thiết (bao gồm con người, phần cứng, hệ thống cơ sở hạ tầng và môi trường cần thiết để thực hiện kiểm thử và đảm bảo chất lượng)

1.6.2 Vai trò của QA trong quản lý yêu cầu phần mềm

Để đảm bảo quy trình QA được thực hiện xuyên suốt dự án thì việc đảm bảo cho các yêu cầu của sản phẩm cần được tổ chức một cách có hệ thống. Quy trình QA cần thực hiện các bước sau:

- Xác định tất cả các khía cạnh quan trọng của các chức năng của hệ thống và cố gắng để nhận được xác nhận từ phía khách hàng
- Xác định các hạn chế và ngoại lệ có thể có từ các yêu cầu của khách hàng
- Thiết lập tất cả các hiểu biết chung về dự án cho tất cả các thành viên của đội dự án, để đảm bảo đội dự án nắm được các vấn đề của dự án
- Cung cấp cơ sở cho các ước tính dự án, phát triển mã nguồn và các quy trình kiểm chứng
- Là một điểm khởi đầu cho việc giao tiếp hiệu quả và quản lý thay đổi một cách hiệu quả trong suốt quy trình phát triển sản phẩm

1.6.3 QA đối với quy trình quản lý cấu hình

Quản lý cấu hình phần mềm là bắt buộc trong mọi dự án phát triển phần mềm. Đảm bảo chất lượng phần mềm yêu cầu phải quản lý cấu hình một cách chặt chẽ và tuân theo các quy trình cụ thể.

- Để đảm bảo cho các đội nhóm phát triển phân tán có thể cùng làm việc trên một mã nguồn
- Đảm bảo cho baseline version có thể độc lập không bị ảnh hưởng bởi bất kỳ sự phá vỡ cấu trúc của một lỗi nào
- Để biết được ở phiên bản mã nguồn nào thì lỗi xuất hiện lần đầu tiên và xác định được những thay đổi nào làm xuất hiện lỗi đó
- Quản lý cấu hình tốt thì sẽ đảm bảo kiểm thử tự động thực hiện tốt nhất chức năng của nó ở mỗi và mọi giai đoạn mà version mã nguồn mới được ra đời

1.6.4 Vai trò của QA đối với quy trình phát triển mã nguồn

Áp dụng quy trình vào phát triển mã nguồn sẽ làm tăng hiệu quả của quy trình đảm bảo chất lượng phần mềm.

- Phát triển mã nguồn theo một kiểu thống nhất sẽ làm tăng hiệu quả và làm cho dễ dàng hơn việc bảo trì sản phẩm mà ko cần lệ thuộc cứng vào một hoặc một nhóm thành viên của dự án. Nhiều dự án phần mềm kéo dài nhiều năm, nên lệ thuộc rất nhiều vào thành viên nòng cốt của dự án, khi những thành viên này chuyển công tác dẫn tới rất nhiều vấn đề về quản lý mã nguồn nếu việc này không được thống nhất và làm cẩn thận theo quy trình ngay từ đầu
- Cho phép phát hiện lỗi sớm nhất và hiệu quả sửa lỗi là thấp nhất
- Cho phép thiết kế mã nguồn hướng kiểm thử
- Cho phép cục bộ hoá lỗi và giảm thiểu các tác động lên các thành phần phụ thuộc
- Chia sẻ tri thức giữa các thành viên trong đội dự án và cho phép cải tiến kỹ năng cá nhân của các thành viên

1.6.5 Quy trình kiểm thử là nền tảng của quy trình QA

Kiểm thử có thể được thực hiện trong các pha riêng biệt hoặc các pha chồng chéo

- Kiểm thử ở nhiều mức độ khác nhau của sản phẩm: đơn vị, tính năng, tính năng tương tác, hệ thống
- Kiểm thử trong suốt quá trình bảo trì sản phẩm: khói, sanity, hồi quy (regression)
- Kiểm thử các khía cạnh đặc biệt: hiệu năng, hiệu suất, độ ổn định, khả năng tương tác, độ tuân thủ các chuẩn/quy định

Việc quản lý kiểm thử cần được thực hiện ngay từ giai đoạn đầu tiên và kéo dài đến tận khi dự án hoàn toàn kết thúc nhằm đạt được các mục tiêu về cải tiến chất lượng sản phẩm. Các mục đích chính của quản lý kiểm thử:

- Chiến lược và phạm vi kiểm thử - Xác định các chiến lược kiểm thử và phạm vi các chức năng của dự án

- Các ràng buộc trong quá trình kiểm thử
- Các thành phần, các tính năng hoặc các chức năng cần phải kiểm thử và không cần phải kiểm thử
- Kiểm thử xem các mục nào thông qua và mục nào bị lỗi cần phải sửa chữa, thay đổi. Quy trình kiểm thử và các tiêu chí quay lại tiếp tục kiểm thử sau khi đã sửa lỗi
- Các công việc kiểm thử và các yêu cầu về môi trường và triển khai cần thiết để tiến hành kiểm thử
- Các trách nhiệm, phân công công việc và việc đào tạo cần thiết hoặc kết hợp với các đội nhóm phát triển khác trong quá trình kiểm thử
- Lên kế hoạch kiểm thử cụ thể và xác định các rủi ro có thể có
- Đánh giá bằng các độ đo chất lượng và viết báo cáo

Mục tiêu của việc chuẩn bị kiểm thử là để xác định được các loại kiểm thử và môi trường kiểm thử cần thiết cho từng pha phát triển dự án theo kế hoạch kiểm thử. Các hoạt động chính của chuẩn bị kiểm thử gồm có:

- Phân tích các yêu cầu chức năng, các đặc tả và phạm vi của các chức năng
- Phát triển ma trận theo vết yêu cầu (RTM – Requirement Traceability Matrix)
- Thiết kế và phát triển các trường hợp kiểm thử cho tất cả các pha đã lên kế hoạch
- Sử dụng các cách tiếp cận và các kỹ thuật để đảm bảo được một tập hợp các ca kiểm thử hiệu quả và tối ưu
- Phát triển hoặc sử dụng thêm các công cụ để thực hiện kiểm thử tự động
- Thiết lập các môi trường đặc biệt để có thể thực hiện kiểm thử phi chức năng ví dụ: thiết lập các kết nối tối để đo lường hiệu năng và độ tin cậy của hệ thống khi số truy nhập đến là nhiều

Áp dụng trực tiếp các định nghĩa và xác định của kiểm thử được đưa ra trong pha chuẩn bị kiểm thử vào quá trình phát triển sản phẩm. Các hoạt động thực thi kiểm thử thông thường:

- Chuẩn bị môi trường thực thi kiểm thử

-
- Thực thi các ca kiểm thử
 - Phân tích các kết quả thực thi
 - Thu thập và ghi lại các kết quả thực thi kiểm thử và ghi lại các lỗi
 - Hỗ trợ đội phát triển để tái hiện lại các lỗi và phân tích các lỗi
 - Kiểm chứng các ca kiểm thử đã được sửa chữa
 - Phân tích các lỗi có thể bị bỏ sót
 - Cập nhật các ca kiểm thử và môi trường kiểm thử theo kết quả của Kiểm thử thực thi và phân tích các lỗi có thể bỏ sót

Chương 2

Độ đo phần mềm

2.1	Giới thiệu chung về độ đo phần mềm	32
2.2	Độ đo độ phức tạp mã nguồn	34
2.3	Độ đo hướng đối tượng	43
2.4	Độ đo gói	51

2.1 Giới thiệu chung về độ đo phần mềm

Bài học này sẽ tập trung giới thiệu một số các độ đo phần mềm thông dụng và vai trò của các độ đo đó khi đo lường và đánh giá chất lượng phần mềm. Chủ yếu các độ đo của phần mềm chỉ chủ yếu tập trung vào đo lường chất lượng mã nguồn. Trong bài học này sinh viên sẽ được giới thiệu về các độ đo cơ bản dùng để đánh giá mã nguồn như Line of code, các độ đo của Halstead,...

Để đảm bảo được một dự án được thực hiện thành công chúng ta cần phải hiểu được:

- Phạm vi công việc cần được thực hiện
- Các rủi ro có thể có trong dự án
- Các nguồn tài nguyên cần thiết
- Các tác vụ công việc cần được hoàn thành
- Các cột mốc cần được theo dõi

- Chi phí cần thiết
- Kế hoạch cần phải tuân theo

Trước khi một dự án được lên kế hoạch, các mục tiêu và phạm vi của dự án cần được đưa ra, các giải pháp thay thế cần được xem xét, và các ràng buộc về kỹ thuật cũng như quản lý cần được xác định. Những thông tin này là cần thiết để có thể đánh giá chi phí, khối lượng công việc và xây dựng kế hoạch cụ thể cho việc thực hiện.

Các độ đo giúp chúng ta hiểu được quy trình kỹ thuật được sử dụng khi phát triển một sản phẩm. Quy trình cần được đo lường để có thể cải tiến và sản phẩm cũng cần được đo lường để nâng cao chất lượng.

Việc đo lường các dự án phần mềm vẫn còn đang gây tranh cãi vì:

- Cách đo lường phần mềm vẫn chưa rõ ràng và còn phụ thuộc cụ thể vào từng dự án.
- Các độ đo cũng chưa thực sự rõ ràng, thường phụ thuộc vào con người, quy trình và sản phẩm, trong khi các yếu tố này lại cần được so sánh dựa trên chính những độ đo đó.

Tại sao chúng ta cần các độ đo phần mềm?

Nếu không có các độ đo thì sẽ không có cách nào để xác định được một quy trình có cần được cải tiến hay không. Các độ đo cho phép thiết lập những mục tiêu có ý nghĩa nhằm cải tiến chất lượng sản phẩm, đồng thời giúp một tổ chức xác định được các nguyên nhân của những lỗi có ảnh hưởng lớn nhất đến quá trình phát triển phần mềm.

Khi các độ đo được áp dụng vào một sản phẩm chúng cho phép xác định: Những yêu cầu nào của người dùng có khả năng sẽ bị thay đổi hoặc cần được thay đổi. Những mô đun nào có khả năng lỗi nhiều nhất. Những cách kiểm thử như thế nào cần được lên kế hoạch cho từng mô đun.

Phân loại độ đo phần mềm

Các độ đo của phần mềm thì thường được phân vào hai loại: các độ đo trực tiếp và các độ đo gián tiếp.

- **Các độ đo trực tiếp** thông thường bắt nguồn từ một tính năng hoặc một đặc tính cụ thể ví dụ như chiều dài code. Các độ đo trực tiếp hay dùng để đánh giá mã nguồn gồm: dòng code, tốc độ thực thi, kích thước bộ nhớ yêu cầu, thông báo lỗi

- **Các độ đo gián tiếp** liên kết một độ đo với một mục tiêu đo đặc cụ thể như chất lượng dựa trên các thất bại đếm được của dự án. Thông thường các mục tiêu cụ thể có thể là: chức năng, chất lượng, độ phức tạp, tính hiệu quả, độ tin cậy, khả năng bảo trì được.

2.2 Độ đo độ phức tạp mã nguồn

2.2.1 Động cơ

Độ phức tạp mã nguồn có tương tác trực tiếp đến tỉ lệ lỗi và độ mạnh của chương trình ứng dụng. Các công cụ để đo đặc độ phức tạp mã nguồn:

- Testwell CMT++ để đánh giá độ phức tạp mã nguồn C/C++ hoặc C#
- Testwell CMTJava cho mã nguồn Java

Mã nguồn có độ phức tạp tốt thì sẽ chứa ít lỗi, dễ hiểu và dễ bảo trì hơn những mã nguồn có độ phức tạp lớn. Các độ đo về độ phức tạp mã nguồn thường được sử dụng để định vị những khu vực mã nguồn phức tạp. Để đạt được phần mềm với chất lượng tốt và giảm thiểu chi phí kiểm thử và bảo trì thì độ phức tạp mã nguồn cần được ước tính càng sớm càng tốt trong quá trình viết mã người phát triển cần sửa chữa lại code của mình khi những giá trị khuyến dùng về độ phức tạp mã nguồn của mình bị vượt quá. Chi tiết của phần này sẽ được giải thích kĩ hơn trong phía sau của bài học.

2.2.2 Phân loại

Các độ đo được sử dụng bởi phần mềm Testwell CMT++ và CMTJava gồm có:

- Số dòng code
- Độ phức tạp McCabe
- Độ đo Halstead
- Chỉ số bảo trì MI

Lines of code metrics - Số dòng code

Số dòng trong mã nguồn là độ đo cơ bản nhất được sử dụng để tính toán độ phức tạp phần mềm. Là một độ đo rất đơn giản, dễ tính toán và rất dễ dùng. Một

chương trình có số dòng code lớn thì đơn giản là phức tạp hơn chương trình đó nhưng chiều dài ngắn hơn. Tuy nhiên độ đo này cũng mang tính tương đối vì nhiều khi viết một chương trình dễ hiểu dễ đọc sẽ dài hơn là viết một mã nguồn ngắn gọn súc tích. Số đo này khi tính toán sẽ không tính đến layout của mã nguồn (các cấu trúc xuống dòng hoặc định dạng mã nguồn).

Testwell CMT++/CMTJava tính toán các số đo sau liên quan đến số dòng code:

- LOCphy: số dòng vật lý (sẽ bao gồm các dòng trống, dòng comment, dòng code)
- LOCbl: số dòng trống – một dòng trống trong khu vực comment sẽ được coi như một dòng comment
- LOCpro: số dòng mã nguồn chương trình (khai báo, định nghĩa và các dòng code)
- LOCcom: số dòng comment trong mã nguồn chương trình

Sau đây là những giá trị khuyến cáo cho độ đo chiều dài mã nguồn đối với chiều dài một hàm:

- Chiều dài hàm cần trong khoảng từ 4 đến 40 dòng code
- Mỗi định nghĩa hàm thì chứa ít nhất một khuôn mẫu hàm, 1 dòng code, 1 cặp ngoặc nhọn xác định thân hàm như vậy là cần tối thiểu 4 dòng
- Một hàm có chiều dài lớn hơn 40 dòng thì có nhiều khả năng là nó là gộp của hơn hai hàm và có thể tách được thành các hàm nhỏ hơn để giảm thiểu chiều dài của hàm
- Ngoại lệ: hàm chứa 1 câu lệnh lựa chọn (ví dụ câu lệnh switch – case trong C/C++) với nhiều nhánh lựa chọn (ví dụ 100 nhánh sẽ làm thân hàm có thể hơn 104 dòng code). **Câu hỏi:** Hỏi sinh viên trong trường hợp này nếu chúng ta tách thành hàm nhỏ thì sẽ ảnh hưởng đến các tiêu chí chất lượng nào?

Trong trường hợp này nếu chúng ta tách hàm thành các hàm nhỏ hơn sẽ làm giảm tính dễ đọc dễ hiểu của hàm làm tăng khả năng sinh lỗi của hàm, làm giảm khả năng bảo trì sửa đổi được của hàm và làm tăng chi phí kiểm thử của hàm.

- Về chiều dài của một file mã nguồn:

- Một file nên có chiều dài trong khoảng từ 4 đến 400 dòng code tức là khoảng 10 hàm.
- Phần tử nhỏ nhất của một file mã nguồn có thể là 1 hàm và chiều dài tối thiểu của nó do đó là 4 dòng code.
- Những file mã nguồn có chiều dài lớn hơn 400 dòng code thì quá dài để hiểu được toàn bộ ý nghĩa của file đó. Do đó nên được tách ra thành các file mã nguồn khác nhau (ví dụ nhóm các hàm lại thành các nhóm chức năng khác nhau).
- Về các chú thích trong mã nguồn, ít nhất từ 30% đến 75% dòng của một file mã nguồn nên là các chú giải để đảm bảo tính dễ hiểu của mã nguồn. Nếu ít hơn 1/3 của một file là comment thì hoặc là file mã nguồn đó cực kỳ tầm thường hoặc là file mã nguồn đó không được giải thích tốt. Nếu nhiều hơn 3/4 của một file là comment thì file đó không phải là mã nguồn mà là một tài liệu đặc tả. Ngoại lệ: Trong một file đặc tả về định dạng file ví dụ headers của một file .mp3, tỉ lệ % comment có thể vượt quá 75%.

Các độ đo Halstead

Các độ đo Halstead được phát triển bởi Maurice Halstead, và được giới thiệu vào năm 1977 và được sử dụng trong thực tế rất nhiều kể từ khi ra đời. Các độ đo Halstead được xem là những phép đo cơ bản nhất về độ phức tạp của chương trình/mã nguồn. Đây là một trong số những chỉ số rất mạnh để chỉ ra độ phức tạp mã nguồn và thường được sử dụng như là một phép đo trong bảo trì mã nguồn. Các độ đo Halstead thì được tính toán dựa trên việc phân tích mã nguồn thành các token (các thẻ) và phân loại các thẻ đó thành hai loại: toán tử và toán hạng.

Chúng ta sẽ tính toán các giá trị độ đo dựa trên các thẻ đã phân loại đó bằng cách tính:

- Số các toán tử phân biệt kí hiệu là n_1
- Số các toán hạng phân biệt kí hiệu là n_2
- Tổng số các toán tử kí hiệu là N_1
- Tổng số các toán hạng kí hiệu là N_2

Tất cả các số đo khác của Halstead thì đều được tính toán dựa trên 4 giá trị cơ bản này và sẽ được mô tả thông qua các công thức trong phần tiếp theo.

Operators – Toán tử

- traditional: +, -, *, /, ++, --, etc.
- keywords: return, if, continue, break, try, catch etc.

Operands – Toán hạng

- Identifiers: tên biến, tên hàm, tên file,...
- constants

Chúng ta sẽ quan tâm đến các độ đo cơ bản sau của Halstead khi đánh giá độ phức tạp của chương trình. Các số đo này thì đều tính toán từ 4 giá trị cơ bản đã nói ở phần trước.

- Chiều dài chương trình (N) là tổng số các toán tử và toán hạng có trong chương trình $N = N1 + N2$
- Kích thước từ vựng chương trình là tổng số các toán tử và toán hạng phân biệt có trong chương trình $n = n1 + n2$
- Kích thước chương trình V là nội dung thông tin của chương trình được tính theo công thức $V = N * \log_2(n)$
- Kích thước chương trình của Halstead là một số đo cho phép miêu tả kích thước cài đặt của một chương trình hay một thuật toán

Một số công cụ ví dụ như Testwell sẽ tính toán các độ đo của Halstead và đưa ra các khuyến cáo cho người phát triển về chất lượng mã nguồn của mình. Kích thước của một hàm nên có giá trị volume halstead từ 20 đến 1000.

- Một hàm chỉ có một dòng và không có tham số thì vào khoảng 20.
- Một hàm có kích thước volume hơn 1000 thì về cơ bản có thể làm quá nhiều việc trong hàm và chúng ta có thể nên tách hàm đó thành các hàm nhỏ.
- Kích thước của một file mã nguồn nên từ 100 đến 8000
- Giá trị volume đo được của một file mã nguồn $V(G)$ và chỉ số LOCpro nên nằm trong khoảng giới hạn quy định của nó

Ngoài kích thước chương trình Program Volume thì Halstead cũng đề nghị một số độ đo khác ví dụ như:

- **Độ khó của chương trình D:** được coi như là mức độ tiềm ẩn lỗi của chương trình thì tỉ lệ với số các toán tử phân biệt của chương trình D cũng được coi như tỉ lệ với tỉ số giữa tổng số toán hạng và số toán hạng phân biệt của chương trình

Ví dụ như, cùng một toán hạng những được sử dụng quá nhiều lần trong chương trình thì khả năng tiềm ẩn lỗi sẽ cao hơn và do đó cần được sửa đổi mã nguồn để làm giảm số lần cùng một toán hạng được sử dụng trong chương trình.

Công thức tính D là $D = (n1/2) * (N2/n2)$

- **Mức độ của chương trình L:** Mức độ của chương trình là số đo có giá trị là đảo ngược của mức độ tiềm ẩn lỗi D Ví dụ, một chương trình ở mức độ thấp thì sẽ nhiều khả năng tiềm ẩn lỗi hơn là một chương trình ở mức độ cao

Công thức tính L là $L = 1/D$

Một số độ đo khác của Halstead như:

- **Công sức cài đặt chương trình (E)** là công sức phải bỏ ra để cài đặt mã nguồn chương trình hoặc để đọc hiểu mã nguồn chương trình – Độ đo này tỉ lệ với kích thước và độ khó của chương trình $E = V * D$
- **Thời gian để cài đặt chương trình (T)** là thời gian cần thiết để cài đặt hoặc để hiểu được mã nguồn của chương trình. Halstead đã tìm ra rằng chia công sức cài đặt E cho 18 sẽ cho một giá trị xấp xỉ về thời gian cài đặt tính theo giây. $T = E/18$
- **Số lượng lỗi khi bàn giao (B)** cũng là một trong số những chỉ số độ phức tạp Halstead. Chỉ số này thì tương quan với độ phức tạp chung của phần mềm và được tính bằng công thức $B = (E^{2/3})/3000$. Chỉ số này ước tính số lỗi có thể có của chương trình Số lượng lỗi của một file mã nguồn nên nhỏ hơn 2. Các kinh nghiệm thực tế đã chỉ ra rằng khi lập trình với C hoặc C++, một file mã nguồn luôn luôn có nhiều lỗi tiềm ẩn hơn chỉ số B gợi ý của mã nguồn B là một trong số những chỉ số rất quan trọng cho kiểm thử tự động. Số lượng lỗi bàn giao cũng xấp xỉ số lượng lỗi thực tế của một module. Mục đích khi kiểm thử (nhất là kiểm thử tự động) là phải tìm được ít nhất B lỗi trong mã nguồn.

Ví dụ: Cho 1 đoạn mã nguồn viết bằng C, hãy tính các chỉ số cơ bản của Halstead $n1, n2, N1, N2$

```

void sort ( int *a, int n ) {
    int i, j, t;

    if ( n < 2 ) return;
    for ( i=0 ; i < n-1; i++ ) {
        for ( j=i+1 ; j < n ; j++ ) {
            if ( a[i] > a[j] ) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
    }
}

```

V = 80 log₂(24) □ 392

Hình 2.1: Mã nguồn chương trình C

Với một đoạn mã nguồn đầu vào, để tính được các chỉ số Halstead cần xác định trong mã nguồn các tokens thuộc về toán tử và toán hạng. Tất cả các biến và các giá trị đều là các toán hạng Toán tử bao gồm các từ khoá, các kiểu dữ liệu... Sau khi tính toán có thể kẻ bảng liệt kê như trong đáp án và đếm số toán tử / toán hạng tổng số và phân biệt để tính toán các chỉ số Halstead như V, D, E...

Độ phức tạp McCabe

Chỉ số phức tạp $v(G)$ được đưa ra bởi Thomas McCabe năm 1976 – là các số đo được tính toán dựa trên các đường độc lập tuyến tính trong một chương trình. Chỉ số phức tạp McCabe được tính toán dựa trên đồ thị luồng điều khiển của chương trình. Đây là một trong số các số đo được sử dụng rộng rãi trong công nghiệp phần mềm để tính toán được độ phức tạp của chương trình/mã nguồn.

$v(G)$ được tính bằng số các nhánh điều kiện trong đồ thị luồng điều khiển của chương trình. $v(G)$ sẽ bằng 1 đối với các chương trình không có lệnh rẽ nhánh chỉ gồm các câu lệnh tuần tự. Đối với một hàm đơn, $v(G)$ sẽ luôn nhỏ hơn các điểm rẽ nhánh trong chương trình. Chỉ số phức tạp của chương trình càng lớn, càng nhiều nhánh cần thực thi trong chương trình và độ khó của chương trình sẽ càng cao.

Công thức tính chỉ số McCabe: Chỉ số độ phức tạp McCabe thể hiện độ phức tạp cấu trúc của chương trình (luồng điều khiển và luồng dữ liệu) Là các độ đo dựa trên đồ thị: số lượng đỉnh, số lượng cạnh, chiều sâu của đồ thị

Một cách tổng quát: $v(G)$ sẽ được tính bằng tổng số cạnh – tổng số đỉnh + 2.

Có thể tính $v(G)$ bằng cách đếm các nhánh rẽ trong đồ thị hoặc đếm câu lệnh IF hoặc số vòng lặp trong đồ thị.

Ý nghĩa của chỉ số McCabe

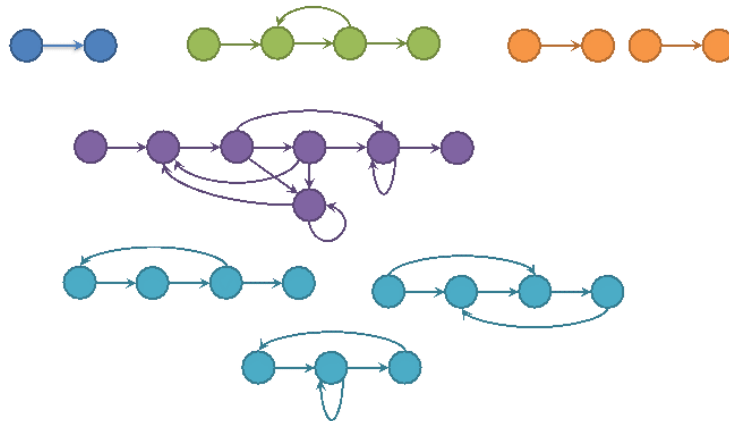
Đối với kiểm thử tự động, chỉ số McCabe là một chỉ số rất quan trọng để ước lượng số các trường hợp kiểm thử cần thiết đối với một function/module

Độ phức tạp McCabe biểu diễn độ phức tạp luồng điều khiển chương trình. Chúng ta có thể rõ ràng thấy rằng các module/hàm có chỉ số McCabe cao hơn thì sẽ cần nhiều ca kiểm thử hơn các module/hàm có chỉ số McCabe nhỏ hơn.

Rõ ràng là mỗi một hàm cần có số lượng test case tối thiểu bằng với chỉ số McCabe của hàm đó. Nói một cách khác, với mỗi một hàm chúng ta phải xây dựng các trường hợp kiểm thử sao cho có thể kiểm thử tất cả các nhánh của hàm.

Ví dụ 1: Yêu cầu sinh viên đếm số lệnh IFs và LOOFs cho mã nguồn chương trình C 2.1, sau đó tính chỉ số McCabe.

Ví dụ 2: Sinh viên sẽ tính toán số các đỉnh và số các cạnh từ đó đưa ra giá trị của chỉ số McCabe cho CFG – Control Flow Graph.



Hình 2.2: Đồ thị CFG

Ví dụ 3: Cho mã nguồn chương trình, yêu cầu sinh viên vẽ sơ đồ luồng điều khiển và tính toán độ phức tạp McCabe của chương trình.

```
1 procedure SpeakingTimeToday (in listOfCalls; out speakingTime);
2 begin
3   callNum = 0
4   speakingTime = 0
5   while (callNum < listOfCalls.Length)
6     if listOfCalls[callNum].date = today
7       speakingTime = speakingTime + listOfCalls[callNum].time
8     end
9     callNum = callNum + 1
10  end
11 end
```

Hình 2.3: Mã nguồn

Một số khuyến cáo đối với chỉ số McCabe – được sử dụng như một lời gợi ý ở các chương trình tối ưu mã nguồn:

- Chỉ số McCabe của một hàm số nên nhỏ hơn 15. Khi một hàm có chỉ số McCabe = 15 ít nhất có 15 nhánh rẽ trong chương trình.
- Những chương trình nhiều hơn 15 nhánh rẽ sẽ khó để hiểu và khó để kiểm thử.
- Chỉ số McCabe của một file mã nguồn nên nhỏ hơn 100.

2.2.3 Chỉ số bảo trì

Chỉ số bảo trì được của chương trình được tính toán dựa trên các độ đo về số dòng mã nguồn, chỉ số McCabe và các độ đo của Halstead. Chỉ số này chỉ ra liệu rằng việc viết lại mã nguồn chương trình có rẻ hơn hoặc rủi ro hơn việc thay đổi nó không?

Có hai phiên bản khác nhau của MI:

- MI có tính toán số dòng comment
- MI không tính số dòng comment

Thực tế có 3 loại số đo khác nhau đối với chỉ số này:

- MIwoc: chỉ số không có comment
- MIcw: chỉ số comment
- $MI = MIwoc + MIcw$

Công thức tính chỉ số MI

$$MI_{woc} = 171 - 5.2 \cdot \ln(\text{aveV}) - 0.23 \cdot \text{aveG} - 16.2 \cdot \ln(\text{aveLOC})$$

$$MI_{cw} = 50 \cdot \sin\left(\sqrt{2.4 \cdot \text{perCM}}\right)$$

Trong đó:

- **aveV**: giá trị trung bình của Halstead Volume V trên mỗi mô-đun.
- **aveG**: giá trị trung bình của độ phức tạp chu trình mở rộng $v(G)$ trên mỗi mô-đun.
- **aveLOC**: số dòng lệnh vật lý (LOC_{phy}) trung bình trên mỗi mô-đun.
- **perCM**: phần trăm số dòng chú thích trung bình trên mỗi mô-đun.

Các khuyến cáo đối với chỉ số MI: Chỉ số MI của một chương trình với comment nếu:

- lớn hơn 85: khả năng thay đổi và bảo trì tốt
- từ 65 đến 85: khả năng bảo trì trung bình
- <65: khó để bảo trì và thực sự chứa các đoạn mã nguồn không tốt

Các đoạn mã nguồn lớn, không được chú thích hoặc không được cấu trúc hoá có thể dẫn tới các giá trị MI nhỏ hơn 0. Những đoạn chương trình đó rất cần được cải thiện để nâng cao giá trị MI.

Bài tập: Yêu cầu sinh viên:

- Vẽ đồ thị luồng điều khiển (Control Flow Graph).
- Tính độ phức tạp McCabe Cyclomatic.
- Tính chỉ số MI.

```
1 public class BinarySearch {  
2     public int binsearch(int x, int[] V, int n) {  
3         int low, high, mid;  
4         low = 0;  
5         high = n - 1;  
6         while (low <= high) {  
7             mid = (low + high) / 2;  
8             if (x < V[mid])  
9                 high = mid - 1;  
10            else if (x > V[mid])  
11                low = mid + 1;  
12            else  
13                return mid;  
14        }  
15        return -1;  
16    }  
17 }
```

Hình 2.4: Mã nguồn chương trình

2.3 Độ đo hướng đối tượng

Phần cuối của bài học sẽ tập trung vào giới thiệu các chỉ số đo đặc thường sử dụng đối với hướng tiếp cận hướng đối tượng khi phát triển phần mềm. Các độ đo đã học là các độ đo chung, cơ bản và có thể áp dụng đối với mọi loại chương trình: lập trình thủ tục, lập trình hướng chức năng/hàm và cả lập trình hướng đối tượng. Tuy nhiên đối với hướng phát triển phần mềm hướng đối tượng, chúng ta đặc biệt quan tâm đến tính mô đun hoá chương trình. Vì vậy một số độ đo được đưa ra để tính toán riêng cho các chương trình/phần mềm như vậy.

Mục đích của độ đo hướng đối tượng

- Hiểu rõ hơn về chất lượng sản phẩm phần mềm
- Đánh giá tính hiệu quả của quy trình
- Cải tiến chất lượng ở 1 phase bất kỳ của dự án

Các đặc điểm của độ đo hướng đối tượng Các độ đo này cần phải cho phép thể hiện được sự khác biệt của các chương trình HĐT với các hệ thống khác.

5 nguyên lý của OOP được sử dụng để xây dựng các độ đo:

- Localization: thể hiện việc tập trung thông tin trong 1 chương trình Trong ngữ cảnh của OOP, localization thể hiện cả dữ liệu và logic nghiệp vụ đều tập trung trong phạm vi của 1 class Vì class là cơ sở trong OOP nên localization

thể hiện việc xoay quanh các đối tượng của class cả về thông tin (data) lẫn nghiệp vụ (methods)

- Encapsulation: thể hiện việc đóng gói (package) các class, các đối tượng với nhau. Đối với các chương trình cấu trúc, tính đóng gói có thể được thể hiện qua các cấu trúc (struct) và mảng (collection/array). Đối với các chương trình OOP, tính đóng gói thể hiện các trách nhiệm của 1 class được gắn liền với các thuộc tính, các hành vi của lớp đó.
- Information hiding: thể hiện việc ẩn đi các chi tiết của việc cài đặt 1 class. Chỉ những thông tin cần thiết mới được cho phép để các đối tượng hoặc thành phần bên ngoài truy cập mới được. Một hệ thống được thiết kế tốt cần phải làm tốt việc che giấu thông tin. Information Hiding = chỉ số cho chất lượng của các hệ thống hướng đối tượng.
- Inheritance: Kế thừa là một cơ chế cho phép các nhiệm vụ của 1 đối tượng có thể được sử dụng lại trong các đối tượng khác. Kế thừa có thể thực hiện ở nhiều cấp. Rất nhiều độ đo OOP được xây dựng dựa trên nguyên lý kế thừa.
- Object abstraction: Tính trừu tượng hoá tập trung vào việc biểu diễn những yếu tố cốt lõi của 1 chương trình/thành phần với ít nhất có thể những chi tiết liên quan đến các tầng thấp (xử lý dữ liệu, xử lý nghiệp vụ...). Trừu tượng hoá là một khái niệm có tính tương đối. Chúng ta có thể có các cấp độ trừu tượng hoá khác nhau. Mức độ trừu tượng của một lớp là một trong các độ đo hướng đối tượng.

Các độ đo hướng đối tượng bao gồm:

- Chidamber and Kemerer (CK) metrics suite
- Lorenz and Kidd (LK) metrics suite
- The Metrics for Object-Oriented Design (MOOD) Metrics Suite

2.3.1 Các độ đo hướng đối tượng HK

Trong kỹ thuật lập trình cũng như trong phát triển phần mềm nói chung, chúng ta thường xuyên nhắc đến tính mô-đun hoá chương trình như một tiêu chí chất lượng để đánh giá chương trình/phần mềm.

Vậy thế nào là tính mô-đun hoá chương trình? Một cách khái quát, mô-đun hoá là khả năng một chương trình có thể chia tách thành các thành phần riêng biệt và

kết hợp lại với nhau nhằm tối đa tính linh hoạt và khả năng tái sử dụng các thành phần. Tính mô đun hoá quan tâm đến hai vấn đề:

- Cohesion: Sự gắn kết trong mô đun
- Coupling: Mối nối giữa các mô đun

Hỏi sinh viên thế nào là một thiết kế tốt? Trả lời: sự gắn kết trong mô đun là lớn và tính lỏng lẻo giữa các mô đun là cao (mối nối giữa các mô đun là thấp).

Chỉ số về tính mô đun hoá

Có hai chỉ số thể hiện tính mô-đun hoá chương trình:

- Chỉ số Fan-in là số lượng các lời gọi hàm bên trong mô-đun thể hiện tính gắn kết bên trong mô-đun. Một mô-đun càng độc lập khi lời gọi hàm bên trong mô-đun càng lớn
- Chỉ số Fan-out là số lượng các lời gọi hàm bên ngoài mô-đun thể hiện độ kết dính giữa các mô đun với nhau. Mô-đun càng độc lập nếu số lượng lời gọi hàm bên ngoài mô-đun càng nhỏ

Chỉ số Henry và Kafura cho phép tính toán tính mô-đun hoá của các chương trình thủ tục sử dụng lập trình thủ tục. Chỉ số HK cho phép tính toán các luồng bên trong chương trình và các luồng bên ngoài chương trình. Với lập trình hướng đối tượng chúng ta có các chỉ số khác cũng dựa trên tính mô đun hoá như: các chỉ số Chidamber & Kemerer.

Các mô-đun có *fan-out* bằng 0 được xem như *deadcode*, tức là nằm bên ngoài phạm vi của hệ thống.

Chỉ số HK của Henry/Kafura cho phép tính mức độ mô-đun hoá của các chương trình thủ tục.

- **fan-in**: luồng thông tin đi vào thủ tục.
- **fan-out**: luồng thông tin đi ra khỏi thủ tục.

$$HK = SLOC \times (fan-in \times fan-out)^2$$

Trong đó *SLOC* là số dòng mã nguồn (không tính các dòng chú thích và dòng trống).

2.3.2 Các độ đo hướng đối tượng CK

Một số chỉ số được đề nghị bởi Chidamber và Kemerer

- Số phương thức trên một lớp – Weighted Methods per Class (WMC).
- Chỉ số phản hồi của một lớp – Response For a Class (RFC).
- Sự thiếu kết dính của các phương thức trong lớp – Lack of Cohesion of Methods (LCOM).
- Sự kết nối giữa các đối tượng – Coupling Between Objects (CBO).
- Độ sâu trên cây kế thừa – Depth of Inheritance Tree (DIT).
- Số lượng các lớp con – Number of Children (NOC).

Phương thức trên một lớp (WMC) tính toán độ phức tạp của một lớp liên quan đến số phương thức của lớp đó hoặc tổng số độ phức tạp của các phương thức trong lớp. Độ phức tạp phương thức được tính thông qua chỉ số McCabe của phương thức đó. Số lượng các phương thức và độ phức tạp của chúng là một chỉ số cho phép xác định thời gian và công sức phải bỏ ra để xây dựng và bảo trì lớp. Số lượng phương thức của một lớp càng lớn thì tiềm năng ảnh hưởng đến các lớp con càng cao. Các lớp có số lượng phương thức càng nhiều thì khả năng tái sử dụng tiềm năng của phương thức có thể sẽ không cao nhưng có thể có những ngoại lệ.

```

Class A
{
Method0(); // complexity0
Method1(); // complexity1
...
Methodn(); // complexityn
}

```

Công thức tính WMC:

$$WMC = \sum_{i=0}^n complexity_{method[i]}$$

Bài tập: Yêu cầu sinh viên tính WMC của class WMPC sau:

```

public class WMPC {
    public void one() {
        if(true) {
            two();
        } else {
        }
        if(true && !false) {
        }
    }
    public void two() {
        if(true) {}
    }
    public void three(int i) {}
}

```

Hình 2.5: Mã nguồn chương trình

Phản hồi của một lớp (RFC) RFC hay chỉ số phản hồi của một lớp là độ đo tính tổng tất cả các phương thức có thể được gọi tới để phản hồi cho một thông điệp tới một đối tượng của lớp hoặc bởi một phương thức nào đó của lớp. Chỉ số này cũng tính cho tất cả các phương thức truy cập được trên cây kế thừa của lớp. Một cách tổng quát thì chỉ số RFC xem xét độ phức tạp của một lớp thông qua số lượng phương thức và số tương tác/giao tiếp với các lớp khác. Số lượng các phương thức được gọi từ một lớp thông qua các thông điệp từ một đối tượng càng lớn thì độ phức tạp của lớp càng lớn. Nếu một lượng lớn các phương thức có thể được gọi để hồi đáp cho một thông điệp thì việc kiểm thử và sửa lỗi lớp trở nên càng phức tạp bởi vì nó yêu cầu một mức độ hiểu biết lớn hơn về lớp và các trường hợp kiểm thử cần tiến hành.

Để tính được RFC:

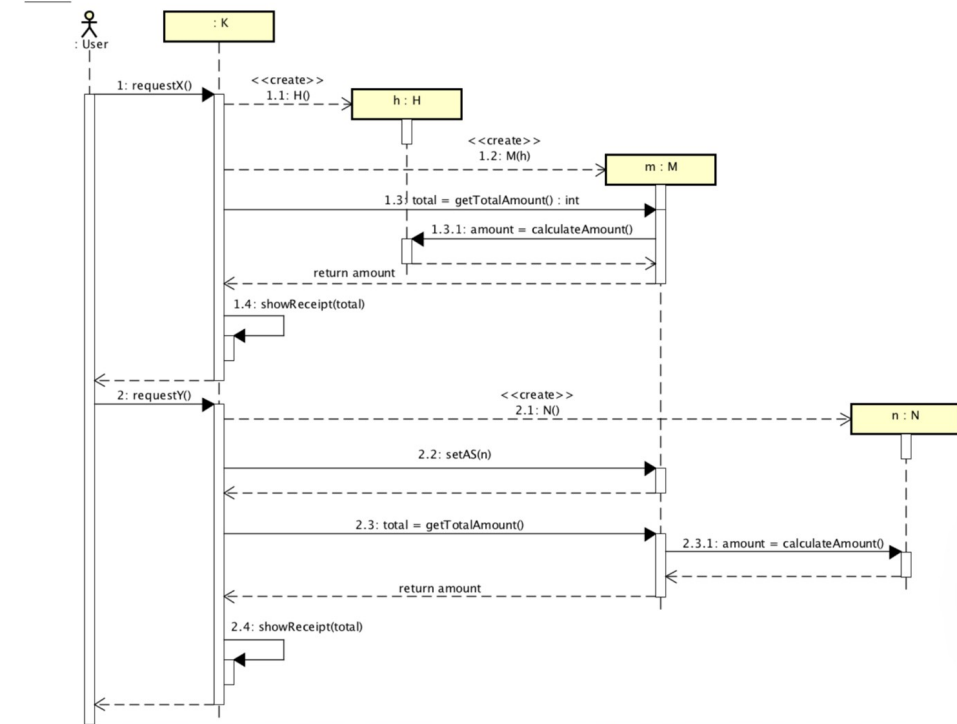
- Xác định tất cả các phương thức ở bên ngoài được gọi bởi các methods bên trong class.
- Tạo một tập hợp là hợp của tập hợp các method bên ngoài và các method của class gọi là RS.
- RFC được tính bằng kích thước của RS.

$$RS = Mc \cup Me, RFC = |RS|$$

Trong đó: Mc tập hợp các method trong class, Me tập hợp các method bên ngoài class được gọi bởi các method bên trong class

Bài tập: Yêu cầu sinh viên tính chỉ số RS cho biểu đồ trình tự sau:

Biểu đồ trình tự 1



Hình 2.6: Biểu đồ trình tự

RFC có thể được coi như một độ đo mức độ kết dính và liên kết của class. High RFC có thể biểu hiện tính coupling cao. Low RFC có thể chỉ tính coupling thấp. High RFC có thể dẫn tới:

- Yêu cầu nhiều test cho class
- Việc hiểu rõ class có thể rất phức tạp
- Tính tái sử dụng kém
- Cần nhiều thời gian và nỗ lực bỏ ra để sửa lỗi

Sự thiếu độ kết dính(LCOM): đo lường sự không tương tự/sự khác nhau của các phương thức trong một lớp dựa trên các biến instance và các thuộc tính. Một module có tính kết dính cao có thể tồn tại một cách độc lập. Độ kết dính cao

thể hiện: sự phân chia lớp tốt, tính đơn giản và tính tái sử dụng cao. Đây là những phẩm chất quan trọng trong thiết kế lớp để làm tăng chất lượng phần mềm (tính tái sử dụng và tính bảo trì kiểm thử được cao)

LCOM được tính bằng công thức

$$LCOM(C) = P - Q$$

nếu $P > Q$ và $= 0$ nếu ngược lại.

Trong đó C là lớp cần tính, P là tổng tất cả các cặp phương thức phân biệt trong lớp C mà các phương thức này không chia sẻ các biến instance. Q là tổng tất cả các cặp phương thức phân biệt trong lớp C mà các phương thức này có chia sẻ các biến với nhau.

Sự thiếu độ kết dính hay độ kết dính thấp sẽ làm tăng độ phức tạp của lớp do đó làm tăng khả năng có thể xuất hiện lỗi trong quá trình phát triển phần mềm. Các lớp có độ kết dính thấp thì có thể chia thành 2 hoặc 3 lớp con để làm tăng tính kết dính trong lớp lên do đó làm tăng tính độc lập của chúng.

Sự kết nối giữa các đối tượng của các lớp (CBO): CBO hay sự kết nối/khớp nối hay tính lỏng lẻo giữa các đối tượng của các lớp là một độ đo tính số lượng các lớp có kết nối với lớp được tính. Kết nối ở đây được hiểu là số lượng các lớp có liên quan không có trên cây kế thừa của lớp được tính mà lớp đó phụ thuộc vào. Sự phụ thuộc ở đây có thể hiểu đơn giản là trong lớp được tính, có gọi đến một phương thức của lớp khác, hoặc trong lớp được tính có một instance object của lớp khác... Bất kì sự liên quan nào đến một lớp khác của lớp được tính cũng được coi như một sự phụ thuộc vào bên ngoài.

Khi sự liên kết với các lớp khác là quá lớn thì nó không tốt cho thiết kế kiểu mô-đun hoá hay cho tính mô-đun hoá chương trình và ngăn ngừa việc tái sử dụng lại mã nguồn. Một lớp càng độc lập (độ phụ thuộc vào lớp khác càng ít) thì nó càng dễ tái sử dụng trong các ứng dụng khác nhau.

Số lượng các mối liên kết càng lớn thì mức độ nhạy cảm với các thay đổi ở các phần khác nhau của thiết kế càng cao do đó càng khó bảo trì. Sự phụ thuộc càng cao thì sẽ làm cho hệ thống càng phức tạp vì một lớp có độ phụ thuộc cao thì sẽ khó hiểu, khó thay đổi và khó sửa chữa. Điều này rất dễ hiểu. Khi một lớp lệ thuộc vào nhiều lớp khác thì rõ ràng khi một trong số lớp đó thay đổi sẽ làm cho lớp lệ thuộc cũng thay đổi theo và khi muốn thay đổi lớp đó thì cũng phải xem xét đến các lớp khác mà nó phụ thuộc.

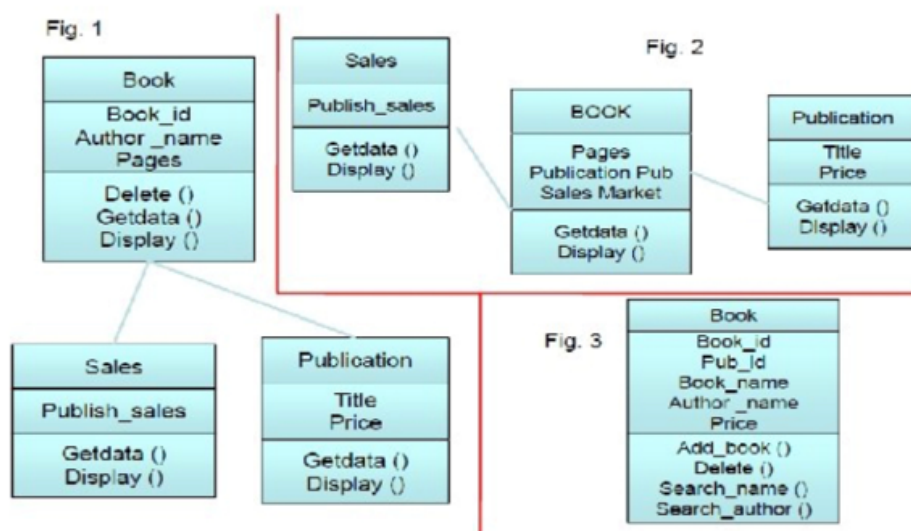
Độ phức tạp có thể giảm bằng cách thiết kế các hệ thống với khả năng tính lỏng lẻo cao giữa các lớp. Nó làm tăng tính mô-đun hoá chương trình và thúc đẩy tính

đóng gói cao trong thiết kế.

Độ sâu cây kế thừa DIT: Độ sâu của một lớp trên cây kế thừa được tính bằng số lượng bước tối đa từ nút lớp đến gốc của cây và được đo bằng số các lớp cha. Một lớp càng nằm sâu hơn trên cây kế thừa thì số lượng phương thức mà lớp đó được thừa kế sẽ càng nhiều làm cho hành vi của lớp càng phức tạp. Những cây kế thừa càng sâu thì độ phức tạp thiết kế càng lớn, càng nhiều lớp và phương thức được bao gồm bên trong nhưng khả năng tái sử dụng các phương thức kế thừa cũng tăng lên.

Số lượng lớp con (NOC): Số lượng lớp con NOC là một chỉ số tính số lượng các lớp con trung gian kế thừa từ một lớp (được tính) trên cây kế thừa. Nếu một lớp có số lượng con càng nhiều, nó sẽ yêu cầu số lượng ca kiểm thử của các phương thức của lớp đó càng nhiều do đó làm tăng thời gian và chi phí kiểm thử của lớp. NOC là một chỉ số về độ ảnh hưởng tiềm ẩn của một lớp có thể có đối với toàn bộ hệ thống. Số lượng lớp con của một lớp càng lớn thì khả năng cao tính trừu tượng hoá hay tính tổng quát hoá của lớp đó càng giảm nhưng số lượng lớp con càng nhiều thì tính tái sử dụng càng cao bởi vì kế thừa là một hình thức của tái sử dụng trong thiết kế hướng đối tượng.

Bài tập: Yêu cầu sinh viên tính WMC, RFC, CBO và LCOM cho ví dụ sau, coi độ phức tạp là 1.



Hình 2.7: Ví dụ với biểu đồ lớp.

2.4 Độ đo gói

Chúng ta đã tìm hiểu một cách tổng quan các khái niệm và các độ đo ở mức thiết kế lớp. Phần tiếp theo sẽ liên quan đến thiết kế ở mức gói (package) và các độ đo liên quan.

Các độ đo ở mức gói được đề nghị bởi Robert Cecil Martin (hay còn gọi là Uncle Bob) một trong những cha đẻ của các lý thuyết, phương pháp luận trong công nghệ phần mềm. Các độ đo này thì tập trung vào mối quan hệ giữa các lớp trong một gói và giữa các gói với nhau để tính đến sự lệ thuộc và sự độc lập trong thiết kế gói.

Các độ đo của Uncle Bob gồm:

- Ce – khớp nối bên ngoài
- Ca – khớp nối bên trong
- I - độ không ổn định
- A – độ trừu tượng hoá
- D – khoảng cách chuẩn hoá từ trình tự chính

Việc tổ chức code theo các package mang lại hai hiệu quả: kết nối các package với nhau, cho phép chúng ta quan sát thiết kế ở mức độ trừu tượng hoá cao, tập trung các class có liên quan trong cùng 1 package giúp tăng độ kết dính của package và cho phép chúng ta dễ dàng sửa đổi và bảo trì.

2.4.1 Stable-Dependencies Principle

Efferent Coupling (CE) (outgoing dependencies)

CE là độ đo đo mối quan hệ hướng ra ngoài của một lớp với các lớp khác, nó dựa trên các sự phụ thuộc bên ngoài của các lớp trong một gói. Độ đo này tính số lượng các lớp trong một gói phụ thuộc vào các lớp của các gói khác. Chỉ số CE càng cao thể hiện độ không ổn định của một gói vì nó có quá nhiều sự lệ thuộc vào các class ở bên ngoài gói. Do đó việc thay đổi của bất kì lớp nào nằm bên ngoài gói sẽ yêu cầu các thay đổi trên gói đó. Giá trị của Ce đối với một gói nên nằm trong khoảng từ 0 đến 20.

Afferent Coupling (CA) (incoming dependencies)

Một chỉ số khác về độ phụ thuộc giữa các gói là CA – tính toán dựa trên các độ lệ thuộc đến gói - incoming dependencies. Chỉ số này được tính bằng số các lớp

hoặc interface từ những gói khác lệ thuộc vào các lớp trong package được tính. Chỉ số Ca càng cao thể hiện tính ổn định của gói càng lớn. Càng nhiều lớp lệ thuộc vào các lớp nằm trong một gói được tính thể hiện tính tái sử dụng của lớp càng cao. Lớp sẽ không thể bị thay đổi một cách dễ dàng vì xác suất thay đổi lan truyền là rất cao vì có quá nhiều lớp lệ thuộc vào lớp đó. Giá trị thích hợp cho CA của một gói nằm trong khoảng từ 0 đến 500.

Instability (I)

Chỉ số này được sử dụng để đo lường một cách tương đối mức độ nhạy cảm của một lớp đối với thay đổi. I được tính bằng

$$Ce/(Ce + Ca)$$

I có giá trị nằm trong khoảng từ 0 đến 1. Đối với một gói hay một thành phần trong hệ thống phần mềm chúng ta có thể chia thành hai loại:

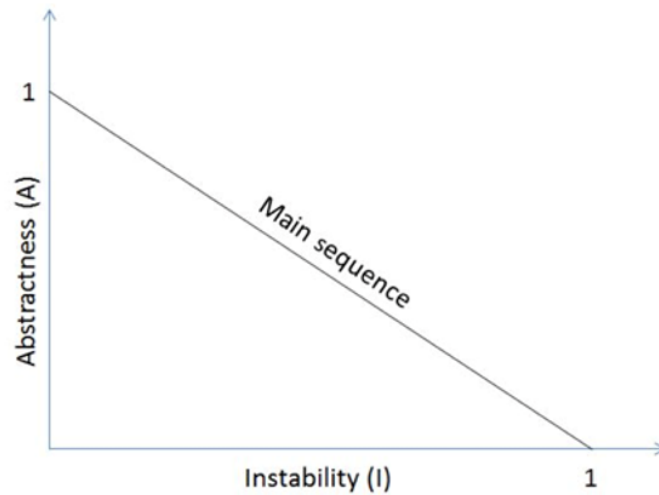
- Những gói hay thành phần có nhiều phụ thuộc đi (Ce cao) mà có không nhiều phụ thuộc đến (Ca thấp), khi đó giá trị của I nằm gần với 1. Những thành phần này ít ổn định do nó lệ thuộc vào nhiều thành phần khác và khi những thành phần này thay đổi thì nó cũng buộc phải thay đổi theo.
- Những gói hay thành phần có nhiều phụ thuộc đến (Ca cao) và có ít phụ thuộc đi (Ce thấp), khi đó giá trị của I nằm gần 0. Những thành phần này khó bị thay đổi hơn do trách nhiệm của nó cao hơn.

Stable-Dependencies Principle: Thay đổi là không tránh khỏi trong quá trình phát triển Một số packages cần phải ổn định Chuyển những phần dễ thay đổi vào các package ở mức cao hơn vì chúng được phép thay đổi Các package ở mức trừu tượng cao Không nên phụ thuộc vào các thành phần ở mức thấp hơn (chi tiết hơn)

2.4.2 Stable Abstraction Principle

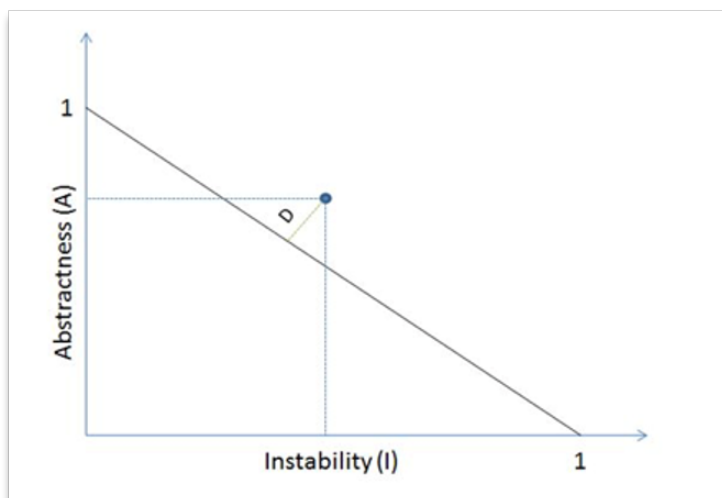
Abstractness (A)

Mức độ trừu tượng hoá A thể hiện mức độ trừu tượng của một gói có phần tương tự với sự bất ổn định gói I. A được tính bằng $Ta/(Ta+Tc)$. Ta là số lượng các lớp trừu tượng trong gói và Tc là số lượng các lớp cụ thể trong gói đó. Một gói càng ổn định (số đo của I càng gần 0) cũng sẽ có chỉ số A cao (A gần 1). Các gói không ổn định (I gần 1) thì sẽ chứa nhiều các lớp cụ thể (A gần 0).

Main Sequence - Đường trình tự chính

Hình 2.8: Đường trình tự chính.

Sự kết hợp của hai chỉ số A và I cho phép xác định sự tồn tại của trình tự chính của hệ thống như trên đồ thị. Các lớp được thiết kế tốt sẽ tập trung quanh đường trình tự chính D.

Normalized distance from main sequence (D)

Hình 2.9: Khoảng cách chuẩn hóa từ đường trình tự chính.

Khoảng cách chuẩn hoá từ đường trình tự chính D là một độ đo cho biết mức độ cân bằng giữa tính trừu tượng và tính ổn định của một lớp được tính theo công thức $D = |A + I - 1|$. Nếu chúng ta đặt một lớp trên đồ thị đường trình tự chính thì khoảng cách từ nó đến đường trình tự chính sẽ tỉ lệ với giá trị của chỉ số D của nó.

Giá trị của D nên càng thấp càng tốt khi đó thành phần tương ứng (lớp, mô-đun, gói) sẽ càng gần với đường trình tự chính.

- Khi $A = 0$ và $I = 0$ thì gói sẽ cực kì ổn định và cụ thể tuy nhiên tình huống đó lại không phải điều chúng ta mong muốn bởi vì gói sẽ rất cứng và khó để có thể mở rộng / thay đổi.
- Khi $A = 1$ và $I = 1$, tình huống này càng không thể xảy ra bởi vì một gói trừu tượng hoàn toàn sẽ vẫn phải có những liên kết nào đó với bên ngoài sao cho các thể hiện (instance object) cài đặt các chức năng hay phương thức định nghĩa trong các lớp trừu tượng có trong gói có thể được tạo ra. Nếu không thì gói chẳng có ý nghĩa sử dụng gì nếu không có bất kì thể hiện nào được tạo ra hay được sử dụng.

Tổng kết: Sinh viên giửu các chỉ số về độ phức tạp mã nguồn, có thể làm các bài tập có liên quan, hiểu được ý tưởng và ý nghĩa của các độ đo trong thiết kế hướng đối tượng, phân tích được ý nghĩa của các độ đo và hiểu được làm thế nào để thay đổi chất lượng thiết kế.

Chương 3

Giới thiệu về kiểm thử phần mềm

3.1	Định nghĩa kiểm thử phần mềm	55
3.2	Các thuật ngữ trong kiểm thử phần mềm	57
3.3	Các hoạt động kiểm thử	61
3.4	Các mức độ kiểm thử	69

3.1 Định nghĩa kiểm thử phần mềm

Kiểm thử phần mềm là một bước quan trọng trong quy trình phát triển phần mềm nhằm đảm bảo chất lượng và độ tin cậy của sản phẩm. Theo định nghĩa chính thức được đưa ra bởi tổ chức IEEE, kiểm thử phần mềm là “một quy trình phân tích một thành phần phần mềm để phát hiện sự khác biệt giữa những gì đang tồn tại và những yêu cầu đặt ra, đồng thời đánh giá các yếu tố của thành phần đó” (các lỗi, sai sót hoặc khiếm khuyết – defects/errors/bugs). Từ định nghĩa này, có thể thấy kiểm thử không đơn thuần là tìm lỗi, mà còn là quá trình so sánh giữa thực tế hoạt động của phần mềm với kỳ vọng hoặc yêu cầu ban đầu.

Một điểm cần đặc biệt lưu ý là: để có thể thực hiện kiểm thử phần mềm một cách hiệu quả, trước hết người kiểm thử cần phải hiểu rõ các yêu cầu chức năng và phi chức năng của thành phần cần kiểm thử, đồng thời phải có khả năng quan sát, theo dõi hoặc đánh giá được trạng thái hiện tại của thành phần đó trong quá trình kiểm thử. Nếu không có thông tin về yêu cầu, hoặc không thể quan sát được kết

quả thực thi, quá trình kiểm thử sẽ trở nên không chính xác hoặc vô nghĩa.

Kiểm thử phần mềm có thể được xem là một quy trình tổng thể nhằm đánh giá một chương trình hoặc hệ thống, với nhiều mục tiêu cụ thể. Trước hết, kiểm thử giúp xác minh (verification) rằng sản phẩm đã được xây dựng đúng với thiết kế kỹ thuật và các yêu cầu kỹ thuật ban đầu. Tiếp theo, nó giúp xác thực (validation) rằng sản phẩm được phát triển phù hợp với mong đợi và yêu cầu thực tế của người dùng. Bên cạnh đó, kiểm thử còn đóng vai trò trong việc đánh giá mức độ chất lượng tổng thể của phần mềm, xét trên các yếu tố như độ ổn định, hiệu năng, khả năng mở rộng và tính bảo mật. Cuối cùng, mục tiêu quan trọng không thể thiếu là phát hiện các lỗi (bugs/defects) để kịp thời khắc phục trước khi sản phẩm được triển khai hoặc đưa vào sử dụng chính thức, qua đó góp phần nâng cao độ tin cậy và hiệu quả vận hành của hệ thống.

- Một cách tổng quát kiểm thử phần mềm là một chu trình thực hiện kiểm chứng rằng sản phẩm phần mềm là đúng với đặc tả phần mềm và xác minh rằng một sản phẩm đúng với mong đợi của người dùng đang được phát triển.
- Kiểm thử phần mềm cũng cho phép đánh giá chất lượng của sản phẩm và góp phần lớn vào việc tìm ra các lỗi trong hệ thống. Tuy nhiên không bao giờ có một phần mềm nào có thể tìm hết lỗi trong giai đoạn kiểm thử vì phần mềm là một đối tượng luôn luôn tiềm ẩn lỗi.

Chi phí và thời gian trong kiểm thử phần mềm

Kiểm thử phần mềm là một giai đoạn tiêu tốn nhiều thời gian và tài nguyên trong quy trình phát triển phần mềm. Nhiều nghiên cứu và báo cáo đã chỉ ra rằng **kiểm thử hồi quy** (*regression testing*) — tức là việc lặp lại kiểm thử sau mỗi lần sửa đổi hoặc cập nhật hệ thống — thường chiếm đến **80% chi phí tài chính dành cho kiểm thử**. Không những thế, kiểm thử còn có thể tiêu tốn tới **50% tổng chi phí bảo trì phần mềm**.

Những con số này phản ánh tầm quan trọng và mức độ đầu tư lớn cho hoạt động kiểm thử trong toàn bộ vòng đời phần mềm, đặc biệt là trong giai đoạn duy trì và nâng cấp hệ thống sau khi triển khai. Do đó, việc tối ưu hóa kiểm thử, đặc biệt là kiểm thử hồi quy, là một yêu cầu cấp thiết nhằm giảm thiểu chi phí và nâng cao hiệu quả phát triển phần mềm.

Một chu trình kiểm thử thường được xem xét ở nhiều góc độ khác nhau. Cụ thể như sau:

- **It does work** - Đảm bảo rằng một đơn vị chương trình (phương thức, lớp,

mô-đun...) hoạt động đúng. Khi cài đặt một chương trình, lập trình viên sẽ cố gắng để kiểm tra xem chương trình đó có chạy đúng trong điều kiện thiết lập hay không? Và khi các thành phần trong hệ thống tích hợp với nhau thì các lập trình viên cũng sẽ cố gắng kiểm tra xem toàn bộ hệ thống có chạy đúng hay không. Ở giai đoạn này, dưới góc độ của người phát triển kiểm thử là nhằm mục đích chỉ ra rằng hệ thống hoạt động đúng với đặc tả của nó.

- **It does not work** - Một khi đội phát triển thoả mãn rằng các chương trình, mô-đun hoặc toàn bộ hệ thống mình viết ra đều chạy đúng, nhiều trường hợp kiểm thử sẽ được tiến hành với mục đích là tìm ra các lỗi có thể có tiềm ẩn trong hệ thống. Lúc này mục đích của kiểm thử là cố gắng để một đơn vị hệ thống bị fail.
- **Reduce the risk of failure** - Giảm thiểu rủi ro thất bại. Hầu hết các hệ thống phức tạp đều chứa lỗi tiềm ẩn, những lỗi này sẽ có thể xảy ra tại một thời điểm nào đó (không thường xuyên). Vấn đề thỉnh thoảng mới xảy ra lỗi này dẫn tới một khái niệm về failure rate – tỉ lệ thất bại của hệ thống. Khi lỗi được phát hiện và được cố định trong một số thiết lập hoặc điều kiện nào đó và tiến hành sửa chữa tỉ lệ thất bại của một hệ thống sẽ giảm đi. Lúc này mục đích của kiểm thử là để làm giảm nguy cơ thất bại ở một mức độ chấp nhận được
- **Reduce the cost of testing** – Giảm thiểu chi phí kiểm thử. Các loại chi phí khác nhau liên quan đến chu trình kiểm thử bao gồm: chi phí thiết kế, bảo trì và thực hiện các trường hợp kiểm thử, chi phí phân tích kết quả của việc thực thi các trường hợp kiểm thử, chi phí làm báo cáo kiểm thử, và chi phí thực thi toàn hệ thống và làm báo cáo kiểm thử toàn hệ thống. Chính vì vậy số lượng ca kiểm thử càng ít thì rõ ràng chi phí cho kiểm thử càng giảm. Tuy nhiên cố gắng giảm thiểu các trường hợp kiểm thử lại không phải là một cách làm tốt để tiết kiệm chi phí. Mục tiêu của kiểm thử lúc này là sản xuất ra các phần mềm có độ rủi ro thấp với càng ít các trường hợp kiểm thử càng tốt. Ý tưởng này đưa chúng ta đến với tính hiệu quả của việc thiết kế các trường hợp kiểm thử.

3.2 Các thuật ngữ trong kiểm thử phần mềm

Trường hợp kiểm thử (hay còn gọi là ca kiểm thử, tiếng Anh: test case) là một đơn vị cơ bản trong quá trình kiểm thử phần mềm. Một cách đơn giản, một test case có thể được hiểu là một cặp gồm dữ liệu đầu vào (input) và kết quả đầu

ra mong đợi (expected outcome). Mục tiêu của test case là kiểm tra xem hệ thống có hoạt động đúng như mong muốn khi nhận một đầu vào cụ thể hay không.

Mỗi test case thường bao gồm nhiều thông tin như: mã định danh (ID), mô tả ngắn gọn về mục đích kiểm thử, các bước thực hiện cụ thể, dữ liệu đầu vào, kết quả mong đợi và trạng thái thực tế sau khi thực hiện (pass/fail). Thông qua việc so sánh kết quả thực tế với kết quả mong đợi, người kiểm thử có thể xác định phần mềm có hoạt động đúng hay tồn tại lỗi cần sửa.

Ví dụ, xét hàm tính căn bậc hai dành cho các số không âm, ta có thể xây dựng một tập hợp các ca kiểm thử như sau:

- Với đầu vào là 0, đầu ra mong đợi là 0 ($<0, 0>$).
- Với đầu vào là 1, đầu ra mong đợi là 1 ($<1, 1>$).
- Với đầu vào là 4, đầu ra mong đợi là 2 ($<4, 2>$).
- Với đầu vào là 9, đầu ra mong đợi là 3 ($<9, 3>$).

Như vậy, trong trường hợp đơn giản này, chỉ cần 4 test case cơ bản để kiểm tra hàm hoạt động đúng trong phạm vi giá trị nhỏ. Tuy nhiên, trong thực tế, việc thiết kế test case thường cần xét thêm các trường hợp đặc biệt (biên, ngoại lệ, dữ liệu không hợp lệ, v.v.) để đảm bảo độ bao phủ và độ tin cậy của quá trình kiểm thử. Một tập hợp đầy đủ các test case sẽ là nền tảng vững chắc giúp phát hiện lỗi sớm và nâng cao chất lượng phần mềm.

Có hai loại hệ thống và tùy thuộc vào từng loại mà các ca kiểm thử có thể đơn giản hoặc phức tạp (một dãy các cặp input, expected value).

- Hệ thống phi trạng thái: ví dụ hàm tính căn bậc 2, rõ ràng là khi thực hiện phép căn bậc 2 của một số, chúng ta không quan tâm đến các số trước đó được tính căn bậc 2 là gì. Tương tự trình biên dịch của ngôn ngữ lập trình C cũng là một loại hệ thống phi trạng thái. Như vậy một ca kiểm thử đơn giản chỉ chứa dữ liệu truyền vào và dữ liệu đầu ra mong đợi.
- Với hệ thống có ghi nhận trạng thái lại không thể đơn giản như vậy, bởi vì việc kiểm thử một dữ liệu đầu vào có thể lệ thuộc vào nhiều dữ liệu khác do đó đầu ra cũng lệ thuộc vào các đầu ra trước đó. Ví dụ kiểm thử chức năng rút tiền của máy ATM có thể là một chuỗi các hoạt động kiểm chứng đầu vào bao gồm kiểm thử xem số tiền có trong tài khoản là bao nhiêu, có đủ cho số tiền muốn rút không, số tiền rút ra có đúng với số tiền mong đợi của khách hàng không và số tiền còn lại trong tài khoản có đúng với mong đợi hay không?

Kết quả mong đợi (Expected Outcome):

Một đầu ra của việc thực thi chương trình có thể chứa:

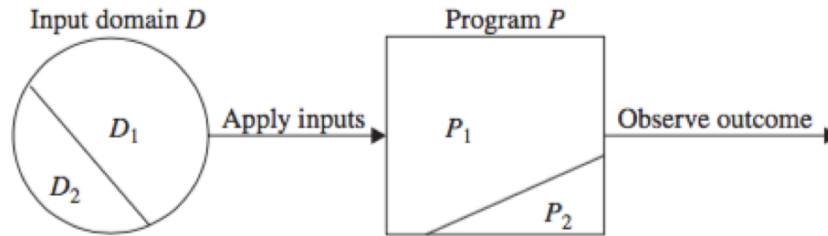
- Dữ liệu đơn giản được sinh ra bởi chương trình (ví dụ: số, chuỗi, hình ảnh, âm thanh...)
- Sự thay đổi trạng thái của hệ thống (ví dụ: sự thay đổi trạng thái của chương trình hay cơ sở dữ liệu)
- Chuỗi các giá trị cần phải quan sát cùng với nhau để đảm bảo rằng đầu ra của chương trình là đúng như mong đợi

Test oracle – là một thực thể (chương trình, quy trình, mong đợi người dùng, dữ liệu...) cho phép cung cấp một đầu ra mong đợi cho một trường hợp kiểm thử cụ thể hoặc một tập hợp các trường hợp kiểm thử. Một ca kiểm thử chỉ có ý nghĩa khi kết quả đầu ra mong đợi thực sự cung cấp bởi chương trình được tiến hành kiểm thử.

Các kết quả đầu ra mong đợi có thể tính toán được khi thiết kế các trường hợp kiểm thử mà không cần thực thi chương trình mà sử dụng các đặc tả yêu cầu của chương trình.

Hoàn tất kiểm thử là quá trình đảm bảo không còn bất kỳ một lỗi tiềm ẩn nào trong hệ thống khi kết thúc pha kiểm thử. Thông thường việc này là không thể thực hiện được vì một số lí do như sau:

- Miền dữ liệu của một chương trình thông thường là quá lớn để có thể hoàn toàn được kiểm tra trong kiểm thử do đó khả năng lỗi tiềm ẩn là luôn luôn có thể do chúng ta không kiểm thử hết toàn bộ dữ liệu đầu vào của chương trình. Ngoài ra dữ liệu đầu vào thì bao gồm cả hai miền: miền dữ liệu hợp lệ và miền dữ liệu không hợp lệ và chúng ta phải kiểm thử cho tất cả các miền này. Hệ thống có thể có một số lượng lớn các hệ thống và có thể có các ràng buộc về thời gian đối với các yếu tố đầu vào tức là một đầu vào có thể hợp lệ tại một thời điểm nhất định và không hợp lệ tại một thời điểm khác. Do đó việc kiểm tra tất cả các trường hợp nhiều khi là bất khả kháng và chúng ta chỉ có thể sử dụng các kĩ thuật để giới hạn các trường hợp kiểm thử.
- Các vấn đề thiết kế có thể quá phức tạp để có thể được thực hiện kiểm thử một cách đầy đủ. Bản thân các thiết kế đã có thể bao gồm các giả định và quyết định ngầm định. Ví dụ một lập trình viên có thể sử dụng một biến toàn cục hoặc một biến tĩnh để kiểm soát việc thực thi chương trình. Với thiết kế



Hình 3.1: Kiểm thử phần mềm

như vậy chúng ta không biết rõ được bên trong chương trình thế nào nên để kiểm chứng toàn bộ vấn đề của thiết kế là bất khả thi

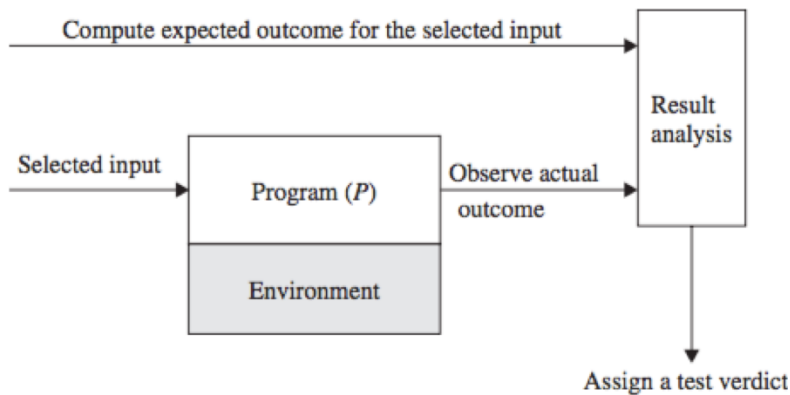
- Không thể tạo ra được tất cả các môi trường thực thi cho hệ thống. Vấn đề này trở nên cực kì quan trọng khi hành vi của hệ thống phụ thuộc vào các yếu tố thực như môi trường, thời tiết, nhiệt độ độ ẩm...

Vấn đề trọng tâm của kiểm thử

Chúng ta đã biết rằng việc kiểm thử một cách hoàn toàn đầy đủ là gần như không thể đối với các hệ thống phần mềm lớn và phức tạp. Việc tốt nhất mà chúng ta có thể làm được khi thiết kế các trường hợp kiểm thử đó là đảm bảo lựa chọn được một tập con đủ tốt của miền dữ liệu đầu vào để thực hiện thiết kế các trường hợp kiểm thử.

Giả sử D là miền dữ liệu đầu vào của chương trình P . Chúng ta lựa chọn được một tập con D_1 của D để test chương trình P với giả thiết rằng tập con D_1 này cho phép chúng ta kiểm thử hành vi thực thi trên miền P_1 của P . Như vậy các lỗi có thể có trên miền P_1 được khám phá hết và các lỗi có thể còn tiềm ẩn trong miền P_2 của chương trình P . Chúng ta nói rằng chúng ta đã cố gắng làm giảm thiểu thuộc tính của chương trình P bằng cách lựa chọn miền dữ liệu con D_1 .

Vấn đề đặt ra là làm sao có thể lựa chọn được một miền con dữ liệu D_1 một cách hệ thống và cẩn thận sao cho việc giảm thiểu hành vi của P là chính xác và đầy đủ nhất có thể. Vì vậy khi thiết kế các trường hợp kiểm thử với kĩ thuật hộp đen và hộp trắng mà chúng ta được học sau này sẽ xuất hiện khái niệm bao phủ (coverage) của miền dữ liệu.



Hình 3.2: Các hoạt động kiểm thử

3.3 Các hoạt động kiểm thử

Để kiểm thử một chương trình, người kiểm thử phải thực hiện một chuỗi các hoạt động kiểm thử. Các hoạt động được mô tả trong phần này tập trung vào giải thích cho một trường hợp kiểm thử đơn giản. Các hoạt động này như sau:

1. Xác định mục tiêu cần kiểm thử: Mục tiêu kiểm thử cho phép xác định động cơ hoặc mục đích của việc sẽ thiết kế một hay nhiều trường hợp kiểm thử để đảm bảo chương trình sẽ hỗ trợ cho mục tiêu kiểm thử đó. Tất cả các ca kiểm thử đều phải gắn với một mục tiêu cụ thể và rõ ràng.
2. Lựa chọn đầu vào: Lựa chọn dữ liệu test đầu vào phải dựa trên đặc tả yêu cầu, mã nguồn và kết quả mong đợi và tất nhiên phải gắn với mục tiêu kiểm thử
3. Tính toán giá trị mong đợi: Bước thứ ba là tính toán các giá trị mong đợi của chương trình với tập dữ liệu đầu vào đã lựa chọn ở bước trên. Nhìn chung việc này có thể thực hiện được thông qua việc hiểu đặc tả yêu cầu của chương trình và mục tiêu kiểm thử.
4. Thiết lập môi trường thực thi của chương trình: Tất cả các giả thiết nằm ngoài của chương trình đều phải thỏa mãn: ví dụ kết nối mạng, kết nối cơ sở dữ liệu... đều cần được thỏa mãn.
5. Thực thi chương trình: kiểm thử viên sẽ thực thi chương trình với dữ liệu đầu vào, và môi trường đã thiết lập và quan sát kết quả thực thi. Để thực thi một ca kiểm thử, dữ liệu đầu vào có thể được cung cấp cho chương trình ở các vị trí vật lý khác nhau ở những thời điểm khác nhau. Khái niệm liên kết kiểm

thử (test coordination) được sử dụng để đồng bộ các thành phần khác nhau của 1 ca kiểm thử

6. Phân tích kết quả kiểm thử: So sánh kết quả thực tế với kết quả mong đợi của ca kiểm thử. Độ phức tạp của việc so sánh thì lệ thuộc vào độ phức tạp của dữ liệu quan sát. Dữ liệu này có thể đơn giản là một số nguyên, 1 chuỗi, 1 hình ảnh hay 1 video... Cuối pha so sánh, các trạng thái kiểm thử sẽ được gán cho ca kiểm thử: pass, fail, inconclusive. Pass là khi chương trình thực thi cho kết quả thực tế giống hệt với kết quả mong đợi. Nếu chương trình không thể cho ra cùng một kết quả như mong đợi thì test bị fail. Thỉnh thoảng không thể đơn giản cứ gán trạng thái pass hay fail cho test case được. Ví dụ nếu hiện tượng timeout xảy ra khi thực thi một ca kiểm thử trên một ứng dụng phân tán chúng ta không thể kết luận test pass hay test fail. Trong trường hợp này thì inconclusive được sử dụng. Inconclusive test case có nghĩa là cần phải thực thi thêm các test khác để có thể kết luận rõ ràng là ca kiểm thử pass hay fail.
7. Cuối ca kiểm thử cần viết báo cáo về trường hợp kiểm thử. Động cơ của việc viết báo cáo là để đảm bảo có được lỗi được sửa chữa nếu như ca kiểm thử bị fail. Báo cáo của ca kiểm thử cần có các thông tin như sau:
 - Giải thích làm thế nào để tái hiện lại lỗi
 - Mô tả lỗi
 - Chỉ ra kết quả mong đợi của hệ thống phải là gì với một mô tả đầy đủ về dữ liệu input, các thiết lập của môi trường và dữ liệu thực tế thu được khi thực thi chương trình

3.3.1 Nguồn dữ liệu sử dụng để thiết kế ca kiểm thử

Mục tiêu của thiết kế ca kiểm thử (test case design) là xây dựng được những ca kiểm thử hiệu quả, có **khả năng phát hiện lỗi** cao, trong khi vẫn **đảm bảo chi phí và thời gian** kiểm thử ở mức thấp nhất có thể. Một ca kiểm thử tốt cần bao phủ đầy đủ các hành vi mong đợi của hệ thống, xử lý các tình huống bất thường, và có khả năng phát hiện sai lệch giữa yêu cầu và thực tế vận hành.

Để thiết kế được các ca kiểm thử như vậy, người kiểm thử cần dựa vào nhiều nguồn thông tin khác nhau. Trước hết, **yêu cầu phần mềm (software requirements)** là tài liệu cơ bản nhất cung cấp thông tin về chức năng và phi chức năng mà hệ thống cần đáp ứng. Thứ hai, **mã nguồn (source code)** của phần mềm, đặc biệt trong kiểm thử mức thấp (unit testing), giúp người kiểm thử hiểu rõ cách triển

khai và xác định các nhánh logic cần kiểm tra. Bên cạnh đó, **đặc tả về miền dữ liệu đầu vào và đầu ra** giúp xây dựng các giá trị kiểm thử đại diện, bao phủ toàn diện các trường hợp hợp lệ và không hợp lệ.

Ngoài ra, **hồ sơ vận hành (operation profile)** – bao gồm thông tin về tần suất sử dụng các chức năng trong thực tế – giúp xác định các khu vực quan trọng cần ưu tiên kiểm thử. Cuối cùng, **mô hình lỗi (fault model)** cung cấp các dạng lỗi phổ biến có thể xảy ra, từ đó hỗ trợ xây dựng các ca kiểm thử có khả năng phát hiện những lỗi thường gặp trong hệ thống.

Việc khai thác kết hợp các nguồn thông tin trên sẽ giúp quy trình thiết kế ca kiểm thử trở nên hệ thống, khoa học và hiệu quả hơn.

3.3.2 Yêu cầu và đặc tả chức năng

Quá trình phát triển phần mềm thường bắt đầu bằng bước thu thập các yêu cầu từ phía khách hàng. Mức độ đầy đủ và chi tiết của các yêu cầu người dùng tại thời điểm khởi đầu dự án phụ thuộc vào mô hình phát triển phần mềm mà nhóm dự án lựa chọn. Ví dụ, **trong mô hình thác nước (Waterfall model)**, nhóm phát triển cố gắng thu thập càng nhiều yêu cầu càng tốt và mô tả chúng một cách rõ ràng ngay từ pha đầu tiên, nhằm tạo ra một bản đặc tả đầy đủ trước khi đi vào các bước tiếp theo. Ngược lại, đối với các mô hình linh hoạt hơn như **mô hình chế thử (Prototyping)**, **mô hình tiến hóa (Evolutionary)** hay **mô hình phát triển nhanh (Rapid Application Development)**, dự án thường bắt đầu chỉ với một tập nhỏ các yêu cầu quan trọng, sau đó **các yêu cầu sẽ được mở rộng, điều chỉnh và phát triển dần dần** trong quá trình triển khai.

Dựa trên các yêu cầu của khách hàng, các ca kiểm thử (test cases) sẽ được thiết kế nhằm đảm bảo rằng phần mềm được phát triển đúng theo mong đợi và phát hiện sớm các sai lệch nếu có. Để thực hiện điều này, các yêu cầu có thể được đặc tả dưới nhiều hình thức khác nhau, tùy vào phương pháp phân tích và thiết kế được áp dụng. Một số hình thức phổ biến bao gồm: mô tả bằng văn bản tự do (text), phương trình toán học, sơ đồ luồng dữ liệu, biểu đồ ca sử dụng (Use Case Diagram), biểu đồ thực thể – quan hệ (ERD), biểu đồ lớp (Class Diagram), hoặc thậm chí là ngôn ngữ hình thức như mô hình máy hữu hạn (Finite State Machine).

Tất cả các dạng đặc tả này đều là nguồn dữ liệu đầu vào quan trọng để phân tích, hiểu và thiết kế ra những ca kiểm thử chất lượng, giúp đảm bảo rằng phần mềm cuối cùng sẽ thỏa mãn các yêu cầu đề ra. Việc lựa chọn và khai thác đúng định dạng đặc tả cũng đóng vai trò quan trọng trong việc nâng cao độ chính xác và hiệu quả của kiểm thử phần mềm.

3.3.3 Mã nguồn

Trong khi đặc tả yêu cầu phần mềm mô tả hành vi dự kiến của hệ thống từ góc nhìn của người sử dụng hoặc khách hàng, thì mã nguồn lại phản ánh hành vi thực tế của hệ thống như được hiện thực bởi lập trình viên. Trong quá trình cài đặt phần mềm, các giả thiết ở mức cao và các ràng buộc được nêu trong yêu cầu sẽ được cụ thể hóa thông qua các cấu trúc lập trình, biểu thức điều kiện, thuật toán và các xử lý chi tiết trong mã nguồn. Tuy nhiên, do lập trình viên có thể áp dụng nhiều cách triển khai khác nhau để biểu diễn cùng một hành vi hoặc cùng một yêu cầu, nên có thể xảy ra sai lệch giữa yêu cầu và thực tế triển khai nếu không có sự kiểm tra kỹ lưỡng.

Chính vì vậy, để đảm bảo chất lượng sản phẩm phần mềm, đặc biệt là tính đúng đắn và nhất quán giữa yêu cầu và hiện thực, việc thiết kế ca kiểm thử dựa trên mã nguồn là cần thiết. Cách tiếp cận này không chỉ giúp kiểm tra hệ thống từ bên trong (kiểm thử hộp trắng), mà còn giúp phát hiện các lỗi logic, lỗi điều kiện biên, lỗi không bao phủ được từ kiểm thử mức yêu cầu. Thông qua đó, quá trình kiểm thử trở nên toàn diện hơn, góp phần nâng cao độ tin cậy của phần mềm trước khi đưa vào sử dụng chính thức.

3.3.4 Miền dữ liệu đầu vào và đầu ra

Miền dữ liệu đầu vào của một chương trình có thể được chia thành hai miền chính: miền dữ liệu hợp lệ và miền dữ liệu không hợp lệ. Miền dữ liệu hợp lệ bao gồm tất cả các giá trị đầu vào mà chương trình được thiết kế để xử lý đúng, trong khi miền dữ liệu không hợp lệ bao gồm những giá trị nằm ngoài phạm vi dự kiến, có thể gây ra lỗi hoặc hành vi không xác định nếu không được xử lý đúng cách.

Ví dụ, xét một module tính giá vé xe buýt dựa trên tuổi của người dùng, trong đó độ tuổi hợp lệ nằm trong khoảng từ 0 đến 99. Khi đó, miền dữ liệu hợp lệ là các giá trị tuổi từ 0 đến 99, còn miền dữ liệu không hợp lệ bao gồm các giá trị nhỏ hơn 0 hoặc lớn hơn 99. Việc kiểm thử cả hai miền dữ liệu này là cần thiết để đảm bảo rằng hệ thống không chỉ hoạt động đúng trong điều kiện bình thường mà còn xử lý được các tình huống ngoại lệ một cách an toàn và hợp lý.

Bên cạnh đó, trong miền dữ liệu đầu vào hợp lệ, một số giá trị đặc biệt có thể mang ý nghĩa riêng biệt và cần được xử lý khác biệt. Ví dụ, tuổi bằng 0 có thể biểu thị cho trẻ sơ sinh và có thể được áp dụng mức giá miễn phí; tuổi bằng 65 trở lên có thể được tính là người cao tuổi và được giảm giá. Những giá trị như vậy cần được xác định rõ trong đặc tả yêu cầu và cần có các ca kiểm thử riêng biệt để đảm bảo

rằng chương trình xử lý đúng các tình huống đặc biệt này.

Do đó, trong thiết kế ca kiểm thử, việc phân tích và phân loại rõ ràng miền dữ liệu đầu vào — bao gồm cả giá trị hợp lệ, không hợp lệ và đặc biệt — là yếu tố then chốt để nâng cao chất lượng và độ tin cậy của phần mềm.

3.3.5 Hồ sơ vận hành

Hồ sơ vận hành (hay còn gọi là hồ sơ sử dụng – operational profile) là một khái niệm quan trọng được giới thiệu lần đầu bởi Mills tại IBM với mục tiêu hỗ trợ phát triển các hệ thống phần mềm có độ tin cậy cao hơn. Hồ sơ vận hành được thiết kế nhằm định hướng cho kiểm thử viên trong việc lựa chọn các ca kiểm thử ưu tiên, tập trung vào những chức năng được sử dụng phổ biến trong thực tế. Thay vì kiểm thử đồng đều toàn bộ hệ thống, hồ sơ vận hành giúp định hướng nguồn lực kiểm thử vào những phần quan trọng nhất, từ đó góp phần nâng cao độ tin cậy tổng thể của phần mềm.

Chẳng hạn, trong một hệ thống ngân hàng (banking system), chức năng quản lý giao dịch thanh toán là một trong những thành phần quan trọng và được sử dụng thường xuyên nhất. Do đó, khi thiết kế các ca kiểm thử, cần ưu tiên tạo ra các tình huống sử dụng cụ thể để kiểm thử chức năng này nhằm đảm bảo hệ thống hoạt động chính xác trong những trường hợp thực tế có khả năng xảy ra cao.

Về bản chất, hồ sơ vận hành là một tập hợp các đặc tả mô tả cách thức hệ thống được sử dụng trong thực tế, bao gồm các kịch bản sử dụng, tần suất truy cập các chức năng và thói quen của người dùng. Dựa vào các kịch bản này, kiểm thử viên sẽ thiết kế các ca kiểm thử tương ứng, qua đó quan sát kết quả kiểm thử và đánh giá được độ tin cậy của hệ thống trong điều kiện sử dụng thực tế.

Một điểm nổi bật trong việc áp dụng hồ sơ vận hành là việc gán xác suất sử dụng cho từng nhóm dữ liệu đầu vào hoặc chức năng, dựa trên thống kê hoặc đánh giá hành vi người dùng. Ví dụ, nếu có 75% người dùng của một hệ thống thư viện điện tử thường xuyên sử dụng chức năng tìm kiếm sách, thì chức năng này sẽ được gán một xác suất là 0,75 trong hồ sơ vận hành. Nhờ đó, các ca kiểm thử liên quan đến chức năng này sẽ được ưu tiên thiết kế và thực hiện nhiều hơn so với các chức năng ít được sử dụng. Việc áp dụng hồ sơ vận hành một cách hiệu quả giúp đảm bảo rằng hệ thống được kiểm thử sát với thực tế sử dụng, từ đó nâng cao độ tin cậy cảm nhận của người dùng cuối.

3.3.6 Mô hình lỗi

Trong kiểm thử phần mềm, các lỗi (faults) không chỉ là mục tiêu cần phát hiện mà còn là nguồn thông tin quan trọng giúp thiết kế các ca kiểm thử hiệu quả hơn. Việc phân tích các loại lỗi thường gặp trong phần mềm có thể cung cấp định hướng cụ thể cho việc xây dựng các trường hợp kiểm thử nhằm phát hiện lỗi tương tự trong tương lai. Thông thường, các lỗi trong phần mềm có thể được phân thành ba nhóm chính:

- Lỗi khởi tạo (initialization faults): xảy ra khi các biến hoặc đối tượng không được khởi tạo đúng cách trước khi sử dụng.
- Lỗi logic (logic faults): xuất phát từ việc lập trình sai thuật toán, điều kiện, hoặc luồng xử lý.
- Lỗi giao tiếp (interface faults): xuất hiện trong quá trình trao đổi dữ liệu giữa các mô-đun, lớp hoặc hệ thống khác nhau, thường do không tương thích định dạng hoặc thiếu kiểm tra dữ liệu đầu vào/đầu ra.

Dựa trên đặc điểm và nguyên nhân của lỗi, kiểm thử phần mềm có thể được tổ chức theo ba loại hình kiểm thử dựa trên lỗi phổ biến như sau:

- Dự đoán lỗi (Error Guessing): dựa vào kinh nghiệm và trực giác của người kiểm thử để phỏng đoán những vị trí dễ xảy ra lỗi, từ đó thiết kế các ca kiểm thử tương ứng.
- Gieo lỗi (Fault Seeding): cố tình đưa các lỗi nhân tạo vào hệ thống nhằm kiểm tra khả năng phát hiện lỗi của quy trình kiểm thử. Nếu các lỗi gieo được phát hiện hiệu quả, có thể suy ra rằng kiểm thử có khả năng phát hiện cả các lỗi thực tế chưa biết.
- Kiểm thử đột biến (Mutation Testing): tạo ra các phiên bản sửa đổi nhẹ (đột biến) của chương trình bằng cách thay đổi nhỏ trong mã nguồn (như thay toán tử, giá trị...), sau đó thực hiện kiểm thử để xem liệu ca kiểm thử hiện tại có phát hiện được sự khác biệt hay không. Kỹ thuật này giúp đánh giá độ hiệu quả của bộ ca kiểm thử hiện có.

Việc áp dụng các phương pháp kiểm thử dựa trên lỗi giúp nâng cao khả năng phát hiện lỗi một cách chủ động và có chiến lược, từ đó cải thiện chất lượng phần mềm một cách đáng kể.

3.3.7 Các kỹ thuật thiết kế ca kiểm thử

Để thiết kế các ca kiểm thử một cách hiệu quả và với chi phí thấp nhất, người ta thường áp dụng các kỹ thuật thiết kế kiểm thử khác nhau, trong đó được chia thành hai nhóm chính: kiểm thử hộp trắng và kiểm thử hộp đen. Mỗi kỹ thuật có phạm vi áp dụng và mục đích riêng, nhưng đều nhằm phát hiện lỗi và đảm bảo phần mềm hoạt động đúng như mong đợi.

- Kiểm thử hộp trắng (white-box testing), còn gọi là kiểm thử cấu trúc, là kỹ thuật kiểm thử dựa trên cấu trúc bên trong của chương trình, bao gồm luồng điều khiển (control flow) và luồng dữ liệu (data flow). Phương pháp này yêu cầu người kiểm thử phải có kiến thức về mã nguồn để xây dựng các ca kiểm thử nhằm bao phủ các nhánh, điều kiện, vòng lặp, hoặc các đường dẫn trong chương trình. Kiểm thử hộp trắng thường được áp dụng ở mức độ đơn vị (unit level), và là phương pháp phổ biến được các lập trình viên hoặc kiểm thử viên kỹ thuật sử dụng trong giai đoạn kiểm thử đơn vị.
- Ngược lại, kiểm thử hộp đen (black-box testing), còn được gọi là kiểm thử chức năng, lại không quan tâm đến cách hiện thực bên trong của chương trình mà tập trung vào đặc tả yêu cầu và hành vi mong đợi của hệ thống. Người kiểm thử chỉ cần biết đầu vào và đầu ra, từ đó thiết kế các ca kiểm thử để kiểm tra xem phần mềm có thực hiện đúng các chức năng được mô tả hay không. Kiểm thử hộp đen có thể được sử dụng ở nhiều mức độ khác nhau, từ kiểm thử đơn vị đến kiểm thử tích hợp hoặc kiểm thử toàn hệ thống. Phương pháp này thường được áp dụng bởi lập trình viên, kiểm thử viên, hoặc chuyên gia kiểm thử độc lập.

Việc kết hợp cả hai kỹ thuật kiểm thử hộp trắng và hộp đen sẽ giúp bao phủ cả phần cấu trúc bên trong và chức năng bên ngoài của phần mềm, từ đó nâng cao hiệu quả phát hiện lỗi và đảm bảo chất lượng hệ thống.

3.3.8 Lập kế hoạch kiểm thử

Mục đích của lập kế hoạch kiểm thử hệ thống là để sẵn sàng và tổ chức tốt cho thực thi kiểm thử

Kế hoạch kiểm thử cung cấp một framework, phạm vi, chi tiết về nguồn tài nguyên yêu cầu, công sức cần thiết, thời gian của các hoạt động và chi phí cần thiết. Thiết kế kiểm thử là một trong những pha quan trọng của kiểm thử hệ thống, trong giai đoạn này, các yêu cầu của hệ thống sẽ được tìm hiểu một cách cẩn thận kĩ

lượng, các yếu tố cần kiểm thử của hệ thống sẽ được xác định cũng như các mục tiêu và hành vi chi tiết của các ca kiểm thử được định nghĩa.

Mục tiêu kiểm thử được xác định từ nhiều nguồn khác nhau như đặc tả yêu cầu, đặc tả chức năng. Mỗi một mục tiêu kiểm thử có thể được kiểm thử bằng nhiều test cases. Mỗi một ca kiểm thử được thiết kế như là một sự kết hợp của nhiều thành phần mô đun được gọi là các bước kiểm thử. Các bước này được kết hợp lại với nhau nhằm tạo ra những ca kiểm thử phức tạp nhiều bước. Một ca kiểm thử cần phải được xác định một cách rõ ràng để những người khác có thể hiểu và dùng nhiều lần trong nhiều ngữ cảnh khác nhau.

Phát triển hướng kiểm thử (Test-Driven Development -TDD) là một cách tiếp cận mới của phát triển phần mềm gắn liền với quy trình xây dựng thiết kế kiểm thử trong đó các lập trình viên sẽ xây dựng và cài đặt các ca kiểm thử trước khi mã nguồn chương trình được xây dựng. Cách tiếp cận này là một nhân tố quan trọng trong các chu trình phát triển phần mềm nhanh như XP. Đặc điểm chính của phát triển nhanh là (i) phát triển tăng dần (ii) lập trình viên viết code unit và khách hàng thực hiện kiểm thử chấp nhận được tiến hành song song với nhau (iii) kiểm thử hồi quy thực hiện liên tục (iv) viết mã kiểm thử và ca kiểm thử đồng thời trước khi viết mã nguồn chương trình.

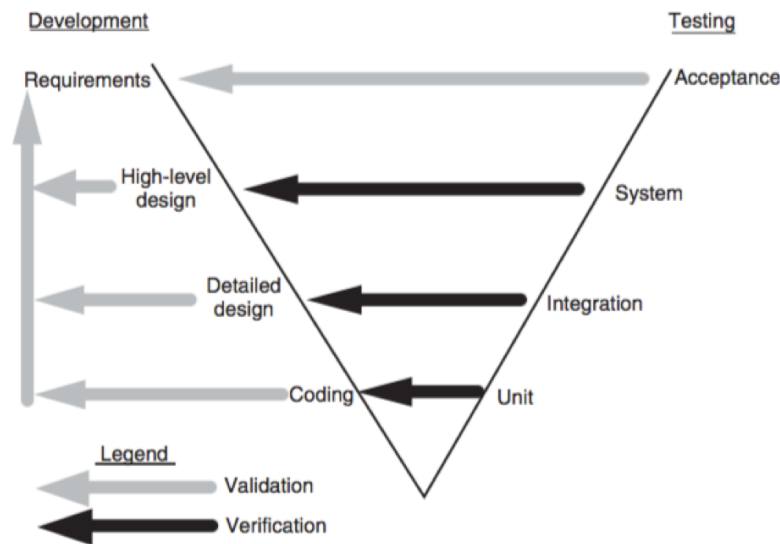
3.3.9 Quản lý và đánh giá kiểm thử

Mục tiêu của việc quản lý và đánh giá việc thực thi kiểm thử là nhằm xác định hệ thống đang đạt đến mức độ chất lượng như thế nào tại từng thời điểm. Căn cứ vào những thông số cụ thể về chất lượng sản phẩm ở các giai đoạn phát triển khác nhau, người quản lý có thể đưa ra những quyết định chính xác và hợp lý hơn, từ đó đảm bảo quá trình phát triển phần mềm được định hướng đúng và hiệu quả.

Độ đo thực thi kiểm thử được chia thành hai nhóm chính:

- **Độ đo kiểm soát quá trình thực thi kiểm thử:** liên quan đến việc theo dõi và đánh giá việc thực hiện các ca kiểm thử, bao gồm tỷ lệ kiểm thử đã thực hiện, tỷ lệ kiểm thử thành công, độ bao phủ mã nguồn, v.v.
- **Độ đo kiểm soát lỗi:** tập trung vào việc theo dõi và phân tích các lỗi được phát hiện trong quá trình kiểm thử, bao gồm số lượng lỗi, mức độ nghiêm trọng, tỷ lệ lỗi còn tồn tại, tỷ lệ khắc phục lỗi thành công, v.v.

Các độ đo này cần được **tính toán và phân tích một cách định kỳ** (theo tuần hoặc theo ngày) để đảm bảo thông tin chất lượng của dự án phần mềm luôn



Hình 3.3: Các mức độ kiểm thử

được cập nhật kịp thời. Điều này giúp nhóm phát triển nhanh chóng phát hiện các vấn đề, điều chỉnh kế hoạch kiểm thử và phát triển, đồng thời đảm bảo tiến độ và chất lượng chung của toàn bộ dự án.

3.4 Các mức độ kiểm thử

Kiểm thử được thực thi ở các mức độ khác nhau bao gồm toàn bộ hệ thống hoặc một phần của nó trong suốt chu trình vòng đời của sản phẩm phần mềm.

Có 4 mức độ kiểm thử: đơn vị, tích hợp, hệ thống và chấp nhận. 3 mức độ đầu tiên được thực hiện bởi các thành phần khác nhau của đội dự án. Mức độ cuối cùng thực hiện bởi người dùng. Bốn mức độ này được mô tả cụ thể trên mô hình chữ V. Mô hình chữ V cũng dựa trên mô hình thác nước tuy nhiên lại tập trung nhiều hơn vào các giai đoạn kiểm thử để đảm bảo được chất lượng sản phẩm qua từng giai đoạn phát triển dự án phần mềm do đó được sử dụng rộng rãi trong công nghiệp phần mềm.

3.4.1 Kiểm thử đơn vị

Lập trình viên sẽ kiểm tra từng đơn vị chương trình một cách độc lập ví dụ các thủ tục, các phương thức, hàm, lớp... Sau khi đảm bảo rằng các phần tử này hoạt động tốt thì các module sẽ được kết hợp lại với nhau để xây dựng thành các thành

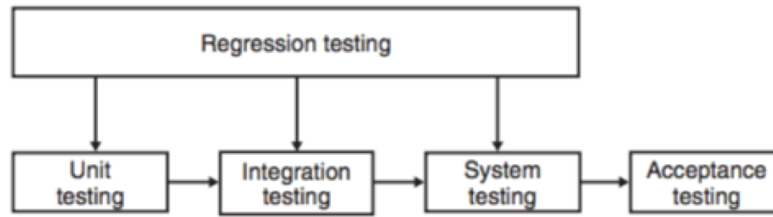
phần hoặc hệ thống con tiến hành kiểm thử tích hợp. Kiểm thử tích hợp thông thường sẽ được các lập trình viên và các kiểm thử viên kết hợp làm. Mục đích của kiểm thử tích hợp là để xây dựng một hệ thống đủ ổn định và có thể tiến hành kiểm thử hệ thống.

3.4.2 Kiểm thử hệ thống

Bao gồm nhiều loại kiểm thử được tiến hành: kiểm thử chức năng, kiểm thử tính an toàn, kiểm thử độ rắn chắc ổn định, kiểm thử hiệu năng và kiểm thử độ tin cậy... Kiểm thử hệ thống là một pha quan trọng trong quá trình phát triển phần mềm do nhu cầu cấp thiết cần phải bàn giao sản phẩm đúng hạn, khám phá hết các lỗi có thể và kiểm chứng tất cả những công việc sửa chữa lỗi đã tiến hành và các lỗi phát hiện đều được sửa và không làm phát sinh các lỗi mới. Kiểm thử hệ thống bao gồm một số hoạt động cụ thể như: tạo kế hoạch kiểm thử, thiết kế các trường hợp kiểm thử, chuẩn bị môi trường kiểm thử, thực thi các ca kiểm thử theo một chiến lược rõ ràng và quản lý toàn bộ quá trình thực thi kiểm thử.

Trong kiểm thử phần mềm, kiểm thử tích hợp hệ thống (system integration testing) là giai đoạn quan trọng nhằm kiểm tra sự tương tác giữa các mô-đun, thành phần hoặc phân hệ sau khi đã được tích hợp lại với nhau. Có nhiều kỹ thuật kiểm thử tích hợp được sử dụng tùy thuộc vào cấu trúc hệ thống và chiến lược phát triển phần mềm. Một số kỹ thuật phổ biến bao gồm:

- Kiểm thử tích hợp tăng dần (Incremental Integration Testing): Các mô-đun được tích hợp và kiểm thử từng bước theo thứ tự, thay vì chờ tất cả mô-đun hoàn thành. Điều này giúp phát hiện lỗi sớm hơn và dễ xác định nguyên nhân. Kỹ thuật này có thể triển khai theo hai hướng: từ trên xuống (top-down) hoặc từ dưới lên (bottom-up).
- Kiểm thử từ trên xuống (Top-down Integration Testing): Quá trình tích hợp bắt đầu từ các mô-đun cấp cao (mô-đun điều khiển) rồi dần dần tích hợp các mô-đun con phía dưới. Các mô-đun chưa sẵn sàng sẽ được thay thế tạm thời bằng các stubs (đoạn mã giả lập chức năng của mô-đun con). Phương pháp này thuận lợi cho việc kiểm thử các luồng điều khiển chính và giao diện ở cấp cao.
- Kiểm thử từ dưới lên (Bottom-up Integration Testing): Bắt đầu tích hợp và kiểm thử từ các mô-đun thấp nhất (mô-đun xử lý dữ liệu) lên các mô-đun cấp cao hơn. Các mô-đun điều khiển chưa có sẽ được thay thế bằng các drivers (chương trình giả lập gọi mô-đun). Phương pháp này thích hợp để kiểm tra



Hình 3.4: Kiểm thử hồi quy

chi tiết chức năng và hoạt động logic của từng mô-đun độc lập trước khi tích hợp vào tổng thể.

- Kiểm thử kiểu sandwich (Sandwich Integration Testing): Là sự kết hợp giữa top-down và bottom-up. Việc tích hợp và kiểm thử được thực hiện đồng thời từ cả hai hướng, sau đó hội tụ ở giữa hệ thống. Kỹ thuật này tận dụng ưu điểm của cả hai phương pháp và giúp giảm thời gian tích hợp tổng thể.
- Kiểm thử tích hợp kiểu big bang (Big Bang Integration Testing): Tất cả các mô-đun được tích hợp cùng lúc sau khi hoàn thành riêng lẻ, và toàn bộ hệ thống được kiểm thử ngay một lần. Tuy nhiên, phương pháp này có nhược điểm lớn là khó xác định nguồn gốc lỗi nếu xảy ra, và thường chỉ phù hợp với các hệ thống nhỏ hoặc khi tích hợp là tương đối đơn giản.

Việc lựa chọn kỹ thuật kiểm thử tích hợp phù hợp giúp đảm bảo rằng các thành phần trong hệ thống phối hợp đúng chức năng, ổn định và đáng tin cậy trước khi bước vào kiểm thử hệ thống hoặc triển khai chính thức.

3.4.3 Kiểm thử hồi quy

Bên cạnh bốn mức độ kiểm thử nói trên, một loại kiểm thử khác cũng được thực hiện trong chu trình phát triển phần mềm là kiểm thử hồi quy (regression test). Kiểm thử hồi quy được thực hiện bất cứ khi nào một thành phần của hệ thống được thay đổi. Ý tưởng chủ đạo của kiểm thử hồi quy là để đảm bảo rằng mọi sự thay đổi được thực hiện trên hệ thống không làm xuất hiện lỗi nào mới. Kiểm thử hồi quy không được tính là một mức độ kiểm thử mà chính xác hơn, nó được coi là một pha con của kiểm thử đơn vị, kiểm thử tích hợp và kiểm thử hệ thống.

Trong kiểm thử hồi quy, chúng ta không thiết kế các test case mới mà sẽ lựa chọn, thiết lập ưu tiên và thực thi các trường hợp kiểm thử đã có để đảm bảo rằng các thay đổi mới thực hiện trên hệ thống không làm xuất hiện các lỗi mới. Kiểm

thử hồi quy là một quá trình tốn kém và để giảm thiểu chi phí kiểm thử trong chu trình phần mềm thì người ta sẽ sử dụng các công cụ để có thể thực hiện tự động việc kiểm thử này.

3.4.4 Kiểm thử chấp nhận

Sau khi hoàn thành giai đoạn kiểm thử hệ thống, sản phẩm phần mềm sẽ được bàn giao cho khách hàng để tiến hành kiểm thử chấp nhận (acceptance testing), nhằm đánh giá mức độ phù hợp của sản phẩm với các yêu cầu đã thỏa thuận ban đầu. Mục tiêu chính của kiểm thử chấp nhận không phải là tìm kiếm lỗi như trong các giai đoạn kiểm thử trước đó, mà là kiểm chứng rằng sản phẩm đã đáp ứng đầy đủ các yêu cầu đặc tả, đảm bảo phần mềm sẵn sàng được đưa vào sử dụng thực tế.

Kiểm thử chấp nhận được tiến hành dựa trên những mong đợi thực tế của khách hàng, giúp họ tự xác định xem sản phẩm có thể triển khai cho người dùng cuối hay không. Việc kiểm thử này đóng vai trò quan trọng trong việc đưa ra quyết định cuối cùng về việc phê duyệt và nghiệm thu phần mềm.

Có hai hình thức kiểm thử chấp nhận phổ biến:

- Kiểm thử chấp nhận người dùng (User Acceptance Testing – UAT): được thực hiện bởi khách hàng hoặc đại diện khách hàng, nhằm đảm bảo rằng hệ thống đáp ứng đầy đủ các tiêu chuẩn chấp nhận đã được quy định trong hợp đồng hoặc tài liệu thỏa thuận giữa khách hàng và nhóm phát triển. Đây là bước kiểm tra cuối cùng trước khi phần mềm được triển khai chính thức.
- Kiểm thử chấp nhận nghiệp vụ (Business Acceptance Testing – BAT): thường được thực hiện bởi khách hàng nội bộ nằm trong tổ chức phát triển (chẳng hạn như bộ phận nghiệp vụ hoặc quản lý sản phẩm). Loại kiểm thử này giúp đội phát triển đảm bảo rằng hệ thống có khả năng vượt qua kiểm thử chấp nhận người dùng (UAT) trước khi chính thức bàn giao cho khách hàng bên ngoài. BAT thường được xem là bước đệm để đánh giá rủi ro và cải thiện sản phẩm trước khi bước vào giai đoạn nghiệm thu chính thức.

Cả hai hình thức kiểm thử này đều đóng vai trò thiết yếu trong việc đảm bảo chất lượng và độ tin cậy cuối cùng của sản phẩm phần mềm, đồng thời tăng cường sự tin tưởng từ phía khách hàng.

Chương 4

Các kĩ thuật kiểm thử hộp đen

4.1	Giới thiệu	73
4.2	Phương pháp phân lớp tương đương	74
4.3	Phương pháp phân tích giá trị biên	78
4.4	Phương pháp bảng quyết định	82
4.5	Phương pháp biểu đồ trạng thái	85

4.1 Giới thiệu

Kiểm thử hộp đen (Black-box Testing) là một phương pháp kiểm thử phần mềm trong đó người kiểm thử không cần biết về cấu trúc bên trong hay mã nguồn của hệ thống. Động lực chính của phương pháp này là đánh giá phần mềm từ góc nhìn của người dùng cuối – những người chỉ quan tâm đến việc phần mềm có thực hiện đúng các chức năng theo yêu cầu hay không, chứ không quan tâm đến cách nó được lập trình.

Kiểm thử hộp đen giúp kiểm tra xem hệ thống có xử lý đúng các đầu vào và cho ra đầu ra mong đợi hay không, đồng thời phát hiện các lỗi như thiếu chức năng, xử lý sai logic, lỗi giao diện hay lỗi khi tương tác với các hệ thống bên ngoài. Phương pháp này đặc biệt hữu ích vì không phụ thuộc vào mã nguồn, cho phép các tester độc lập hoặc khách hàng thực hiện kiểm thử chấp nhận dễ dàng. Ngoài ra, kiểm thử hộp đen còn là cơ sở tốt cho việc tự động hóa kiểm thử, giúp tăng hiệu quả và độ chính xác khi kiểm thử lặp đi lặp lại. Khi được kết hợp với các phương pháp kiểm thử khác như kiểm thử hộp trắng, nó góp phần nâng cao độ bao phủ và chất lượng

kiểm thử toàn diện cho phần mềm.

Kiểm thử hộp đen được xem là một phương pháp kiểm thử chức năng, trong khi kiểm thử hộp trắng là kiểm thử cấu trúc bên trong của hệ thống. Kiểm thử hộp đen tập trung vào việc kiểm tra hành vi của phần mềm dựa trên yêu cầu chức năng mà không cần quan tâm đến cách cài đặt bên trong. Phương pháp này bao gồm các mức kiểm thử như kiểm thử đơn vị (Unit Test), kiểm thử tích hợp (Integration Test) và kiểm thử hệ thống (System Test). Những người thực hiện kiểm thử hộp đen thường bao gồm lập trình viên, kỹ sư kiểm thử (Test Engineers) và kỹ sư đảm bảo chất lượng phần mềm (Quality Assurance Engineers). Trong khi đó, kiểm thử hộp trắng đòi hỏi người kiểm thử phải hiểu rõ cấu trúc mã nguồn và được áp dụng chủ yếu trong giai đoạn kiểm thử đơn vị và kiểm thử tích hợp. Các đối tượng thường thực hiện kiểm thử hộp trắng là lập trình viên và kỹ sư kiểm thử, những người có kiến thức sâu về mã nguồn và luồng xử lý nội tại của chương trình.

4.2 Phương pháp phân lớp tương đương

4.2.1 Giới thiệu

Phân lớp tương đương (Equivalence Partitioning) là một kỹ thuật kiểm thử phần mềm trong đó tập dữ liệu đầu vào được chia thành các lớp, gọi là các lớp tương đương. Một lớp được coi là tương đương nếu tất cả các giá trị trong lớp đó khiến hệ thống phản hồi theo cùng một cách, tức là chúng có tính chất giống nhau trong quá trình xử lý. Việc chia lớp như vậy giúp giảm số lượng ca kiểm thử cần thiết, đồng thời vẫn đảm bảo hiệu quả kiểm thử.

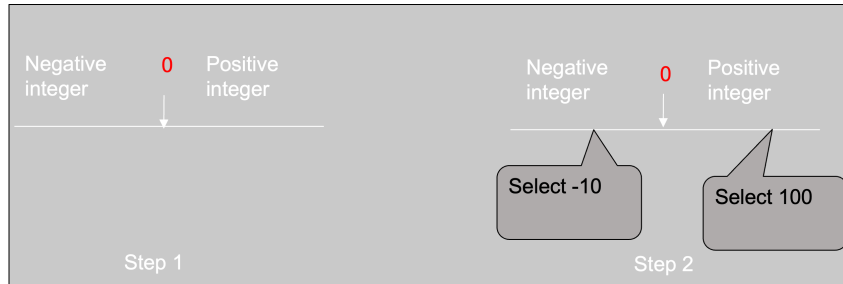
Thông thường, dữ liệu đầu vào một cách đơn giản có thể được chia thành hai loại lớp: lớp hợp lệ (valid class) – bao gồm các giá trị đầu vào hợp lệ mà hệ thống được thiết kế để xử lý, và lớp không hợp lệ (invalid class) – bao gồm các giá trị không hợp lệ mà hệ thống cần xử lý lỗi hoặc từ chối.

4.2.2 Phương pháp thực hiện

Quy trình thực hiện kiểm thử theo phương pháp phân lớp tương đương gồm hai bước chính. Bước đầu tiên là xác định tất cả các lớp tương đương có thể có từ tập dữ liệu đầu vào. Sau đó, ở bước thứ hai, từ mỗi lớp tương đương, chọn ra một giá trị đại diện cụ thể để làm dữ liệu kiểm thử. Việc kiểm thử với mỗi giá trị đại diện được xem là đủ để bao phủ toàn bộ hành vi của lớp đó.

Ví dụ hàm kiểm tra `check(int x)` kiểm tra xem có trả về số dương không? Áp

dùng phương pháp phân lớp tương đương, ta có thể chia dữ liệu đầu vào thành hai lớp tương đương là lớp các số âm và lớp các số dương. Ở mỗi lớp sẽ lựa chọn ra một giá trị để kiểm thử. Chúng ta sẽ có hai test case ứng với dữ liệu test là -10 và 100.



Hình 4.1: Ví dụ phân lớp tương đương

Việc chọn một giá trị đại diện từ mỗi lớp giúp đơn giản hóa quá trình kiểm thử mà vẫn đảm bảo được việc đánh giá đầy đủ hành vi của hệ thống đối với từng nhóm dữ liệu tương đương.

Ví dụ: Hệ thống bảo hiểm xã hội

- Mô tả: Hệ thống bảo hiểm xã hội với đặc tả là: chỉ chi trả cho các đối tượng là nữ tuổi dưới 85 và nam giới tuổi dưới 80.
- Yêu cầu: xác định các phân lớp tương đương và đưa ra các test case cụ thể.
- Phân tích
 - Đầu vào: Giới tính (Gender) và tuổi (Age)
 - Kết quả: Chấp nhận (Accept), không chấp nhận (Reject)

Căn cứ vào đặc tả, có thể xác định được đầu vào của dữ liệu gồm giới tính và tuổi. Từ đó xác định được các lớp tương đương.

- Với giới tính là **nữ (Female)**:
 - tuổi < 85 $\Rightarrow$ Accept.
 - tuổi $\geq$ 85 $\Rightarrow$ Reject.
- Với giới tính là **nam (Male)**:
 - tuổi < 80 $\Rightarrow$ Accept.
 - tuổi $\geq$ 80 $\Rightarrow$ Reject.

Classes	Test Cases
C1: Input: Males over 80	T1: male, 83, reject
C2: Input: Males 80 or under	T2: male, 56, accept
C3: Input: Females 85 or under	T3: female, 83, accept
C4: Input: Females over 85	T4: female, 87, reject

Hình 4.2: Các lớp tương đương

Các lớp C1, C2, C3, C4 là tương đương với dữ liệu hợp lệ. Đối với dữ liệu không hợp lệ như >99 và <0 thì chúng ta cũng đưa ra được thêm 4 lớp nữa tương ứng với hai giới tính nam và nữ.

Một số chú ý khi sử dụng phân lớp tương đương

Khi sử dụng kỹ thuật phân lớp tương đương trong kiểm thử phần mềm, cần chú ý đến cách phân chia dữ liệu đầu vào để đảm bảo việc kiểm thử bao phủ đầy đủ các tình huống quan trọng.

- Nếu điều kiện của dữ liệu đầu vào là một dải các giá trị tuyến tính hoặc một danh sách có sắp xếp các giá trị thì chúng ta sẽ lấy đại diện một giá trị hợp lệ thuộc dải/danh sách và hai giá trị ko hợp lệ nằm ngoài dải/danh sách.
- Nếu đầu vào là một tập hợp với các giá trị cụ thể hoặc một điều kiện boolean thì chúng ta sẽ lấy một giá trị hợp lệ nằm trong tập hợp và một giá trị ko hợp lệ nằm ngoài tập hợp.
- Các trường hợp khác thì cần chia lớp nhỏ hơn nữa.

Bài tập 1: Yêu cầu sinh viên xác định các lớp tương đương và các ca kiểm thử cho module xác định tuyển dụng như bên dưới.

Age	Employment Status
0-15	Don't hire
16-17	Can hire part-time
18-55	Can hire full-time
56-99	Don't hire

Hình 4.3: Bài tập 1

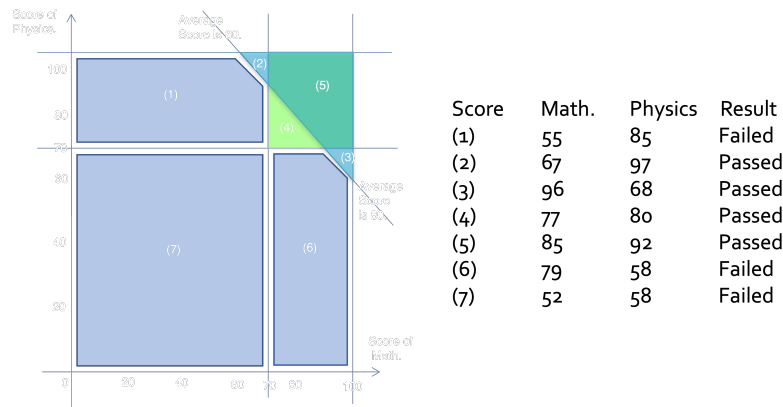
Câu hỏi: Có bao nhiêu lớp tương đương? Có bao nhiêu test case có thể được thiết kế?

Hướng dẫn: sinh viên vẽ trục tọa độ để xác định các lớp tương đương.

Bài tập 2: Xét kết quả thi

Mô tả: Sinh viên thi hai môn toán và vật lý. Sinh viên sẽ đỗ nếu thỏa một trong 2 trường hợp sau:

1. Điểm của 2 môn đều lớn hơn hoặc bằng 70/100
2. Điểm trung bình của hai môn lớn hơn hoặc bằng 80/100
3. Trong trường hợp khác sinh viên sẽ trượt.



Hình 4.4: Bài tập 2

4.2.3 Ưu và nhược điểm

Ưu điểm của phương pháp phân lớp tương đương

Phương pháp phân lớp tương đương mang lại một số ưu điểm rõ rệt trong quá trình kiểm thử phần mềm. Quan trọng nhất, kỹ thuật này giúp giảm đáng kể số lượng các ca kiểm thử cần thực hiện bằng cách tránh kiểm thử lặp lại trên các giá trị đầu vào có tính chất tương tự nhau. Thay vì kiểm tra mọi giá trị có thể, người kiểm thử chỉ cần chọn một giá trị đại diện từ mỗi lớp tương đương để kiểm thử, từ đó tiết kiệm thời gian và công sức. Đồng thời, phương pháp vẫn đảm bảo được độ bao phủ chức năng của kiểm thử, vì mỗi lớp dữ liệu đều được đại diện và kiểm tra đầy đủ. Điều này giúp nâng cao hiệu quả và chất lượng của quá trình kiểm thử mà không làm giảm độ tin cậy của kết quả.

Nhược điểm của phương pháp phân lớp tương đương

Mặc dù phương pháp phân lớp tương đương giúp giảm số lượng ca kiểm thử, nhưng nó vẫn tồn tại một số nhược điểm đáng lưu ý. Nhược điểm lớn nhất là việc xác định các lớp tương đương phụ thuộc rất nhiều vào kinh nghiệm và sự phán đoán của người kiểm thử. Trong những trường hợp đặc tả đầu vào rõ ràng, việc phân chia các lớp là khá dễ dàng. Tuy nhiên, đối với những yêu cầu đơn giản hoặc không mô tả cụ thể, việc xác định các lớp có thể mang tính chủ quan và không đồng nhất. Ví dụ, với một module đơn giản dùng để phân loại độ tuổi, trong đó người từ 18 tuổi trở lên được coi là người lớn (Adult), và dưới 18 tuổi là trẻ em (Children), người A có thể chỉ đưa ra hai giá trị kiểm thử là 10 và 30. Trong khi đó, người B có thể cẩn trọng hơn và thêm một trường hợp không hợp lệ như -5, dẫn đến ba ca kiểm thử. Điều này cho thấy kết quả kiểm thử có thể không nhất quán giữa các kiểm thử viên nếu không có hướng dẫn rõ ràng.

Bên cạnh đó, phương pháp phân lớp tương đương không tập trung vào các giá trị biên – nơi mà phần lớn lỗi phần mềm thường xảy ra. Việc bỏ qua các giá trị tại ranh giới giữa các lớp hợp lệ và không hợp lệ có thể dẫn đến việc bỏ sót các lỗi quan trọng. Do đó, để đảm bảo độ chính xác và độ bao phủ kiểm thử cao hơn, phương pháp phân lớp tương đương nên được kết hợp với kiểm thử giá trị biên (Boundary Value Analysis) trong thực tế.

4.3 Phương pháp phân tích giá trị biên

4.3.1 Giới thiệu

Phân tích giá trị biên (Boundary Value Analysis – BVA) là một phương pháp kiểm thử thường được sử dụng kết hợp với phương pháp phân lớp tương đương nhằm tăng độ bao phủ kiểm thử và khả năng phát hiện lỗi. Nếu như trong phân lớp tương đương, người kiểm thử chỉ chọn một giá trị đại diện bất kỳ từ mỗi lớp để kiểm thử, thì với phương pháp phân tích giá trị biên, các trường hợp kiểm thử được lựa chọn tập trung vào các giá trị nằm tại hoặc gần ranh giới giữa các lớp.

Lý do là bởi các lỗi trong phần mềm thường xuất hiện tại các điểm biên – nơi lập trình viên dễ mắc lỗi trong quá trình xử lý các điều kiện logic. Do đó, thay vì chỉ kiểm tra bên trong vùng hợp lệ hoặc không hợp lệ, phương pháp này đề xuất kiểm thử các giá trị:

- Ngay tại giá trị biên.
- Ngay trên giá trị biên (biên trên).

- Ngay dưới giá trị biên (biên dưới).

Bằng cách kết hợp với phân lớp tương đương, kiểm thử giá trị biên giúp xây dựng các ca kiểm thử có khả năng phát hiện các lỗi phổ biến như sai dấu so sánh, thiếu hoặc dư điều kiện biên, hoặc lỗi do xử lý logic không đầy đủ. Đây là một trong những phương pháp đơn giản nhưng rất hiệu quả trong việc đảm bảo chất lượng phần mềm.

4.3.2 Các bước thực hiện

Để áp dụng hiệu quả phương pháp phân tích giá trị biên trong kiểm thử phần mềm, người kiểm thử có thể thực hiện theo các bước sau:

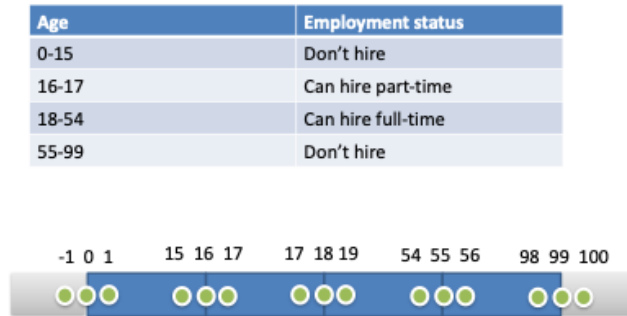
1. Xác định các lớp tương đương: Trước tiên, cần xác định các lớp dữ liệu đầu vào theo phương pháp phân lớp tương đương. Việc này giúp chia nhỏ không gian kiểm thử và định hướng các vùng giá trị cần quan tâm.
2. Xác định các giá trị biên của từng lớp: Tiếp theo, xác định rõ các ranh giới giữa các lớp hợp lệ và không hợp lệ. Đây chính là những điểm mà hệ thống dễ phát sinh lỗi khi xử lý.
3. Với mỗi giá trị biên được xác định, thiết kế các ca kiểm thử bao gồm:
 - Một giá trị ngay tại biên (boundary value).
 - Một giá trị nhỏ hơn biên (just below the boundary).
 - Một giá trị lớn hơn biên (just above the boundary).

Các giá trị này giúp đảm bảo rằng hệ thống phản hồi chính xác trong các tình huống sát ranh giới, nơi lỗi thường hay xuất hiện.

Ví dụ 1: Employability Module

- 6 lớp tương đương (4 lớp hợp lệ)
- Với mỗi giá trị biên ta lấy 3 giá trị on/below/upper
- Tạo thành 15 Test cases.

Lưu ý: khi lựa chọn các giá trị để thiết kế test case nếu không có yêu cầu cụ thể về việc lấy giá trị chính xác below là nhỏ hơn on là 1 đơn vị hay giá trị upper



Hình 4.5: Ví dụ 1

là lớn hơn on là 1 đơn vị thì chúng ta có thể rút gọn các giá trị below của biên này nhưng lại là upper của biên khác.

Ví dụ 2: Human's age

Cách khác để lựa chọn giá trị

- Một trên biên (One point on the boundary)
- Một cạnh biên thuộc lớp không hợp lệ

Có một cách khác đối với việc lựa chọn giá trị dữ liệu để test: Chọn các giá trị trên biên và hai giá trị hai bên biên trong các lớp khác (below 1 biên và above một biên khác). Ví dụ thay vì các test case tương ứng với các giá trị đầu vào: 17, 18, 19, 59, 60, 61 chúng ta chỉ cần 4 test case là: 17, 18, 60 và 61 vì 19 và 59 cho cùng hoạt động với 18 và 60

4.3.3 Ưu và nhược điểm

Ưu điểm

Phân tích giá trị biên là một phương pháp kiểm thử đơn giản, dễ hiểu và dễ áp dụng trong thực tế. Ưu điểm nổi bật của kỹ thuật này là khả năng phát hiện lỗi hiệu quả tại các ranh giới – nơi mà lập trình viên thường dễ mắc sai sót nhất, như nhầm lẫn trong toán tử so sánh hoặc xử lý không đầy đủ điều kiện. Khi được sử dụng kết hợp với phương pháp phân lớp tương đương, phân tích giá trị biên góp phần nâng cao độ bao phủ kiểm thử và đảm bảo rằng các ca kiểm thử được thiết kế bao quát cả các tình huống đặc biệt sát giới hạn. Điều này giúp tránh bỏ sót nhiều lỗi quan trọng, từ đó cải thiện chất lượng và độ tin cậy của phần mềm. Phương pháp này đặc biệt hữu ích trong các hệ thống có các điều kiện kiểm tra ràng buộc hoặc giới hạn rõ ràng.

Các hạn chế

Mặc dù phương pháp phân lớp tương đương và phân tích giá trị biên đều rất hiệu quả trong việc giảm số lượng ca kiểm thử và phát hiện lỗi tại các vị trí nhạy cảm, nhưng cả hai phương pháp này đều có những hạn chế nhất định. Nhược điểm chính là chúng chỉ thực sự phù hợp khi dữ liệu đầu vào thuộc một miền liên tục hoặc là một tập hợp các giá trị rời rạc, và khi các đầu vào tương đối độc lập với nhau. Trong những trường hợp mà dữ liệu đầu vào có mối quan hệ phụ thuộc lẫn nhau, hoặc có nhiều điều kiện ràng buộc phức tạp giữa các biến (ví dụ như "nếu A đúng thì B phải nằm trong khoảng X-Y"), thì cả hai phương pháp trên đều trở nên không hiệu quả hoặc không thể áp dụng trực tiếp. Trong những tình huống đó, cần phải sử dụng các kỹ thuật kiểm thử nâng cao hơn như kiểm thử tổ hợp (combinatorial testing), bảng quyết định (decision table testing), hay kỹ thuật kiểm thử theo biểu đồ trạng thái.

Bài tập: The nextDate function

Hàm nextDate thực hiện trả về ngày tiếp theo của ngày truyền vào tham số. Dữ liệu đầu vào gồm có Month, Day và Year với các điều kiện C1, C2 C3 tương ứng như sau:

- C1: $1 \leq M \leq 12$
- C2: $1 \leq D \leq 31$
- C3: $1850 \leq Y \leq 2050$

Yêu cầu: liệt kê tất cả các lớp tương đương của từng loại dữ liệu (D, M và Y)

Phân tích: Ở đây, ta chia thành hai loại lớn là miền dữ liệu hợp lệ (valid) và miền dữ liệu không hợp lệ (invalid). Tuy nhiên liệu có cách nào để xác định các lớp tốt hơn? Ví dụ các tháng có 30 ngày hay 31 ngày? Tháng 2 có 28 hoặc 29 ngày dựa vào năm không nhuận hoặc nhuận?

Liệu chúng ta có thể định nghĩa các lớp tương đương một cách tốt hơn, thể hiện được nhiều hơn mối quan hệ của dữ liệu? Cụ thể một tháng có thể có 30 hoặc 31 ngày, tháng 2 có thể có 28 hoặc 29 ngày tùy thuộc năm có nhuận không? Căn cứ vào đó, chúng ta chia dữ liệu thành các lớp tương đương một cách tốt hơn. Thực tế là có nhiều sự kết hợp giữa M, D và Y cho nên có thể có rất nhiều test case. Theo tinh thần của phân lớp tương đương, chúng ta lấy từ mỗi lớp tương đương các giá trị cụ thể của dữ liệu để làm test case.

Tuy nhiên các dữ liệu cho M, D, Y thì có quan hệ với nhau, ví dụ: Tháng 3 có 31 ngày, tháng 4 có 30 ngày, tháng 2 năm nhuận có 29 ngày năm thường có 28 ngày...

Do đó để thiết kế dữ liệu một cách tối ưu chúng ta cần phải áp dụng phương pháp khác bên cạnh phương pháp phân lớp tương đương.

4.4 Phương pháp bảng quyết định

4.4.1 Giới thiệu

Bảng quyết định là một công cụ hiệu quả để mô tả các yêu cầu của hệ thống, đặc biệt thích hợp với các luật logic phức tạp. Phương pháp này hỗ trợ việc phân tích và xác định các tổ hợp đầu vào khác nhau, từ đó xây dựng các test case một cách có hệ thống và đầy đủ.

Cấu trúc của một bảng quyết định bao gồm:

- **Conditions (Điều kiện):** Các điều kiện cụ thể tương ứng với từng loại dữ liệu đầu vào.
- **Rules (Luật):** Các tổ hợp khác nhau của các điều kiện đầu vào, thể hiện các tình huống kiểm thử có thể xảy ra.
- **Actions (Hành động):** Các kết quả hoặc hành động đầu ra của hệ thống tương ứng với từng tổ hợp điều kiện.

Việc sử dụng bảng quyết định giúp đảm bảo bao phủ đầy đủ các kịch bản có thể xảy ra, đồng thời làm rõ mối quan hệ giữa các điều kiện và hành động, từ đó nâng cao chất lượng kiểm thử.

		Rule 1	Rule 2	...	Rule p
Conditions	Condition 1				
	Condition 2				
	...				
	Condition n				
Actions	Action 1				
	Action 2				
	...				
	Action m				

Hình 4.6: Hình thức của bảng quyết định

Ví dụ: Giảm giá bảo hiểm ô tô (Car Insurance Discount)

Mô tả:

- Nếu có gia đình và số năm làm việc ≥ 3 thì giảm giá 70
- Nếu có gia đình và số năm làm việc < 3 thì
 - nếu là sinh viên giỏi thì giảm giá 50%
 - ngược lại giảm giá 20%
- Nếu chưa lập gia đình và số năm làm việc ≥ 3 thì giảm giá 60
- Trong trường hợp chưa lập gia đình và số năm làm việc < 3 thì
 - nếu là sinh viên giỏi thì giảm giá 40%
 - ngược lại giảm giá 0%

		Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8
C	Married?	Yes	Yes	Yes	Yes	No	No	No	No
	# years employed	≥ 3	≥ 3	< 3	< 3	≥ 3	≥ 3	< 3	< 3
	Good student?	Yes	No	Yes	No	Yes	No	yes	No
A	Discount	70	70	50	20	60	60	40	0

Hình 4.7: Ví dụ

Ta vẽ bảng và xác định các conditions cũng như các luật tương ứng. Theo bảng quyết định ở trên, chúng ta sẽ có tất cả 8 test cases.

Nhận xét thấy rằng, với trường hợp số năm công tác ≥ 3 thì điều kiện sinh viên giỏi không cần xem xét. Khi đó chúng ta có thể kết hợp hai luật lại với nhau để giảm số lượng test case đi. Kết quả sau khi rút gọn ta có tất cả 6 test cases

4.4.2 Thiết kế test case

Quá trình chuyển đổi từ bảng quyết định sang các ca kiểm thử được thực hiện như sau:

- **Rules** (Các luật) trở thành **test cases** (các ca kiểm thử). Mỗi luật trong bảng quyết định đại diện cho một tổ hợp điều kiện đầu vào cụ thể và tương ứng với một ca kiểm thử riêng biệt.
- **Conditions** (Các điều kiện) trở thành **inputs** (dữ liệu đầu vào) trong ca kiểm thử. Mỗi điều kiện thể hiện một biến hoặc đặc điểm cần kiểm tra.

		Rule 1-2	Rule 3	Rule 4	Rule 5-6	Rule 7	Rule 8
C	Married?	Yes	Yes	Yes	No	No	No
	# years employed	≥ 3	< 3	< 3	≥ 3	< 3	< 3
	Good student?	-	Yes	No	-	Yes	No
A	Discount	70	50	20	60	40	0

Hình 4.8: Rút gọn bảng quyết định

- **Actions** (Các hành động) trở thành **expected results** (kết quả mong đợi). Đây là phản hồi hoặc hành vi mà hệ thống phải thực hiện khi thỏa mãn các điều kiện tương ứng.

4.4.3 Ưu và nhược điểm

Điểm mạnh của phương pháp bảng quyết định:

Phương pháp bảng quyết định đặc biệt hữu ích trong các ứng dụng có đặc điểm sau:

- **Logic điều kiện phức tạp (if-then-else):** Khi hệ thống có nhiều nhánh điều kiện, bảng quyết định giúp tổ chức các nhánh logic một cách rõ ràng và có hệ thống.
- **Mối quan hệ logic giữa các biến đầu vào:** Khi giá trị của các đầu vào phụ thuộc lẫn nhau hoặc ảnh hưởng lẫn nhau theo các quy tắc logic.
- **Tính toán phụ thuộc vào một phần tập hợp biến đầu vào:** Khi đầu ra được xác định dựa trên một phần hoặc tổ hợp cụ thể của các biến đầu vào.
- **Quan hệ nhân quả giữa đầu vào và đầu ra (cause-effect):** Khi có sự liên kết chặt chẽ giữa nguyên nhân (input) và kết quả (output), bảng quyết định giúp làm rõ các kịch bản kiểm thử cần thiết để bao phủ đầy đủ các tình huống.

Nhờ những đặc điểm trên, bảng quyết định thường được áp dụng trong các hệ thống như:

- Hệ thống xử lý nghiệp vụ (Business Rule Systems)
- Các trình xác thực dữ liệu đầu vào
- Ứng dụng có nhiều quy tắc kiểm tra điều kiện
- Các hệ thống có yêu cầu kiểm thử đầy đủ theo ma trận điều kiện

Hạn chế và cách khắc phục của bảng quyết định

Mặc dù bảng quyết định là một công cụ hữu ích để mô hình hóa các quy tắc logic phức tạp, nhưng nó cũng có một số hạn chế cần lưu ý:

- **Kích thước bảng có thể trở nên rất lớn:** Khi số lượng điều kiện tăng lên, số lượng tổ hợp (rules) sẽ tăng theo cấp số nhân (2^n với n điều kiện nhị phân). Điều này khiến bảng trở nên cồng kềnh, khó quản lý và khó phân tích.
- **Dễ trùng lặp hoặc dư thừa thông tin:** Nhiều rules trong bảng có thể dẫn đến cùng một kết quả (action), gây dư thừa và lãng phí trong việc kiểm thử.

Giải pháp: Để khắc phục những hạn chế trên, có thể áp dụng các kỹ thuật sau:

- **Phân tách bảng lớn thành các bảng nhỏ hơn:** Tách bảng tổng thành các bảng con dựa trên nhóm điều kiện có liên quan, giúp đơn giản hóa và tập trung vào từng phần logic cụ thể.
- **Loại bỏ sự trùng lặp:** Gộp các rule có cùng kết quả và điều kiện tương đồng để giảm số lượng rule cần thiết.
- **Sử dụng kỹ thuật rút gọn bảng quyết định (decision table minimization):** Áp dụng các phương pháp logic để rút gọn số lượng rule nhưng vẫn đảm bảo bao phủ đầy đủ các tình huống cần thiết.

4.5 Phương pháp biểu đồ trạng thái

4.5.1 Giới thiệu

Kỹ thuật kiểm thử dựa trên chuyển trạng thái được áp dụng khi hệ thống có hành vi khác nhau đối với cùng một đầu vào, tùy thuộc vào trạng thái hiện tại của hệ thống. Phương pháp này phù hợp với các ứng dụng có mô hình trạng thái hữu hạn, trong đó các hành vi của hệ thống được xác định bởi:

- **Các trạng thái (States):** Hệ thống có thể tồn tại trong một tập hợp hữu hạn các trạng thái. Mỗi trạng thái phản ánh một tình huống hoặc giai đoạn cụ thể trong chuỗi hoạt động của hệ thống.
- **Các sự kiện (Events):** Là các đầu vào hoặc tác nhân kích hoạt gây ra sự thay đổi trạng thái.
- **Chuyển trạng thái (Transitions):** Là các quy tắc xác định việc hệ thống chuyển từ trạng thái này sang trạng thái khác khi một sự kiện xảy ra.
- **Các hành động (Actions):** Là các kết quả hoặc phản hồi mà hệ thống thực hiện khi một chuyển trạng thái diễn ra.

4.5.2 Áp dụng phương pháp biểu đồ trạng thái

Phương pháp này đặc biệt phù hợp trong các trường hợp:

- Ứng dụng có hành vi phụ thuộc vào trạng thái trước đó.
- Hệ thống chứa logic dạng "finite state machine" (máy trạng thái hữu hạn), ví dụ: ATM, hệ thống xác thực người dùng, ứng dụng đặt vé, v.v.
- Cần đảm bảo mọi trạng thái và các chuyển tiếp giữa chúng đều được kiểm thử đầy đủ.

4.5.3 Các bước thiết kế kiểm thử trạng thái

1. Xác định các trạng thái có thể có của hệ thống.
2. Xác định các sự kiện (inputs) dẫn đến thay đổi trạng thái.
3. Mô tả các hành động tương ứng với mỗi sự kiện/trạng thái.
4. Vẽ sơ đồ chuyển trạng thái hoặc lập bảng chuyển trạng thái.
5. Từ sơ đồ hoặc bảng, thiết kế các ca kiểm thử để bao phủ đầy đủ các trạng thái và chuyển tiếp.

Chương 5

Các kĩ thuật kiểm thử hộp trắng

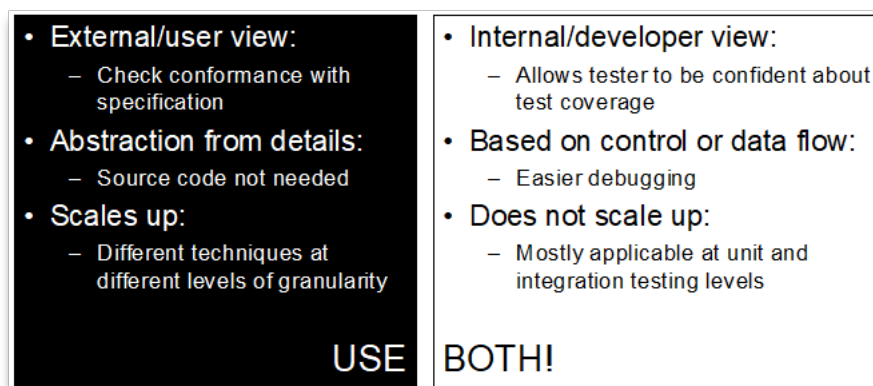
5.1	Giới thiệu	87
5.2	Kiểm thử luồng điều khiển	89
5.3	Kiểm thử luồng dữ liệu	113

5.1 Giới thiệu

Trong kiểm thử phần mềm, hai cách tiếp cận cơ bản thường được sử dụng là kiểm thử hộp đen (*Black Box Testing*) và kiểm thử hộp trắng (*White Box Testing*).

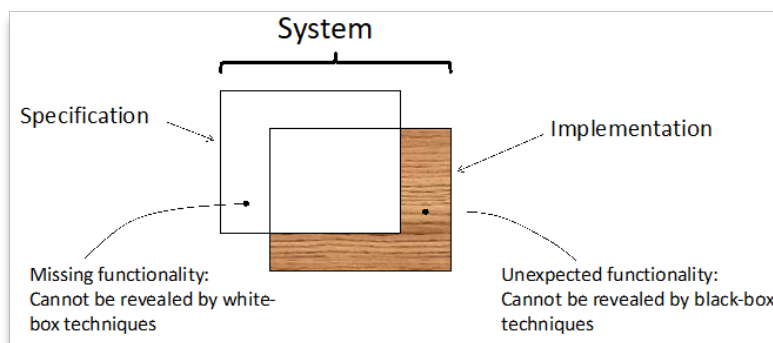
Kiểm thử hộp đen tập trung vào chức năng của hệ thống, dựa trên tài liệu yêu cầu và đặc tả. Người kiểm thử chỉ quan sát đầu vào và đầu ra mà không cần quan tâm đến mã nguồn. Các kỹ thuật thường dùng gồm phân hoạch lớp tương đương, phân tích giá trị biên, kiểm thử chuyển trạng thái,... Phương pháp này giúp phát hiện các lỗi thiếu chức năng so với đặc tả.

Ngược lại, kiểm thử hộp trắng là kiểm thử cấu trúc, dựa trên mã nguồn và cách chương trình được cài đặt. Nó thường áp dụng ở mức kiểm thử đơn vị và kiểm thử tích hợp, do lập trình viên và kỹ sư kiểm thử thực hiện. Các kỹ thuật tiêu biểu gồm kiểm thử luồng điều khiển, kiểm thử nhánh, kiểm thử đường đi, kiểm thử luồng dữ liệu, kiểm thử đột biến và kiểm thử vòng lặp. Phương pháp này cho phép đánh giá độ bao phủ của mã nguồn và phát hiện các chức năng không mong đợi trong quá trình triển khai.



Hình 5.1: So sánh kiểm thử hộp đen và hộp trắng.

Tuy nhiên, kiểm thử toàn diện/trọn vẹn bằng cách sử dụng tất cả các điều kiện và dữ liệu đầu vào là không khả thi trong thực tế. Vì vậy chúng ta cần phải cố gắng lựa chọn được một tập con tối thiểu các ca kiểm thử sao cho các ca kiểm thử đó cho phép xác định nhiều lỗi nhất có thể. Các kỹ thuật thiết kế ca kiểm thử ra đời nhằm hỗ trợ việc này, giúp phát hiện nhiều lỗi nhất với chi phí thấp nhất. Đây chính là động lực cho sự phát triển của các chiến lược kiểm thử.



Hình 5.2: Giới hạn của kiểm thử hộp đen và hộp trắng.

Mỗi phương pháp đều có giới hạn riêng. Hình 5.2 minh họa hai góc nhìn chính: *đặc tả* (specification) và *cài đặt* (implementation). Phần giao nhau giữa chúng là hệ thống thực sự đáp ứng đúng yêu cầu. Tuy nhiên, vẫn có hai loại vấn đề có thể xảy ra:

- **Missing functionality** (thiếu chức năng): tồn tại trong đặc tả nhưng không được cài đặt; những lỗi này không thể phát hiện bằng kỹ thuật hộp trắng.
- **Unexpected functionality** (chức năng không mong đợi): xuất hiện trong

cài đặt nhưng không có trong đặc tả; những lỗi này không thể phát hiện bằng kỹ thuật hộp đen.

Chính vì vậy, trong kiểm thử phần mềm, kỹ thuật kiểm thử hộp đen và kỹ thuật kiểm thử hộp trắng không thể thay thế nhau mà cần được kết hợp để bổ sung, giúp đảm bảo chất lượng hệ thống một cách toàn diện.

5.2 Kiểm thử luồng điều khiển

5.2.1 Các thuật ngữ cơ bản

1. **Đơn vị chương trình (Program unit):** Một đơn vị chương trình được hiểu là một thành phần hay một thực thể nào đó có chương trình có thể thực thi ứng với một đầu vào cụ thể của dữ liệu trả về một giá trị đầu ra hoặc một hành vi nào đó.

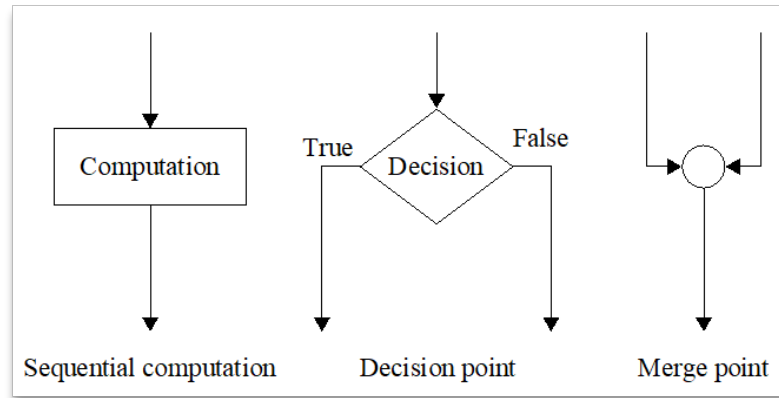
Ví dụ: các thủ tục, các hàm, các mô-đun hay thành phần trong hệ thống đều có thể coi là một đơn vị chương trình

2. **Câu lệnh (Statement):** Có hai loại câu lệnh chính trong chương trình:

- Lệnh gán (*assignment statement*).
- Lệnh điều kiện/ra quyết định (*conditional statement*).

Ví dụ: các lệnh lựa chọn if, switch, các vòng lặp... đều là các lệnh điều kiện

3. **Đường dẫn chương trình (Program path):** Một đường dẫn chương trình là một trình tự các câu lệnh từ điểm bắt đầu đến điểm kết thúc của một đơn vị chương trình có thể thực thi được. Một đơn vị chương trình có thể có nhiều hơn một đường dẫn chương trình.
4. **Đồ thị luồng điều khiển (Control Flow Graph – CFG):** Xuất phát từ khái niệm về đường dẫn chương trình mà chúng ta đưa ra khái niệm đồ thị luồng điều khiển chương trình. CFG thể hiện cấu trúc của một đơn vị chương trình, là tập hợp tất cả các đường dẫn chương trình từ điểm bắt đầu tới điểm kết thúc của chương trình được biểu diễn bằng các khái niệm của đồ thị.



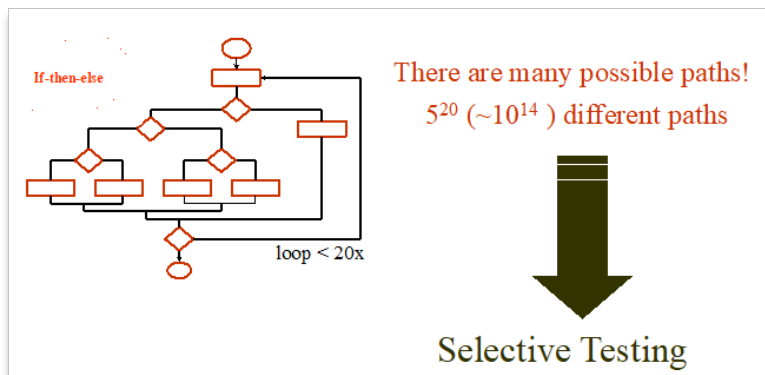
Hình 5.3: Các ký hiệu cơ bản trong đồ thị luồng điều khiển

Chúng ta có 3 khái niệm chính:

- Các câu lệnh tuần tự, lệnh gán được biểu diễn bằng các hình chữ nhật.
- Các câu lệnh quyết định hay điều kiện, vòng lặp sử dụng hình thoi để biểu diễn điều kiện và nhánh rẽ.
- Hình tròn được sử dụng để thể hiện các điểm gộp của chương trình để thực thi các câu lệnh cần đồng bộ.

5.2.2 Kĩ thuật kiểm thử luồng điều khiển

Kiểm thử luồng điều khiển là một kỹ thuật kiểm thử dựa trên cấu trúc của chương trình. Các công cụ kiểm thử tự động hiện nay có thể sinh ra những đường dẫn khác nhau trong đồ thị luồng điều khiển (CFG) của chương trình. Ý tưởng chính của kỹ thuật này là lựa chọn một số ít đường dẫn đại diện nhưng vẫn đảm bảo đánh giá được toàn bộ hành vi của chương trình.



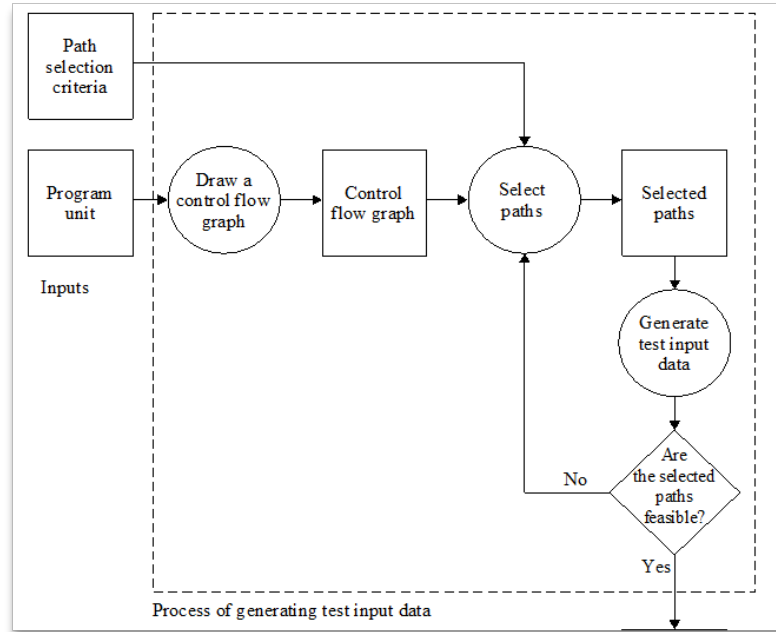
Hình 5.4: Chương trình có thể có rất nhiều đường dẫn cần kiểm thử

Trong thực tế, một đơn vị chương trình có thể có số lượng đường dẫn rất lớn. Ví dụ, với 10^{14} đường dẫn có thể có, việc kiểm thử toàn bộ là không khả thi. Do đó, cần phải lựa chọn các đường dẫn thích hợp nhất để vừa giảm số lượng ca kiểm thử, vừa vẫn đảm bảo độ bao phủ cần thiết.

Để lựa chọn được tập đường dẫn phù hợp, ta sử dụng các tiêu chí khác nhau:

- **Kiểm thử toàn diện (All Path coverage):** lựa chọn tất cả các đường dẫn có thể.
- **Bao phủ câu lệnh (Statement coverage):** lựa chọn đường dẫn sao cho tất cả các lệnh trong chương trình được thực thi ít nhất một lần.
- **Bao phủ nhánh (Branch coverage):** lựa chọn đường dẫn sao cho tất cả các nhánh rẽ trong chương trình đều được kiểm thử.
- **Bao phủ điều kiện (Predicate coverage):** lựa chọn đường dẫn sao cho tất cả các điều kiện logic đều được kiểm thử.

5.2.3 Quy trình kiểm thử luồng điều khiển



Hình 5.5: Quy trình kiểm thử luồng điều khiển.

Ý tưởng chính của việc thực hiện kiểm thử luồng điều khiển được thể hiện trong Hình 5.5. Đầu vào của quá trình này là mã nguồn của chương trình và tiêu chí lựa chọn đường dẫn.

Từ mã nguồn, đồ thị luồng điều khiển (*Control Flow Graph – CFG*) được sinh ra. Sau đó, các đường dẫn được lựa chọn dựa trên các tiêu chí đã đặt ra. Mỗi đường dẫn chỉ có thể thực thi nếu tồn tại một tập dữ liệu đầu vào cụ thể. Do đó, với mỗi đường dẫn, cần phải xác định tập dữ liệu đầu vào thích hợp để đảm bảo đường dẫn đó được thực hiện trong quá trình kiểm thử.

5.2.4 Tiêu chí lựa chọn đường đi

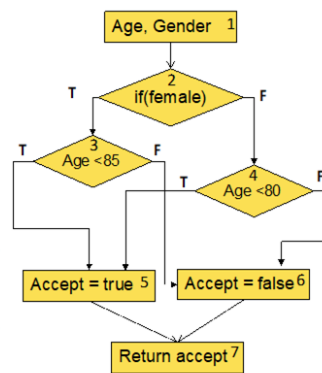
Để làm rõ khái niệm về tiêu chí lựa chọn đường dẫn trong kiểm thử luồng điều khiển, ta xét một ví dụ trong lĩnh vực bảo hiểm nhân thọ. Hàm `AccClient` nhận vào hai tham số: `age` (tuổi) và `gender` (giới tính). Nhiệm vụ của hàm là xác định xem một khách hàng có được chấp nhận tham gia bảo hiểm hay không.

- Nếu giới tính là nữ, điều kiện chấp nhận là tuổi nhỏ hơn 85.
- Nếu giới tính là nam, điều kiện chấp nhận là tuổi nhỏ hơn 80.

Từ đoạn mã nguồn, có thể xây dựng sơ đồ luồng điều khiển (*Control Flow Graph* – *CFG*) để minh họa các quyết định và luồng thực thi trong chương trình. Hình 5.6 cho thấy CFG tương ứng với hàm `AccClient`.

Life Insurance Example

```
bool AccClient(agetype
    age; gndrtype gender)
bool accept
if(gender=female)
    accept := age < 85;
else
    accept := age < 80;
return accept
```



Hình 5.6: Ví dụ minh họa CFG cho hàm `AccClient`.

Tiêu chí lựa chọn tất cả các đường dẫn - All Path Coverage Criteria

Mục tiêu của tiêu chí lựa chọn tất cả các đường dẫn là làm sao để thiết kế tập test case tối thiểu sao cho tất cả các đường dẫn trong đồ thị đều được thực hiện. Với hàm `AccClient`, từ sơ đồ luồng điều khiển có thể xác định được rằng chương trình chỉ có 4 đường dẫn khả dĩ. Mỗi đường dẫn tương ứng với một trường hợp kiểm thử cụ thể dựa trên điều kiện đầu vào (`Female`, `Age<85`, `Age<80`).

	Female	Age < 85	Age < 80
{	Yes	Yes	Yes
	Yes	Yes	No
	Yes	No	Yes
	Yes	No	No
	No	Yes	Yes
{	No	Yes	No
	No	No	Yes
	No	No	No

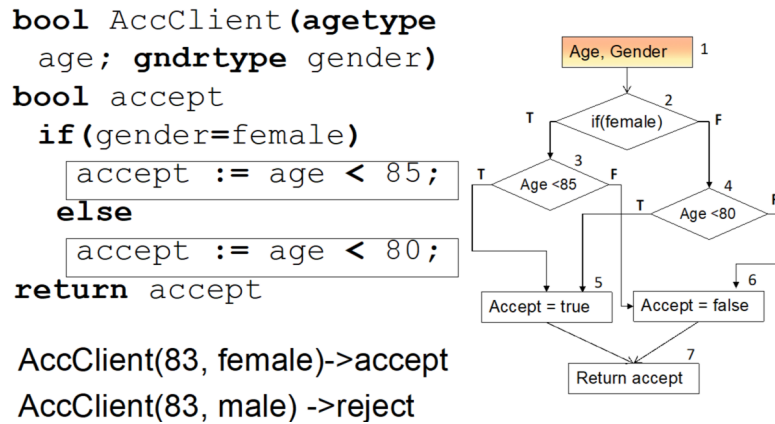
Hình 5.7: Các trường hợp kiểm thử cụ thể dựa trên điều kiện đầu vào (Female, Age<85, Age<80).

- Đường dẫn 1: <Yes, Yes, * > $\rightarrow 1 \rightarrow 2(T) \rightarrow 3(T) \rightarrow 5 \rightarrow 7$.
- Đường dẫn 2: <Yes, No, No> $\rightarrow 1 \rightarrow 2(T) \rightarrow 3(F) \rightarrow 6 \rightarrow 7$.
- Đường dẫn 3: <No, Yes, Yes> $\rightarrow 1 \rightarrow 2(F) \rightarrow 4(T) \rightarrow 5 \rightarrow 7$.
- Đường dẫn 4: <No, *, No> $\rightarrow 1 \rightarrow 2(F) \rightarrow 4(F) \rightarrow 6 \rightarrow 7$.

Như vậy, để đảm bảo tiêu chí **All Path Coverage**, ta cần tối thiểu 4 ca kiểm thử, mỗi ca đại diện cho một đường dẫn trong sơ đồ luồng điều khiển.

Tiêu chí bao phủ hết tập câu lệnh - Statement coverage criterion

Mục tiêu của tiêu chí bao phủ hết tập câu lệnh là đảm bảo tất cả các câu lệnh trong chương trình đều được thực hiện ít nhất một lần. Các ca kiểm thử tối thiểu sẽ được thiết kế sao cho điều này được đảm bảo. Vấn đề đặt ra là trong chương trình có thể có các dead code, đó là những đoạn code không được thực thi, vậy sẽ không tồn tại ca kiểm thử nào cho phép thực hiện các đoạn code này được.



Hình 5.8: Các ca kiểm thử đáp ứng tiêu chí bao phủ hết các tập câu lệnh

Với ví dụ đặt ra ban đầu, tiêu chí bao phủ tập câu lệnh sẽ thoả mãn nếu chúng ta thiết kế được các trường hợp kiểm thử sao cho hai câu lệnh của chương trình ứng với node 5 và 6 trên đồ thị luồng điều khiển thoả mãn.

Tuy nhiên, bao phủ tập lệnh là tiêu chí yếu nhất khi lựa chọn các đường dẫn chương trình. Các lỗi có thể dễ dàng xảy ra trong các ca kiểm thử khác mà tiêu chí này không cho phép xác định. Ví dụ, với bao phủ tập lệnh chỉ chúng ta chỉ cần các ca kiểm thử sao cho các điều kiện True được thoả mãn, do đó sẽ không thể kiểm thử được các ca kiểm thử với điều kiện False.

Tiêu chí bao phủ nhánh - Branch coverage criterion

Tiêu chí thứ ba khi lựa chọn các đường dẫn là bao phủ nhánh rẽ, hay còn gọi là bao phủ quyết định (*Decision Coverage*).

Một nhánh được định nghĩa là một cạnh xuất phát từ một nút trong đồ thị luồng điều khiển.

- Với các nút hình chữ nhật (câu lệnh tuần tự), mỗi nút chỉ có tối đa một nhánh đi ra.
- Với các nút hình thoi (câu lệnh điều kiện), mỗi nút sẽ có hai nhánh đi ra tương ứng với giá trị **true** và **false**.

Trong chương trình, một phần tử quyết định có thể thuộc một trong các dạng: **if-then**, **switch-case**, hoặc **loop**. Ý tưởng chính của tiêu chí này là thiết kế các ca kiểm thử sao cho tất cả các nhánh trong chương trình đều được tham chiếu ít

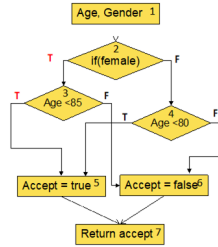
nhất một lần trong tập các đường dẫn được lựa chọn. Nhờ đó, có thể phát hiện các lỗi tiềm ẩn do việc xử lý nhánh chưa đầy đủ.

Ví dụ, để đảm bảo tiêu chí **Branch Coverage**, các ca kiểm thử cần bao phủ tất cả các nhánh rẽ trong đồ thị luồng điều khiển của hàm `AccClient`. Cụ thể, ta cần 4 đường dẫn sau:

Branch Coverage /1

AccClient(83,
female)->accept

```
bool AccClient(agetyp
age; gndrtype gender)
bool accept
if (gender=female)
    accept := age < 85;
else
    accept := age < 80;
return accept
```

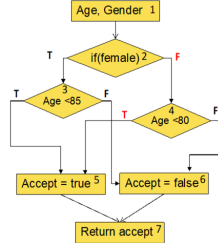


(a) $1 \rightarrow 2(T) \rightarrow 3(T) \rightarrow 5 \rightarrow 7$

AccClient(78, male)-
>accept

Branch Coverage /3

```
bool AccClient(agetyp
age; gndrtype gender)
bool accept
if (gender=female)
    accept := age < 85;
else
    accept := age < 80;
return accept
```

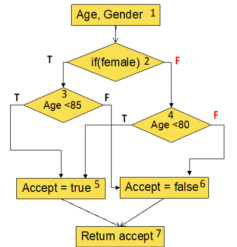


(c) $1 \rightarrow 2(F) \rightarrow 4(T) \rightarrow 5 \rightarrow 7$

Branch Coverage /2

AccClient(83, male)
->reject

```
bool AccClient(agetyp
age; gndrtype gender)
bool accept
if (gender=female)
    accept := age < 85;
else
    accept := age < 80;
return accept
```

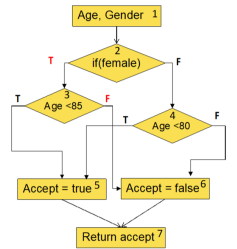


(b) $1 \rightarrow 2(F) \rightarrow 4(F) \rightarrow 6 \rightarrow 7$

AccClient(88,
female) ->reject

Branch Coverage /4

```
bool AccClient(agetyp
age; gndrtype gender)
bool accept
if (gender=female)
    accept := age < 85;
else
    accept := age < 80;
return accept
```



(d) $1 \rightarrow 2(T) \rightarrow 3(F) \rightarrow 6 \rightarrow 7$

Hình 5.9: Ví dụ minh họa 4 ca kiểm thử cho tiêu chí Branch Coverage.

Như vậy, trong ví dụ này, all paths coverage và branch coverage cùng cho số test case là 4. Và có thể thấy rằng branch coverage trong ví dụ này cũng khiến cho tất cả các path được lựa chọn

So sánh ba tiêu chí bao phủ

Ba tiêu chí chính được sử dụng trong kiểm thử luồng điều khiển gồm:

1. **All path coverage**: đảm bảo 100% các đường dẫn trong chương trình được thực thi.

2. **Statement coverage:** lựa chọn đủ các đường dẫn để đảm bảo mọi câu lệnh trong mã nguồn đều được thực hiện ít nhất một lần.
3. **Branch coverage:** đảm bảo rằng mọi nhánh rẽ trong chương trình đều được thực hiện ít nhất một lần bởi một ca kiểm thử nào đó.

Có thể thấy rằng:

- Tiêu chí (1) *All path coverage* bao hàm tiêu chí (3) *Branch coverage*.
- Tiêu chí (3) *Branch coverage* lại bao hàm tiêu chí (2) *Statement coverage*.

Điều này có nghĩa là tập kiểm thử của (1) sẽ chứa tập kiểm thử của (3), và tập kiểm thử của (3) sẽ chứa tập kiểm thử của (2). Ba tiêu chí này thường được gọi chung là **các kỹ thuật kiểm thử đường dẫn** (*Path Testing Techniques*).

5.2.5 Tiêu chí bao phủ điều kiện - Predicate Coverage

Các khái niệm cơ bản

- **Predicate** hay điều kiện là những biểu thức mà giá trị của nó có kiểu *boolean*, tức là **true** hoặc **false** (trong một số ngôn ngữ lập trình không có kiểu *boolean* như C, giá trị này có thể được biểu diễn bởi 0 hoặc 1).
- Một biểu thức điều kiện có thể bao gồm:
 - Các biến mang giá trị **boolean**.
 - Các biến không phải **boolean** nhưng được so sánh bằng các toán tử quan hệ ($\geq, \leq, =, \neq, <, >$).
 - Các lời gọi hàm trả về giá trị **boolean**.
- Cấu trúc bên trong của các biểu thức điều kiện thường được tạo thành bởi các **toán tử logic**, bao gồm: phủ định ($\neg$), suy diễn ($\rightarrow$), tương đương ($\leftrightarrow$), phép AND ($\wedge$), phép OR ($\vee$), và phép XOR ($\oplus$).
- **Clause (mệnh đề)** là một biểu thức điều kiện không chứa bất kỳ toán tử logic nào. Ví dụ: biểu thức $(a = b) \vee C \wedge p(x)$ chứa 3 mệnh đề: quan hệ $(a = b)$, biến boolean C , và lời gọi hàm boolean $p(x)$.
- Ngoài ra, mỗi một đường dẫn trong chương trình sẽ tương ứng với một dãy các giá trị **true/false** cho các biểu thức điều kiện xuất hiện trên đường dẫn đó.

- **Path Predicate Expression** là một biểu thức boolean đặc trưng cho tập các giá trị đầu vào khiến đường dẫn cụ thể được thực thi.

Cách tạo Path Predicate Expression

Để xác định được biểu thức điều kiện của một đường dẫn (*path*), ta cần duyệt từ đầu đến cuối đường dẫn và xác định các điểm quyết định. Tại mỗi điểm quyết định, ta viết lại các biểu thức điều kiện tương ứng.

Kết quả thu được là một tập hợp các biểu thức điều kiện có trong mã nguồn. Tất cả các biểu thức này phải được thoả mãn thì đường dẫn đã chọn mới có thể được thực thi.

```

X1,X2,X3,X4,X5,X6
if (X5 > 0 || X6 < 0)          /* predicates A,B */
...
if(X1 + 3 * X2 + 17 >= 0)      /* predicate C */
...
if(X3 == 17)                  /* predicate D */
...
if(X4 - X1 >= 14 * X2)        /* predicate E */
...
Path Predicate Expression is: (A+B)CDE

```

Hình 5.10: Ví dụ về các biểu thức điều kiện (predicates) trong một đoạn mã.

Các lỗi điều kiện có thể xảy ra

Các lỗi điều kiện có thể xảy ra bao gồm:

- Sử dụng sai toán tử Boolean.
- Sử dụng sai biến Boolean.
- Thiếu hoặc thừa biến Boolean.
- Sử dụng sai toán tử quan hệ.
- Dấu ngoặc (*parentheses*) được sử dụng không chính xác.

Tiêu chí bao phủ điều kiện - Predicate Coverage

Tiêu chí bao phủ điều kiện (Predicate Coverage) nhằm đảm bảo rằng tất cả các điều kiện (predicates) trong mã nguồn đều được kiểm thử đúng đắn.

Ý tưởng chính đó là với mỗi biểu thức điều kiện p , các ca kiểm thử phải đảm bảo rằng p được đánh giá **true** ít nhất một lần và được đánh giá **false** ít nhất một lần.

Ví dụ trên Hình 5.11 minh họa với biểu thức:

$$p = ((a > b) \vee C) \wedge p(x)$$

Trong đó tập dữ liệu kiểm thử được chọn để p có thể nhận cả hai giá trị **true** và **false**.

	a	b	C	p(x)
1	5	4	true	true
2	5	6	false	false

"decision coverage" in literature

Hình 5.11: Ví dụ bao phủ điều kiện (Predicate Coverage).

Để minh họa rõ hơn cho Predicate Coverage, ta xét ví dụ sau: Biểu thức p sẽ được kiểm thử với hai trường hợp để đảm bảo kết quả bao gồm cả **true** và **false**.

$$p = ((a < b) \vee D) \wedge (m \geq n * o)$$

Predicate = true
 $a = 5, b = 10, D = \text{true}, m = 1, n = 1, o = 1$
 $= (5 < 10) \vee \text{true} \wedge (1 \geq 1 * 1)$
 $= \text{true} \vee \text{true} \wedge \text{TRUE}$
 $= \text{true}$

Predicate = false
 $a = 10, b = 5, D = \text{false}, m = 1, n = 1, o = 1$
 $= (10 < 5) \vee \text{false} \wedge (1 \geq 1 * 1)$
 $= \text{false} \vee \text{false} \wedge \text{TRUE}$
 $= \text{false}$

Hình 5.12: Ví dụ minh họa Predicate Coverage với hai trường hợp $p = \text{true}$ và $p = \text{false}$.

Tiêu chí bao phủ mệnh đề - Clause Coverage

Với *Clause Coverage*, chúng ta cần đảm bảo rằng mỗi điều kiện đơn lẻ (mỗi mệnh đề) đều được kiểm thử sao cho nó nhận giá trị **true** và **false** ít nhất một lần.

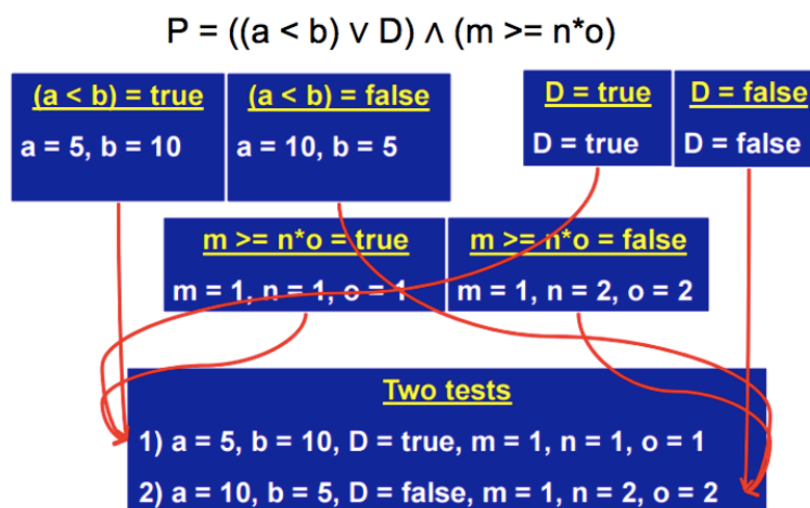
Example: $((a > b) \vee C) \wedge p(x)$

	a	b	C	p(x)
1	5	4	true	true
2	5	6	false	false

"condition coverage" in literature

Hình 5.13: Ví dụ về bao phủ mệnh đề (Clause Coverage)

Tuy nhiên, cần lưu ý rằng *Clause Coverage* **không đảm bảo** toàn bộ Predicate sẽ nhận cả hai giá trị **true** và **false**.



Hình 5.14: Ví dụ về Clause Coverage: đảm bảo mỗi mệnh đề được kiểm thử với cả giá trị **true** và **false**.

Câu hỏi đặt ra: **Predicate Coverage** có bao hàm **Clause Coverage** hay **ngược lại**? Ví dụ với $p = a \vee b$, ta có bảng Giá trị chân lý như hình 5.18

	a	b	$a \vee b$
1	T	T	T
2	T	F	T
3	F	T	T
4	F	F	F

Hình 5.15: Truth table của $p = a \vee b$

Từ ví dụ này có thể thấy không phải lúc nào PC cũng bao hàm CC và ngược lại. Rõ ràng là các test cases chưa chắc thoả mãn clause coverage sẽ đảm bảo predicate nhận giá trị cả true và false và ngược lại. Do đó chúng ta cần một cách tiếp cận khác để có thể kiểm thử được tất cả các điểm quyết định và mối quan hệ logic giữa chúng. Một cách tự nhiên, chúng ta muốn kiểm thử được từng giá trị true false của các điều kiện đơn lẻ và kiểm thử được giá trị true false của Predicate.

Tiêu chí bao phủ kết hợp - Combinatorial Coverage

Ý tưởng chính: Với mỗi vị từ/biểu thức điều kiện p , các ca kiểm thử cần bảo đảm rằng các mệnh đề (clause) cấu thành p được đánh giá qua *mọi tổ hợp giá trị chân lý* (đúng/sai) có thể có.

Example: $(a \vee b) \wedge c$

CoC requires every possible combination

	a	b	c	$(a \vee b) \wedge c$
1	T	T	T	T
2	T	T	F	F
3	T	F	T	T
4	T	F	F	F
5	F	T	T	T
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Hình 5.16: CoC yêu cầu kiểm thử tất cả các tổ hợp giá trị chân lý của các mệnh đề a, b, c .

Bài tập 1:

Yêu cầu sinh viên liệt kê tất cả các mệnh đề (clauses) và các tổ hợp mệnh đề (Combination of Clauses – CoC) của biểu thức điều kiện sau:

$$P = ((a > b) \vee C) \wedge p(x)$$

Vậy vấn đề của bao phủ kết hợp là gì? Nếu chúng ta có n mệnh đề độc lập, chúng ta sẽ có 2^n sự kết hợp có thể của các mệnh đề. Dẫn đến số lượng trường hợp test cho CoC là rất lớn. Do đó sẽ không khả thi khi có nhiều mệnh đề. Như vậy, biểu thức điều kiện (predicate) cần được kiểm thử sao cho có thể cho ra cả hai giá trị đúng và sai. Mỗi điều kiện thành phần (clause) phải được kiểm thử một cách độc lập với nhau, và mỗi mệnh đề thành phần cần có ảnh hưởng đến toàn bộ biểu thức, tức là khi thay đổi giá trị của một mệnh đề thì giá trị của biểu thức cũng thay đổi theo.

5.2.6 Mệnh đề chủ động

Khái niệm

Chúng ta gọi một mệnh đề là quan trọng nếu nó quyết định giá trị của predicate. Một mệnh đề c xác định predicate p nếu với một giá trị cụ thể của tất cả các mệnh đề khác, nếu thay đổi giá trị của c sẽ làm giá trị của p thay đổi.



Hình 5.17: Ví dụ cho phép and và phép or logic.

Lưu ý sinh viên rằng: với mỗi predicate, chúng ta lần lượt cho một mệnh đề là active clause, chúng ta cần xác định giá trị của các mệnh đề còn lại sao cho giá trị của mệnh đề đó không ảnh hưởng gì đến giá trị của Predicate (i.e., giá trị của predicate được quyết định chỉ bởi mệnh đề active clause). Ví dụ với phép *and*: $P = a \text{ and } b$, a là active clause thì b có giá trị = true, tương tự b là active clause thì a có giá trị bằng true.

Lấy ví dụ cho biểu thức điều kiện của P :

$$P = a \wedge (b \vee c)$$

Yêu cầu: Sinh viên xác định, với từng vai trò *active clause* của a, b, c , giá trị cần phải gán cho các mệnh đề phụ là gì?

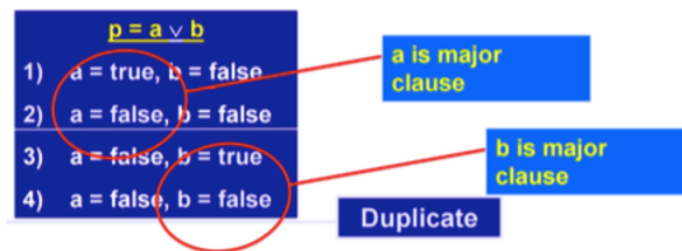
Ví dụ: khi a là *active clause*, ta cần $b \vee c = 1$. Khi đó, a sẽ quyết định giá trị của P . Có thể gán bất kỳ giá trị nào cho b và c miễn sao $b \vee c = 1$, chẳng hạn:

$$(b = 0, c = 1) \quad \text{hoặc} \quad (b = 1, c = 1) \quad \text{hoặc} \quad (b = 1, c = 0).$$

Làm tương tự khi b hoặc c đóng vai trò là *active clause*.

Tiêu chí bao phủ mệnh đề chủ động - Active Clause Coverage (ACC)

Bao phủ các mệnh đề chủ động yêu cầu rằng, với mỗi biểu thức điều kiện p , chúng ta xác định các *major clause* và chọn các giá trị cần thiết của *minor clause* sao cho mệnh đề chủ động quyết định giá trị của p . Khi đó, để đảm bảo bao phủ các mệnh đề chủ động, ta chỉ cần làm sao cho mỗi mệnh đề chủ động nhận cả hai giá trị *true* và *false* ít nhất một lần.



Hình 5.18: Ví dụ cho phép toán logic OR.

Trong ví dụ với phép toán logic OR, ta thấy rằng khi $b = \text{false}$, thì a sẽ quyết định giá trị của P , do đó a là mệnh đề chủ động. Vì vậy cần 2 test case để a nhận cả giá trị *true* và *false*. Khi $a = \text{false}$, thì b trở thành mệnh đề chủ động, khi đó b xác định P . Chúng ta cần thêm 2 test case để b nhận cả *true* và *false*. Tuy nhiên, test case thứ 4 sẽ bị trùng lặp, do đó không cần sử dụng. Vậy, để đảm bảo **ACC**, chúng ta chỉ cần 3 test case.

Câu hỏi: Tại sao với bao phủ mệnh đề chính, tại mỗi điểm quyết định trong code có N điều kiện, chúng ta chỉ cần $N + 1$ test case?

```

public static void printHonorRollStatus(double cumulativeGPA,
    double termGPA, int creditsCompleted, boolean fullTimeStatus) {
    // Determine if the student is on the deans list.
    if ((creditsCompleted > 30) && (cumulativeGPA > 3.20)
        && (fullTimeStatus == true) && (termGPA > 2.0)) {
        System.out.println("You are on the dean's list.");
    } else if ((creditsCompleted > 30) && (cumulativeGPA > 3.70)
        && (fullTimeStatus == true) && (termGPA > 2.0)) {
        System.out.println("You are on the high honors dean's list.");
    } else if ((creditsCompleted > 30) && (cumulativeGPA > 2.0)
        && (fullTimeStatus == true) && (termGPA > 3.2)) {
        System.out.println("You are on the honor list.");
    } else {

```

Hình 5.19: Mã nguồn chương trình

Bài tập: Cho ví dụ code trên slide, yêu cầu sinh viên thực hành **ACC** với predicate đầu tiên.

Cần lưu ý rằng với các đoạn code như vậy, **Predicate Coverage (PC)** và **Clause Coverage (CC)** thường chỉ có ý nghĩa về mặt lý thuyết. Trong thực tế, khi chương trình chạy, hầu hết các ngôn ngữ lập trình đều giản lược việc tính toán các điều kiện phía sau nếu điều kiện trước đó đã xác định kết quả. Ví dụ, trong biểu thức $a \ \&\& \ b$, nếu $a = \text{false}$ thì b sẽ không hề được kiểm tra. Chính vì vậy, **ACC** ra đời để đảm bảo rằng mỗi điều kiện riêng lẻ đều được cho cơ hội quyết định giá trị của Predicate ít nhất một lần.

Giải quyết vấn đề nhập nhằng Khi xác định được một mệnh đề là xác định điều kiện, chúng ta cần lựa chọn các giá trị của mệnh đề còn lại để đảm bảo mệnh đề đó xác định điều kiện p . Tuy nhiên làm thế nào để lựa chọn giá trị của các mệnh đề đó, có tiêu chí gì không? Câu hỏi đó đã làm các kỹ sư kiểm thử “đau đầu” nhiều năm. Chúng ta đưa ra 3 tiêu chí để lựa chọn giá trị của các mệnh đề minor.

General Active Clause Coverage (GACC)

GACC là trường hợp đơn giản nhất, không có bất kỳ một ràng buộc nào cho các minor clause, ngoại trừ việc phải đảm bảo cho major clause quyết định giá trị của predicate.

	a	b	c	$a \wedge (b \vee c)$
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Hình 5.20: Bảng giá trị chân lý

Với ví dụ trên hình, yêu cầu sinh viên lựa chọn từng mệnh đề a, b, c là active clause.

Khi a là active clause, thì giá trị của a sẽ quyết định giá trị của P, hay nói cách khác khi a thay đổi giá trị thì P cũng phải thay đổi giá trị theo. Vậy khi a là active clause, a = True, P = true hỏi sinh viên xem b và c phải nhận những giá trị nào?

Đáp án: $b \vee c = \text{true}$ do đó row 1, 2, 3 thỏa mãn. Tương tự với trường hợp a = false, P = false xác định giá trị của b và c. GACC không bao phủ Predicate, ví dụ với biểu thức logic $P = a \leftrightarrow b$, GACC thỏa mãn với 2 test case TT, FF và Predicate chỉ nhận mỗi giá trị là T.

Bài tập: Sinh viên hãy xây dựng bảng chân lý cho: $P = ((a > b) \vee C) \wedge p(x)$ xác định các hàng thỏa mãn ACC, và thảo luận rằng với GACC không cần các minor clause khác nhau.

Xác định các mệnh đề phụ (minor clauses)

Khi biểu thức điều kiện (predicate) đơn giản, việc xác định giá trị của các mệnh đề phụ không khó. Tuy nhiên, khi predicate phức tạp thì không hề dễ dàng. Theo cách tiếp cận định nghĩa, ta có: $P(c = \text{true})$ là biểu thức thu được khi thay mọi lần xuất hiện của c bằng true, và $P(c = \text{false})$ là biểu thức thu được khi thay mọi lần xuất hiện của c bằng false.

Để tìm giá trị cho các mệnh đề phụ, ta giải biểu thức logic:

$$P(c = \text{true}) \oplus P(c = \text{false})$$

Ví dụ: $p = a \vee b$ Với biểu thức $p = a \vee b$, ta có:

$$p_a = p_{a=\text{true}} \oplus p_{a=\text{false}} = (\text{true} \vee b) \oplus (\text{false} \vee b) = \text{true} \oplus b = \neg b$$

$$p_b = p_{b=\text{true}} \oplus p_{b=\text{false}} = (a \vee \text{true}) \oplus (a \vee \text{false}) = \text{true} \oplus a = \neg a$$

Kết luận: p_a là điều kiện để a là mệnh đề chủ động, nghĩa là b phải bằng *false*. Tương tự, p_b là điều kiện để b là mệnh đề chủ động, nghĩa là a phải bằng *false*.

Ví dụ: $p = a \wedge (b \vee c)$ Với biểu thức $p = a \wedge (b \vee c)$, ta có:

$$p_a = p_{a=\text{true}} \oplus p_{a=\text{false}} = (\text{true} \wedge (b \vee c)) \oplus (\text{false} \wedge (b \vee c)) = (b \vee c) \oplus \text{false} = b \vee c$$

Kết luận: a là mệnh đề chủ động khi ít nhất một trong hai b hoặc c bằng *true* (các trường hợp: TT, TF, FT).

Theo định nghĩa của GACC, thoả mãn GACC sẽ đảm bảo mỗi mệnh đề được test với giá trị *true* và *false* tuy nhiên không đảm bảo PC được thoả mãn. Lấy ví dụ a tương đương b, P chỉ nhận giá trị *true* khi a và b cùng *true* hoặc cùng *false* ngược lại P = *false*. Khi đó tập test case thoả mãn GACC là 2 test cases. Tuy nhiên trong tất cả các trường hợp thì P đều luôn luôn là *True* hoặc *False*. Do đó ko thoả mãn PC. Ở đây chúng ta lấy giá trị của mệnh đề phụ là khác nhau với hai trường hợp mệnh đề chính là *true* và *false*. Lưu ý sinh viên, có những trường hợp mà chúng ta bắt buộc phải lấy giá trị của mệnh đề phụ là giống nhau khi mệnh đề chính nhận giá trị là *true* hoặc *false*. Ví dụ: $P = a \text{ and } b$. a là active clause khi $b = 1$. Tức là khi $b = 1$ thì giá trị của a sẽ quyết định giá trị của P. Hay thay đổi giá trị của a sẽ làm giá trị của P thay đổi.

Correlated Active Clause Coverage (CACC)

Với mỗi mệnh đề c_i trong P , ta chọn các mệnh đề phụ c_j sao cho c_i quyết định giá trị của P . Mệnh đề c_i phải được kiểm thử với cả hai giá trị *true* và *false*.

Các giá trị được chọn cho các mệnh đề phụ c_j phải làm cho P nhận giá trị *true* khi c_i nhận một giá trị và nhận giá trị *false* khi c_i nhận giá trị còn lại. Điều kiện là:

$$P(c_i = \text{true}) \neq P(c_i = \text{false})$$

Các mệnh đề phụ không nhất thiết phải giống nhau trong hai trường hợp. Nói cách khác:

$$c_j(c_i = \text{true}) = c_j(c_i = \text{false}), \quad \forall c_j$$

hoặc

$$c_j(c_i = \text{true}) \neq c_j(c_i = \text{false}), \quad \forall c_j$$

Bài tập: Khi a là *active clause*, hãy trả lời các câu hỏi sau:

- Những hàng nào có thể được chọn để thoả mãn CACC?
- Những hàng nào thoả mãn khi $a = \text{true}$ và khi $a = \text{false}$?
- CACC = GACC ?

	a	b	c	$a \wedge (b \vee c)$
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Hình 5.21: Bảng chân lý

CACC và sự bao hàm của GACC/CC/PC

Theo định nghĩa, **CACC** bao hàm **GACC**, do đó cũng bao hàm **CC** và cả **PC**.

Ví dụ với $p = (a \Leftrightarrow b)$:

- Hai test case thoả mãn **PC**: $(a = \text{true}, b = \text{true})$, $(a = \text{true}, b = \text{false})$.
- Ba test case thoả mãn **CACC**: $(a = \text{true}, b = \text{true})$, $(a = \text{true}, b = \text{false})$, $(a = \text{false}, b = \text{true})$.

Như vậy, **CACC** bao phủ **PC**, **CC** và cả **GACC**.

p = (a ↔ b)

For major clause a

row	a	b	p	p _a	p _b
1	T	T	T	T	T
2	T	F	F	T	T
3	F	T	F	T	T
4	F	F	T	T	T

p_a = T

a = T

p_a = T

a = F

p evaluates to T and F

Only (1,3) and (2,4) satisfy CACC

Hình 5.22: Ví dụ minh họa CACC và PC với $p = (a \leftrightarrow b)$ **Bài tập:** Xét biểu thức

$$p = (\neg a \wedge \neg b) \vee (a \wedge \neg c) \vee (\neg a \wedge c).$$

Hãy vẽ bảng chân lý cho p (như trên slide). Sau đó, lần lượt cho a, b, c là *active clause* và xác định các giá trị có thể gán cho các *minor clause* để mệnh đề chủ động quyết định p . Cuối cùng, xác định tập các test case thoả mãn **GACC** và tập các test case thoả mãn **CACC**.

Restricted Active Clause Coverage (RACC)

Với mỗi biểu thức P và mỗi mệnh đề chủ động c_i , chọn các mệnh đề phụ c_j sao cho c_i quyết định giá trị của P . Khi đó, c_i phải được kiểm thử với cả giá trị *true* và *false*, và P phải nhận giá trị *true* trong một trường hợp và *false* trong trường hợp còn lại (như trong CACC). Điểm khác biệt của **RACC** là các mệnh đề phụ c_j phải giữ nguyên giá trị khi $c_i = \text{true}$ và khi $c_i = \text{false}$. Đây là phiên bản nghiêm ngặt hơn của CACC.

Câu hỏi: Khi a là *active clause*, những hàng nào có thể được chọn để thoả mãn **RACC**?

	a	b	c	$a \wedge (b \vee c)$
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Hình 5.23: Bảng chân lý cho $P = a \wedge (b \vee c)$

Lưu ý:

- Hàng 1 và 8 không được chọn vì trong hai trường hợp này a không là *active*.
- Các giá trị của b và c phải giống nhau khi $a = \text{true}$ cũng như khi $a = \text{false}$.
- Một trong ba cặp $\{1, 5\}$, $\{2, 6\}$, $\{3, 7\}$ sẽ thoả mãn **RACC**.

RACC và sự bao hàm của **CACC**/**GACC**/**PC**/**CC**

Theo định nghĩa, **RACC** bao hàm **CACC**, do đó cũng bao hàm **GACC**, **CC** và **PC**.

Tuy nhiên, **RACC** thường dẫn đến các yêu cầu kiểm thử *không khả thi*.

Ví dụ: Xét biểu thức điều kiện

$$P = ((a > b) \wedge (b < c)) \vee (c > a) \equiv (X \wedge Y) \vee Z.$$

Khi Z là mệnh đề chủ động, ta chọn $X = 1, Y = 0$. Trong trường hợp $Z = 0, X = 1, Y = 0$ ta sẽ thu được một *infeasible test case*.

Bài tập: Xét biểu thức

$$P = a \wedge (b \Leftrightarrow c).$$

Yêu cầu sinh viên thực hành:

- Vẽ bảng chân lý cho biểu thức P .
- Lần lượt cho a, b, c là *active clause*, xác định các giá trị có thể gán cho các *minor clause*.
- Ví dụ: khi a là *active clause*, điều kiện $b \Leftrightarrow c = 1$, tức là $b = c = 1$ hoặc $b = c = 0$.
- Tương tự, phân tích cho các trường hợp b và c là *active clause*.
- Cuối cùng, xác định các test case thoả mãn **GACC** và các test case thoả mãn **CACC**.

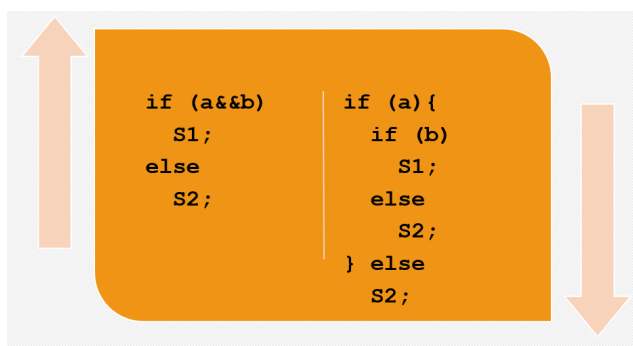
Logical Operators in Source Code

- $\&$ | $\mid$ | $\sim$ | $\wedge$ cũng là các toán tử logic.
- Sự khác nhau giữa $(a\&\&b)$ và $(a\&b)$? $(a\mid\mid b)$ và $(a\mid b)$?

Logical expression	Java expression
$a \wedge b$	<code>a && b</code>
$a \vee b$	<code>a b</code>
$\neg a$	<code>!a</code>
$a \rightarrow b$	<code>a ? b : true;</code>
$a \leftrightarrow b$	<code>a == b</code>
$a \oplus b$	<code>a != b</code>

Code transformation

Cách code các biểu thức logic cũng ảnh hưởng đến số lượng test case khi thực hiện các tiêu chuẩn bao phủ.



Hình 5.24: Code transformation

Ví dụ với đoạn code bên trái:

- Với **Branch Coverage** chỉ cần 2 test case: (a, b) và $(\neg a, b)$.
- Với **ACC** cần 3 test case: (a, b) , $(a, \neg b)$ và $(\neg a, b)$.

Ví dụ với đoạn code bên phải:

- Với **Branch Coverage** cần 3 test case: (a, b) , $(a, \neg b)$, $(\neg a, b)$ hoặc (a, b) , $(a, \neg b)$, $(\neg a, \neg b)$.
- Với **ACC** vẫn cần 3 test case.

Rõ ràng, với **Branch Coverage** thì đoạn code bên phải kiểm thử tốt hơn đoạn code bên trái, còn với **ACC** thì số lượng test case yêu cầu là như nhau.

Bài tập 1: Cho đoạn mã nguồn hàm `isLeapYear` như sau:

```
public static boolean isLeapYear(int y) {  
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);  
}
```

Hình 5.25: Mã nguồn hàm `isLeapYear`

Yêu cầu sinh viên xác định các test case thoả mãn các tiêu chí sau:

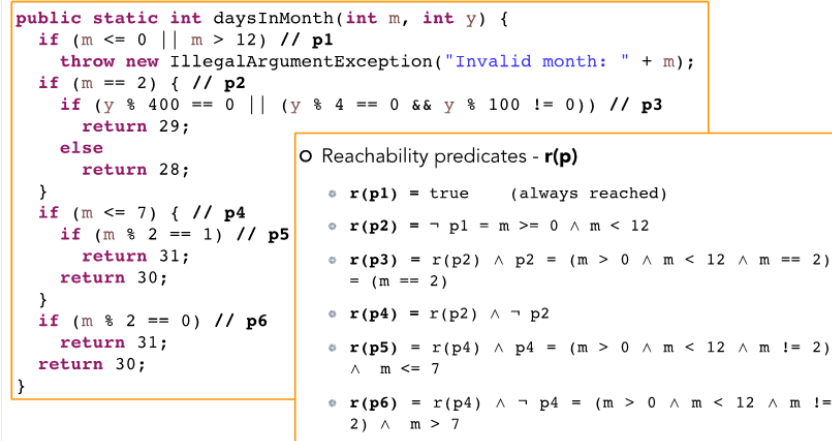
- PC
- CC
- CoC
- GACC
- CACC
- RACC

Bài tập 2: Cho đoạn mã nguồn hàm `daysInMonth` như trên slide.

Yêu cầu sinh viên:

- Xác định các *predicate* và *clause* trong mã nguồn.
- Xác định các *predicate* có thể đạt tới (reachable) trong mã nguồn.

- Đưa ra các test case thoả mãn **PC**, **CC** và **CoC**.
- Xem xét có yêu cầu kiểm thử nào là *không khả thi* (infeasible) hay không.



Hình 5.26: Mã nguồn hàm `daysInMonth` và các predicate, clause

Bài tập 3: Cho đoạn mã nguồn hàm `triangleType(int a, int b, int c)` như trên slide.

Yêu cầu sinh viên:

1. Xác định các *predicate* có thể đạt tới (reachability predicates).
2. Xác định tập yêu cầu kiểm thử theo **TR(CC)** và **TR(PC)**.
3. Đưa ra các test case thoả mãn:
 - a) PC
 - b) CC
4. Xác định các *determination predicates* cho các mệnh đề (clauses) của $p1$ và $p2$.
5. Xác định tập yêu cầu kiểm thử theo **TR(CACC)**.
6. Đưa ra các test case thoả mãn:
 - a) CACC
 - b) RACC (có yêu cầu nào là *infeasible* không?)

```

public static TClass triangleType(int a, int b, int c) {
    if (a <= 0 || b <= 0 || c <= 0) /* p1 */
        return INVALID;
    if (a >= b + c || b >= a + c || c >= a + b) /* p2 */
        return INVALID;
    int count = 0;
    if (a == b) /* p3 */
        count++;
    if (a == c) /* p4 */
        count++;
    if (b == c) /* p5 */
        count++;
    if (count == 0) /* p6 */
        return SCALENE;
    if (count == 1) /* p7 */
        return ISOSCELES;
    return EQUILATERAL;
}

```

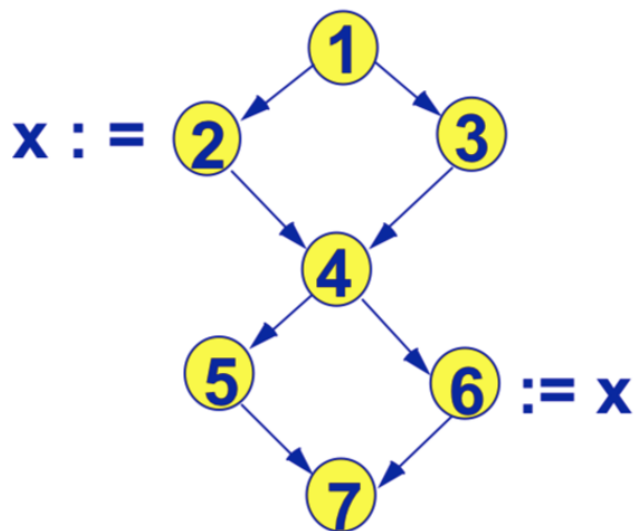
○ Identify:

- ⊗ 1) the reachability predicates;
- ⊗ 2) TR(CC) and TR(PC)
- ⊗ 3) test cases that satisfy a) PC b) CC
- ⊗ 4) determination predicates for the clauses of p1 and p2
- ⊗ 5) TR(CACC)
- ⊗ 6) test cases that satisfy a) CACC b) RACC [are there infeasible requirements?]

Hình 5.27: Mã nguồn hàm triangleType

5.3 Kiểm thử luồng dữ liệu

5.3.1 Động lực



Hình 5.28: Ví dụ về câu lệnh gán

Xét ví dụ trên, tại câu lệnh thứ 2, x được định nghĩa, hay nói cách khác x được gán một giá trị. Tại câu lệnh thứ 6, x được sử dụng, hay nói cách khác x được dùng để gán giá trị cho một biến khác. Nếu sử dụng bao phủ quyết định hay bao phủ nhánh, chúng ta sẽ có 2 ca kiểm thử thoả mãn là 1-2-4-5-7 và 1-3-4-6-7. Trong trường hợp này chúng ta không thể kiểm thử được đồng thời việc định nghĩa biến x và việc sử dụng x . Mục tiêu của bao phủ dữ liệu để xét xem các giá trị được tính toán đúng và sử dụng đúng hay không. Chúng ta sẽ phải xem xét dữ liệu được truy cập thế nào và được thay đổi ra sao trong hệ thống và xác định được những giá trị dữ liệu làm hệ thống không hoạt động. Hầu hết các lỗi tiềm ẩn của chương trình liên quan đến việc sử dụng dữ liệu ra sao, và các lỗi đó thường rất khó nắm bắt và đôi khi hệ thống phát sinh lỗi mà chúng ta phải rất lâu mới khám phá ra vì lỗi liên quan đến dữ liệu. Một số lỗi thường gặp: Sử dụng một biến chưa được định nghĩa hoặc chưa được khởi tạo. Do đó giá trị của biến thông thường là được gán một cách ngẫu nhiên bởi trình biên dịch. Ví dụ C Khởi tạo lại một biến trước khi nó được xây dựng, khởi tạo và sử dụng. Xoá một tập hợp các đối tượng sẽ khiến cho các thành viên của tập hợp không thể truy cập được.

5.3.2 Các định nghĩa

Định nghĩa biến

Biến được định nghĩa bất cứ khi nào giá trị bị thay đổi. Ví dụ, trong lệnh gán giá trị của biến, trong câu lệnh đọc vào dữ liệu, hoặc trong tham số truyền kiểu tham chiếu trong lời gọi hàm của ngôn ngữ lập trình C/C++

Sử dụng biến

Một biến được sử dụng bất cứ khi nào giá trị của nó được đọc. Ví dụ trên biểu thức phía bên phải của một câu lệnh, bởi việc truyền tham trị vào lời gọi hàm hoặc trong các biểu thức điều kiện của các câu lệnh rẽ nhánh.

Có hai cách sử dụng biến:

- Sử dụng biến trong các điều kiện.
- Sử dụng biến trong các câu lệnh khác.

```
if (x > 0) {
    print(y);
}
```

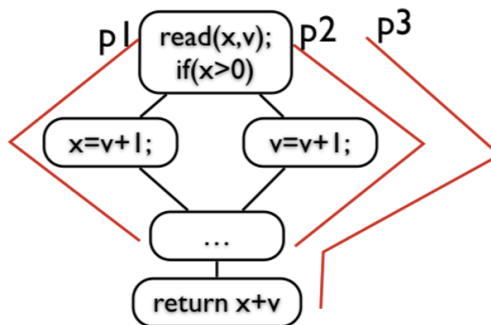
Hình 5.29: Mã nguồn

Ví dụ, trong đoạn code phía trên, chúng ta thấy x được sử dụng là p-use và y được sử dụng là c-use. Một biến có thể vừa được sử dụng và vừa được định nghĩa lại đồng thời trong một câu lệnh. Ví dụ: ở cả hai phía của câu lệnh gán chúng ta có thể vừa sử dụng biến vừa định nghĩa biến. Hoặc trong lời gọi hàm tham số truyền vào kiểu tham chiếu của ngôn ngữ lập trình C. Trong hàm `increment(y)` sẽ có lệnh $y = y + 1$ do đó, y vừa được sử dụng vừa được định nghĩa.

Các thuật ngữ khác

Một đường định nghĩa rõ ràng - def-clear path

Một đường định nghĩa rõ ràng của một biến v là một đường trên đồ thị luồng điều khiển sao cho không có một node nào tại đó v được định nghĩa lại.



Hình 5.30: Ví dụ minh họa

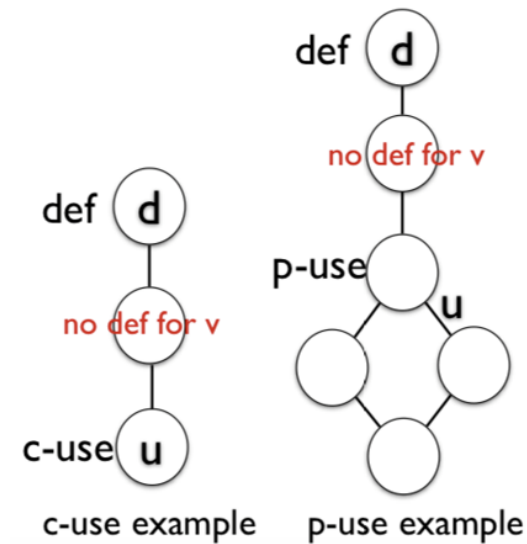
Ví dụ, $p1$ là def-clear path của v nhưng $p2$ thì không vì v được định nghĩa lại ở một node trên $p2$.

Đường đầy đủ - complete path

Đường đầy đủ nếu node bắt đầu chính là node entry của đồ thị và node kết thúc là node exit của đồ thị luồng điều khiển $p1$ và $p2$ không phải là đường đầy đủ trong

khi p3 là đường đầy đủ.

Cặp định nghĩa-sử dụng - du-pair



Hình 5.31: Ví dụ minh họa

Cặp định nghĩa-sử dụng của một biến v là một cặp (d,u) sao cho d là node tại đó v được định nghĩa hoặc định nghĩa lại, u là node hoặc cạnh tại đó có sử dụng v . Khi việc sử dụng v là p-use (tức là v được sử dụng trong điều kiện của một lệnh rẽ nhánh) thì u là cạnh xuất phát ra từ chính câu lệnh đó hay nhánh rẽ của điểm quyết định tại đó v được sử dụng. Với một cặp d,u thì luôn luôn tồn tại một def-clear path từ d tới u .

Bài tập: Ví dụ cho đoạn code như trong hình. Yêu cầu sinh viên vẽ đồ thị luồng điều khiển và ghi rõ các node tại đó có sử dụng hoặc định nghĩa biến. Sau đó liệt kê tất cả các du-pair của biến A và các path tương ứng.

```
1. input (A,B)
   if (B>1) {
2.   A = A + 7
   }
3. if (A > 10) {
4.   B = A + B
   }
5. output (A,B)
```

Hình 5.32: Mã nguồn chương trình

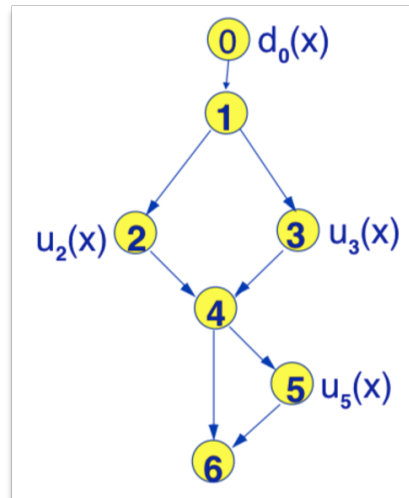
5.3.3 Các tiêu chí kiểm thử bao phủ luồng dữ liệu

- All-defs coverage – Bao phủ tất cả các điểm định nghĩa dữ liệu trên đồ thị.
- All-uses coverage – Bao phủ tất cả các điểm sử dụng dữ liệu trên đồ thị
- All-DU-Paths coverage – Bao phủ tất cả các đường dẫn DU trên đồ thị
- All-p-uses/Some-c-uses coverage – Bao phủ tất cả các điểm sử dụng dữ liệu trong các câu lệnh rẽ nhánh và một vài điểm sử dụng dữ liệu
- All-c-uses/Some-p-uses coverage – Bao phủ tất cả các điểm sử dụng dữ liệu và một vài điểm sử dụng dữ liệu trong các câu lệnh rẽ nhánh
- All-p-uses coverage – Bao phủ tất cả các loại sử dụng dữ liệu trong điều kiện
- All-c-uses coverage – Bao phủ tất cả các loại sử dụng thông thường của dữ liệu

Đó là các thuật ngữ quan trọng trong phần kiểm thử dựa trên luồng dữ liệu. Sinh viên cần nắm được và phân biệt được p-use, c-use, du pair trước khi vào phần tiếp theo là tìm hiểu về các tiêu chí kiểm thử luồng dữ liệu. Cũng giống như luồng điều khiển có các tiêu chí về kiểm thử đường dẫn và kiểm thử điều kiện, kiểm thử luồng dữ liệu cũng có một vài tiêu chí căn cứ vào mục đích muốn bao phủ cái gì mà sử dụng loại tiêu chí phù hợp. Cụ thể có 7 loại tiêu chí bao phủ luồng dữ liệu liệt kê như trên slide. Chúng ta sẽ tập trung vào 3 loại đầu tiên. 4 loại còn lại sẽ được nhắc đến để sinh viên nắm được khái niệm.

All-Defs Coverage

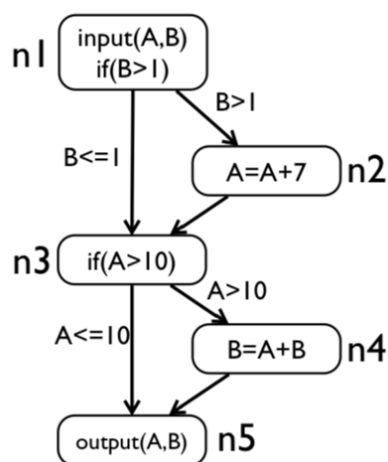
Với tất cả các biến trong chương trình, cần đảm bảo ít nhất một đường dẫn định nghĩa rõ ràng từ mọi điểm định nghĩa của biến đến ít nhất một điểm c-use hoặc một điểm p-use của biến được kiểm thử. Tức là chúng ta sẽ thực hiện bao phủ hết tất cả các đường từ 1 điểm biến được định nghĩa đến 1 điểm biến được sử dụng (chỉ cần ít nhất 1 điểm biến được sử dụng).



Hình 5.33: Biểu đồ luồng điều khiển.

Trong ví dụ trên, chỉ có duy nhất một biến x . Để *All-defs* được thoả mãn thì chỉ cần chọn một đường dẫn tới một điểm *use* của x , ví dụ: $d_0(x) \rightarrow u_2(x)$.

Trở lại ví dụ trước đó, hãy trả lời câu hỏi: với một đường dẫn đầy đủ $\langle 1, 2, 3, 4, 5 \rangle$ liệu tiêu chí All-defs có thoả mãn không? Yêu cầu sinh viên xác định tất cả các def-clear paths trên đường dẫn $\langle 1, 2, 3, 4, 5 \rangle$ cho variable A và variable B.



Hình 5.34: Đồ thị luồng điều khiển của chương trình.

Từ đó yêu cầu sinh viên trả lời câu hỏi ban đầu. Nhận xét rằng: đường dẫn $\langle 1, 2, 3, 4, 5 \rangle$ bao phủ ít nhất 1 def-clear path cho mọi định nghĩa của A và B đến 1 điểm use tương ứng của A và B. Vậy tiêu chí All-Defs thoả mãn.

All-Uses Coverage - Bao phủ tất cả các điểm sử dụng

Với mọi biến chương trình v , cần có ít nhất một đường đi *def-clear* từ mỗi định nghĩa của v tới mọi *c-use* và mọi *p-use* (bao gồm tất cả các cạnh xuất phát từ câu lệnh điều kiện) của v . Yêu cầu rằng tất cả các cặp *du-pair* phải được bao phủ. Xét hai đường thực thi:

- $t_2 : \langle 1, 3, 4, 5 \rangle$
- $t_3 : \langle 1, 2, 3, 5 \rangle$

Câu hỏi: Liệu cả ba test case t_1, t_2, t_3 có đảm bảo *All-Uses coverage* hay không? Yêu cầu sinh viên xác định các def-clear path trên đường dẫn t_2 cho variable A và variable B. Yêu cầu sinh viên xác định các def-clear path trên đường dẫn t_3 cho variable A.

Kết quả: Vì tất cả các cặp *du-pairs* đều được bao phủ nên tiêu chí All-Uses thoả mãn.

Bài tập: Cho ví dụ đoạn code như sau. Yêu cầu sinh viên vẽ sơ đồ luồng điều khiển và xác định tất cả các cặp DU-Pairs cho biến X và biến Y.

```

1. input(X,Y)
2. while (Y>0) {
3.   if (X>0)
4.     Y := Y-X
5.   else
6.     input(X)
7. }
8. output(X,Y)

```

Hình 5.35: Mã nguồn chương trình

Một số thuật ngữ và định nghĩa về Dataflow

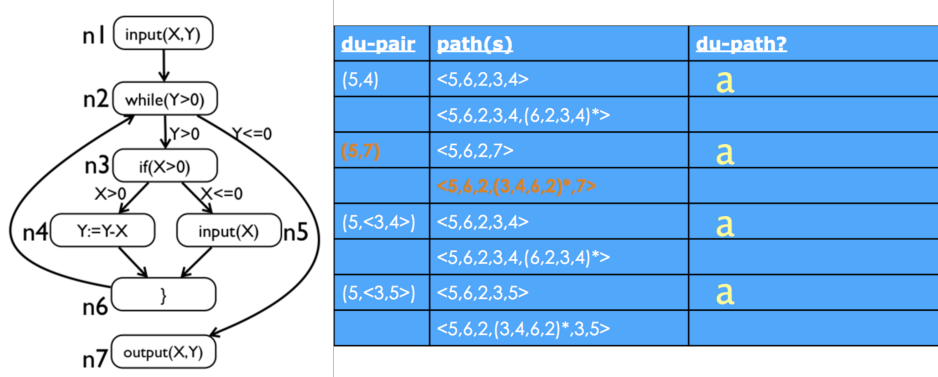
- Một đường đi (có thể là một phần hoặc toàn bộ) được gọi là **đơn giản (simple)** nếu tất cả các cạnh trong đường đi đều khác nhau.
- Một đường đi được gọi là **không vòng lặp (loop-free)** nếu tất cả các nút trong đường đi đều khác nhau.

Câu hỏi: Hỏi sinh viên path nào là simple và path nào là loop-free theo định nghĩa?

path	Simple?	Loop-free?
<1,3,4,2>	a	a
<1,2,3,2>	a	
<1,2,3,1,2>		
<1,2,3,2,4>	a	

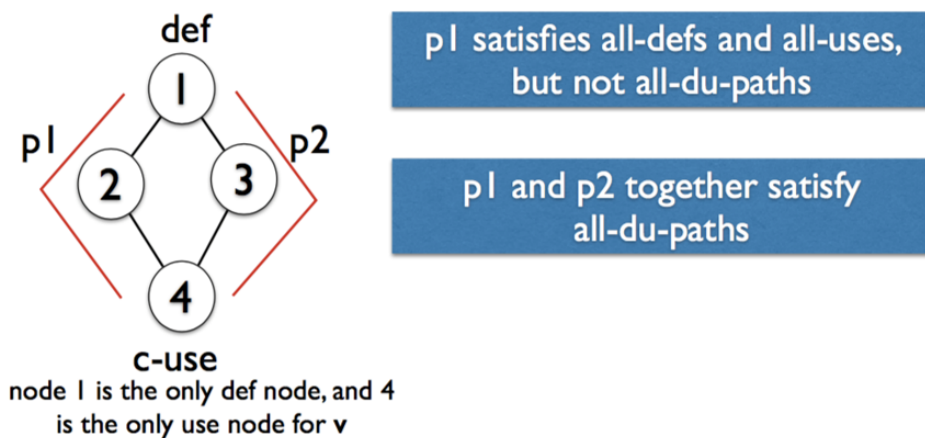
- Một DU-Path được định nghĩa là một chuỗi các nốt trên đồ thị luồng, tại đó biến được định nghĩa tại nốt đầu tiên và có 1 c-use của biến tại nốt cuối cùng và chuỗi đường dẫn đó là một simple-path. Tại nốt sát cuối cùng của DU-Path cũng có thể là một p-use của biến, khi đó đường dẫn trước đó phải là một loop-free path.

Câu hỏi: Với các paths như trên bảng, hãy xác định path nào thoả mãn là du-path và du-path nào có nốt cuối là c-use và du-path nào có node cuối là p-use?



All DU-paths Coverage

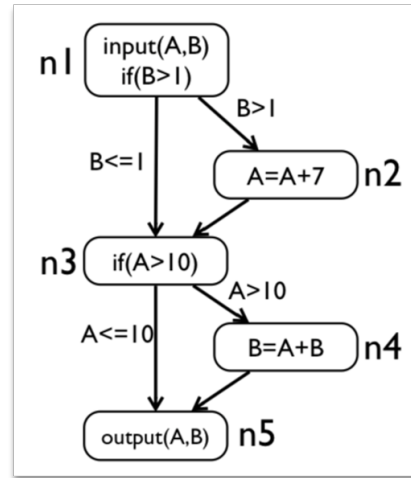
Với mỗi biến chương trình v , mọi *du-path* từ mỗi định nghĩa của v tới mọi *c-use* và mọi *p-use* của v đều phải được bao phủ.



Hình 5.36: Minh họa quan hệ all-defs, all-uses và all-du-paths

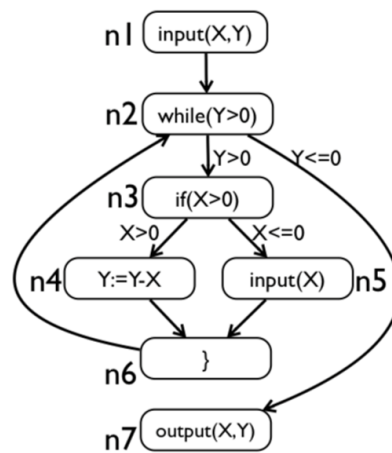
- p1 thoả mãn *all-defs* và *all-uses*, nhưng không thoả mãn *all-du-paths*.
- p1 và p2 kết hợp lại thì thoả mãn *all-du-paths*.

Ví dụ 1: Yêu cầu sinh viên xác định tất cả DU-Paths cho variable A và B



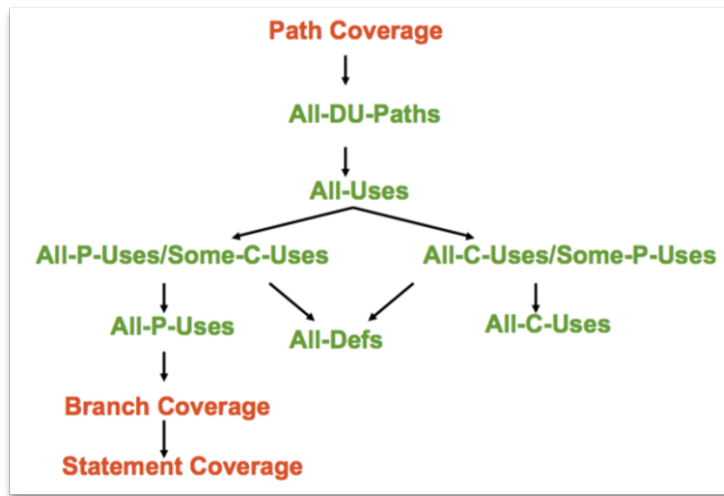
Hình 5.37: Biểu đồ luồng điều khiển của chương trình

Ví dụ 2: Yêu cầu sinh viên xác định tất cả DU-Paths cho variable X và Y



Hình 5.38: Biểu đồ luồng điều khiển của chương trình

Tóm tắt



Hình 5.39: Mối quan hệ giữa data flow testing và path testing

Giữa Data flow testing và path testing có liên hệ với nhau. Ví dụ branch coverage và p-use có thể coi là bao phủ của nhau. Cấu trúc cây trên slide biểu diễn cấp độ bao phủ của các tiêu chí kiểm thử trong kiểm thử đường dẫn và kiểm thử luồng dữ liệu. Rất nhiều kinh nghiệm thực tế đã cho thấy rằng, với kiểm thử code, sự kết hợp của kiểm thử luồng điều khiển với tiêu chí bao phủ nhánh và kiểm thử luồng dữ liệu với tiêu chí bao phủ tất cả các điểm sử dụng là một sự kết hợp vô cùng tốt cho việc thiết kế các ca kiểm thử.

Bài tập 1: Cho mã nguồn như sau, yêu cầu sinh viên thực hiện:

```

1 public class BinarySearch {
2     public int binsearch(int x, int[] V, int n) {
3         int low, high, mid;
4         low = 0;
5         high = n - 1;
6         while (low <= high) {
7             mid = (low + high) / 2;
8             if (x < V[mid])
9                 high = mid - 1;
10            else if (x > V[mid])
11                low = mid + 1;
12            else
13                return mid;
14        }
15        return -1;
16    }
17 }
  
```

Hình 5.40: Mã nguồn chương trình

- Vẽ đồ thị luồng dữ liệu (*data flow graph*) cho hàm `binsearch`.

- Tìm một tập các đường đi đầy đủ thoả mãn tiêu chí *all-defs* đối với biến `mid`.
- Thực hiện tương tự cho biến `high`.

Bài tập 2: Cho mã nguồn như sau, yêu cầu sinh viên:

```
1 public class Pow {
2     public void pow(int x, int y) {
3         float z;
4         int p;
5         if (y < 0)
6             p = 0 - y;
7         else p = y;
8         z = 1.0f;
9         while (p!=0)
10        {
11            z = z * x;
12            p = p - 1;
13        }
14        if (y < 0)
15            z = 1.0f/z;
16        System.out.println(z);
17    }
18 }
```

Hình 5.41: Mã nguồn chương trình

- Sử dụng tiêu chí *All-DU paths* để kiểm thử hàm `pow`.
- Sử dụng tiêu chí *All-Uses* kết hợp với *Branch Coverage* để kiểm thử hàm này.

Chương 6

Độ tin cậy phần mềm

6.1	Định nghĩa độ tin cậy phần mềm - Software Reliability	125
6.2	Mô hình toán học cho độ tin cậy phần mềm	131
6.3	Các nhân tố ảnh hưởng đến độ tin cậy phần mềm	133
6.4	Hồ sơ vận hành - Operational Profile	138

6.1 Định nghĩa độ tin cậy phần mềm - Software Reliability

Độ tin cậy phần mềm (software reliability) được định nghĩa là xác suất mà một hệ thống phần mềm có thể vận hành mà không xảy ra lỗi trong một khoảng thời gian xác định, dưới một môi trường thực thi cụ thể. Đây là một trong những thuộc tính quan trọng của chất lượng phần mềm, phản ánh khả năng hoạt động ổn định và đúng đắn của hệ thống trong điều kiện thực tế.

Dựa trên định nghĩa trên, độ tin cậy phần mềm phụ thuộc vào ba yếu tố chính:

- **Xác suất vận hành không lỗi (Probability of failure-free operation):**
Phản ánh khả năng mà hệ thống hoạt động chính xác trong mỗi lần chạy. Đây là yếu tố cốt lõi thể hiện tính chính xác và ổn định của phần mềm.
- **Khoảng thời gian vận hành không lỗi (Specified time of operation):**
Thời gian mà phần mềm được yêu cầu hoạt động liên tục mà không phát sinh lỗi. Đây có thể là giờ, ngày, hoặc số lần sử dụng.

- **Môi trường thực thi xác định (Specified operating environment):**
Điều kiện cụ thể mà phần mềm được triển khai và sử dụng như phần cứng, hệ điều hành, mạng, dữ liệu đầu vào, v.v. Môi trường này ảnh hưởng lớn đến khả năng sinh lỗi của hệ thống.

Từ đó, việc đánh giá và kiểm thử độ tin cậy phần mềm đòi hỏi phải xác định rõ cả ba yếu tố trên để đảm bảo kết quả đo lường có ý nghĩa và có thể tái lập trong thực tế.

Độ tin cậy phần mềm thường được biểu diễn dưới dạng một **biến ngẫu nhiên liên tục**, bởi vì trong thực tế, hầu hết các hệ thống lớn đều tiềm ẩn một số lỗi và có thể gặp sự cố tại bất kỳ thời điểm nào trong quá trình vận hành. Tính ngẫu nhiên này phản ánh sự không chắc chắn về thời điểm xảy ra lỗi, cũng như mức độ phức tạp của hệ thống phần mềm.

Yếu tố **thời gian vận hành không lỗi** đóng vai trò đặc biệt quan trọng trong định nghĩa độ tin cậy phần mềm. Người dùng thường thực thi các tác vụ trong một khoảng thời gian xác định và kỳ vọng rằng hệ thống có thể hoạt động ổn định trong suốt khoảng thời gian đó.

Ví dụ: Một hệ thống phần mềm được sử dụng để quản lý hoạt động của một cửa hàng lớn, trong đó cửa hàng mở cửa từ 8 giờ sáng đến 8 giờ tối. Người chủ cửa hàng kỳ vọng hệ thống có thể vận hành liên tục và không phát sinh lỗi trong toàn bộ thời gian mở cửa — tức là trong vòng **12 giờ liên tục**. Do đó, độ tin cậy của hệ thống trong trường hợp này không chỉ được đánh giá dựa trên khả năng tránh lỗi, mà còn phải được xem xét trong bối cảnh thời gian cụ thể của phiên làm việc.

Điều này nhấn mạnh rằng việc đánh giá độ tin cậy phần mềm không thể tách rời khỏi yếu tố thời gian và ngữ cảnh sử dụng thực tế.

Ví dụ minh họa:

Giả sử một hệ thống phần mềm được cài đặt trên máy tính cá nhân của một nhân viên văn phòng. Nhân viên này làm việc hàng ngày từ 8h30 sáng đến 4h30 chiều, tức là trong vòng **8 giờ liên tục** mỗi ngày. Như vậy, kỳ vọng của người dùng là hệ thống có thể hoạt động ổn định và không phát sinh lỗi trong suốt 8 giờ làm việc mỗi ngày.

Giả định rằng nhân viên làm việc **200 ngày trong một năm**, và trong khoảng thời gian đó, hệ thống chỉ gặp sự cố (chẳng hạn như bị treo, crash hoặc lỗi nghiêm trọng) **5 lần** vào các ngày khác nhau. Từ đó, ta có thể ước lượng **xác suất để hệ thống vận hành không lỗi trong một ngày làm việc 8 giờ** như sau:

$$P = \frac{\text{Số ngày không lỗi}}{\text{Tổng số ngày làm việc}} = \frac{200 - 5}{200} = \frac{195}{200} = 0.975 = 97.5\%$$

Kết quả này được xem như một **ước lượng thực nghiệm** cho độ tin cậy của phần mềm trong một phiên làm việc 8 giờ. Nó cho thấy rằng, với xác suất 97.5%, hệ thống có khả năng vận hành liên tục mà không phát sinh lỗi trong suốt một ngày làm việc thông thường.

Yếu tố thứ ba trong định nghĩa độ tin cậy phần mềm – môi trường thực thi xác định – đóng vai trò rất quan trọng. Môi trường thực thi phản ánh cách thức mà người dùng tương tác và sử dụng hệ thống phần mềm trong thực tế. Điều này bao gồm các đặc điểm như loại tác vụ được thực hiện, khối lượng dữ liệu xử lý, cấu hình phần cứng, hệ điều hành, và thói quen sử dụng của người dùng.

Không phải tất cả người dùng đều sử dụng hệ thống theo cùng một cách. Ví dụ, với phần mềm Microsoft Word, có thể tồn tại một nhóm người dùng chỉ thao tác với các tài liệu nhỏ (khoảng 100 trang), trong khi một nhóm khác lại thường xuyên làm việc với các tài liệu lớn (hơn 1000 trang). Rõ ràng, hai nhóm người dùng này vận hành hệ thống trong những môi trường rất khác nhau.

Đối với tài liệu lớn, hệ thống có thể cần sử dụng nhiều tài nguyên hơn (chẳng hạn như bộ nhớ hoặc CPU), dẫn đến thời gian xử lý lâu hơn và nguy cơ xảy ra lỗi hoặc treo máy cũng cao hơn. Ngược lại, việc thao tác với tài liệu nhỏ thường ít tạo áp lực lên hệ thống và có xác suất lỗi thấp hơn.

Do đó, khi đánh giá độ tin cậy phần mềm, việc xác định rõ môi trường thực thi là điều thiết yếu. Nó giúp phản ánh chính xác ngữ cảnh sử dụng thực tế và là cơ sở quan trọng để đo lường và cải thiện độ tin cậy. Khái niệm về “môi trường thực thi xác định” chính là sự mô hình hóa cách mà phần mềm được sử dụng trong các tình huống cụ thể và đa dạng của người dùng.

Khái niệm này đóng vai trò thiết yếu đối với độ tin cậy của phần mềm. Giả sử phần mềm có 10 functions $f_1, \dots, f_{10}$ với hai nhóm người dùng: nhóm 1 chỉ sử dụng các functions từ f_1 đến f_7 và nhóm còn lại sử dụng tất cả các phần mềm. Chúng ta hãy coi rằng các functions f_1 đến f_7 không lỗi, nhưng lỗi tiềm ẩn ở các f_8, f_9 , và f_{10} .

Như vậy chúng ta có thể nói rằng với nhóm đầu tiên hệ thống chẳng bao giờ gặp lỗi bởi vì cách người dùng sử dụng hệ thống không làm phát sinh bất kỳ một lỗi nào trên các thành phần của hệ thống. Độ tin cậy của phần mềm do đó là 1.

Xác suất xảy ra lỗi của phần mềm đối với nhóm thứ 2 phụ thuộc vào việc nhóm này sử dụng các thành phần f_8, f_9, f_{10} thường xuyên thế nào.

Vì vậy môi trường thực thi hệ thống phần mềm đóng một vai trò vô cùng quan

trọng đối với việc xác định độ tin cậy của hệ thống phần mềm.

Mật độ thất bại là một số đo của độ tin cậy của một hệ thống phần mềm vận hành trong một môi trường xác định. Như vậy, mật độ thất bại càng thấp thì độ tin cậy sẽ càng cao.

Để xác định độ tin cậy của hệ thống chúng ta đơn giản xác định số thất bại trong một đơn vị thời gian của hệ thống. Ví dụ: quan sát một hệ thống phần mềm trong pha test hệ thống, người ta thấy rằng có 2 thất bại trên 8h vận hành của hệ thống. Do đó chúng ta có thể chỉ ra độ tin cậy của hệ thống là 0.25 failure per hour.

Định nghĩa đầu tiên nhấn mạnh sự quan trọng của việc hệ thống vận hành suôn sẻ trong một khoảng thời gian tối thiểu để hoàn thành một giao dịch nào đó. Ở đây, độ tin cậy được tính bằng tỉ lệ các giao dịch được thực thi thành công của hệ thống trên tổng số các giao dịch. Giả sử chúng ta có một bài toán về một robot quan sát môi trường nước biển, giả sử robot mất 3 ngày để thực hiện một vòng quan sát. chúng ta sẽ mong muốn hệ thống phần mềm cài đặt trên robot vận hành liên tục không lỗi trong tối thiểu 3 ngày. Nếu hệ thống hoàn thành 99% tổng số vòng quan sát, chúng ta có thể nói rằng độ tin cậy của hệ thống sẽ là 0.99.

Định nghĩa thứ 2 đơn giản yêu cầu rằng hệ thống có ít thất bại nhất có thể. Với định nghĩa này, rủi ro để hệ thống có thể bị fail tại bất cứ thời điểm vận hành nào đều có ý nghĩa quan trọng. Điều này đặc biệt quan trọng với các phần mềm quản lý đường bay của máy bay tại các sân bay. Đối với những hệ thống này, việc hệ thống vận hành suôn sẻ trong khoảng thời gian bao lâu không quan trọng. Mà việc xuất hiện một failure tại một thời điểm nào đó mới đặc biệt quan trọng, bởi vì việc thất bại của hệ thống quản lý đường bay của một sân bay mà bị lỗi có thể dẫn tới một thảm họa thảm khốc.

Lỗi phần mềm và các khái niệm liên quan - Failures, Faults

Lỗi phần mềm (Software Failure):

Lỗi phần mềm là hiện tượng hệ thống không thực hiện đúng như mong đợi theo đặc tả (specification), hoặc thể hiện một hành vi bất thường mà người dùng có thể quan sát thấy trong quá trình sử dụng phần mềm. Đây là biểu hiện cụ thể của sự cố trong hoạt động của phần mềm, thường xảy ra khi một lỗi bên trong chương trình được kích hoạt trong một môi trường thực thi cụ thể.

Ví dụ: Khi người dùng nhấn nút “In hóa đơn” nhưng phần mềm không phản hồi, hoặc hóa đơn được in ra với thông tin sai lệch (sai tên khách hàng, sai tổng tiền, thiếu mặt hàng, v.v.), thì đó được xem là một lỗi phần mềm. Lỗi này gây ảnh hưởng trực tiếp đến trải nghiệm và sự tin cậy của người dùng đối với hệ thống.

Sai sót phần mềm (Software Fault): Sai sót phần mềm là nguyên nhân được xác định hoặc giả định là đã dẫn đến lỗi phần mềm trong quá trình vận hành. Fault có thể tồn tại dưới nhiều dạng khác nhau như:

- Đoạn mã lập trình sai cú pháp hoặc sai logic.
- Thiết kế sai lệch so với yêu cầu ban đầu.
- Việc hiểu sai đặc tả trong quá trình phát triển phần mềm.

Nói cách khác, fault là một phần không chính xác trong hệ thống phần mềm mà, khi được thực thi trong một điều kiện nhất định, có thể gây ra lỗi (software failure). Một fault có thể tồn tại trong hệ thống mà không bị phát hiện cho đến khi nó được kích hoạt bởi một tình huống đầu vào cụ thể hoặc trạng thái hệ thống nhất định.

Ví dụ: Câu lệnh điều kiện bị viết sai hoặc xử lý sai dữ liệu đầu vào gây ra lỗi hiển thị.

Phân biệt giữa Failure và Fault:

- Failure là hiệu ứng (effect) mà người dùng nhìn thấy.
- Fault là nguyên nhân (cause) bên trong hệ thống dẫn đến hiệu ứng đó.

Nói cách khác, fault là nguồn gốc kỹ thuật gây ra failure.

Khiếm khuyết (Defect): Defect là một thuật ngữ tổng quát dùng để chỉ cả lỗi phần mềm (failure) và sai sót bên trong hệ thống (fault). Một defect có thể biểu hiện thành lỗi trong quá trình vận hành, nhưng cũng có thể được phát hiện và khắc phục sớm trong quá trình phát triển trước khi gây ra hậu quả. Không phải tất cả các lỗi quan sát được trong hệ thống đều xuất phát từ một fault hoặc bug cụ thể trong mã nguồn. Trong nhiều trường hợp, lỗi phần mềm có thể đơn giản là kết quả của quá trình vận hành sai do điều kiện cấu hình thiếu, môi trường triển khai không đầy đủ hoặc sự nhầm lẫn trong quá trình sử dụng. Điều này dẫn đến một khái niệm quan trọng: lỗi (error) là hành động của con người dẫn đến một kết quả sai (a human action that produces an incorrect result).

Một defect có thể biểu hiện thành lỗi hoặc có thể được phát hiện trước khi gây ra lỗi.

Không phải tất cả các failure của hệ thống thì đều gây ra bởi một fault hay một bug nào đó, có thể đơn giản là một sự vận hành sai do các điều kiện cấu hình bị thiếu. Như vậy lỗi lại là hành động của con người dẫn tới kết quả sai (a human action that produces an incorrect result).

Chúng ta có thể đặt ra một số câu hỏi phản biện tại bất kỳ thời điểm nào trong vòng đời phát triển hệ thống để đánh giá độ tin cậy hiện tại của phần mềm. Việc trả lời những câu hỏi này trong giai đoạn phát triển cho phép nhóm phát triển xác định các điểm yếu tiềm ẩn, từ đó đề xuất các biện pháp cải tiến nhằm nâng cao độ tin cậy trước khi phần mềm được triển khai chính thức. Trái lại, nếu các câu hỏi này được đặt ra trong giai đoạn vận hành hệ thống, chúng sẽ giúp đánh giá chất lượng triển khai thực tế, mức độ ổn định và khả năng đáp ứng kỳ vọng của người dùng cuối. Do đó, việc đặt đúng câu hỏi vào đúng thời điểm không chỉ hỗ trợ kiểm soát chất lượng, mà còn đóng vai trò quan trọng trong việc đảm bảo độ tin cậy lâu dài của phần mềm.

Các khoảng thời gian:

Thời gian thực thi (Execution Time): Thời gian thực thi (Execution Time) là khoảng thời gian CPU thực sự được sử dụng để thực thi phần mềm. Trong bối cảnh đánh giá hiệu suất và độ tin cậy của hệ thống, thời gian thực thi là một khái niệm quan trọng vì nó phản ánh mức độ sử dụng tài nguyên xử lý một cách trực tiếp. Khác với tổng thời gian chạy của chương trình, thời gian thực thi chỉ tính phần thời gian mà CPU dành để thực sự xử lý mã nguồn của phần mềm, không bao gồm các khoảng thời gian chờ, thời gian nhập/xuất (I/O) hoặc các hoạt động hệ thống khác không liên quan đến quá trình xử lý lệnh. Việc đo lường thời gian thực thi giúp đánh giá hiệu quả thuật toán, tối ưu hóa hiệu suất và xác định các điểm nghẽn trong quá trình vận hành phần mềm.

Thời gian lịch biểu (Calendar Time): Thời gian lịch biểu là khoảng thời gian được đo lường theo đơn vị thời gian mà con người thường sử dụng và cảm nhận, chẳng hạn như năm, tháng, tuần, ngày, giờ hoặc phút. Đây là khái niệm thời gian theo thực tế ngoài đời (real-world time), thường được dùng để lập kế hoạch, quản lý tiến độ dự án hoặc đánh giá các mốc thời gian vận hành phần mềm trong thực tế sử dụng. Trong các phân tích liên quan đến độ tin cậy và bảo trì phần mềm, thời gian lịch biểu giúp phản ánh chu kỳ hoạt động của hệ thống theo quan điểm của người dùng, thay vì chỉ tính riêng thời gian thực thi trên máy tính.

Thời gian đồng hồ (Clock Time): Thời gian đồng hồ là khoảng thời gian thực tế trôi qua từ thời điểm bắt đầu đến thời điểm kết thúc quá trình thực thi phần mềm trên máy tính, như được đo bằng đồng hồ thông thường. Đây là thời gian toàn phần mà người dùng cảm nhận được trong suốt quá trình chương trình chạy. Thời gian đồng hồ bao gồm cả thời gian thực thi của CPU, thời gian chờ do hệ điều hành, thời gian I/O, và bất kỳ độ trễ nào khác trong hệ thống. Trong kiểm thử hiệu năng hoặc đo lường độ trễ thực tế của phần mềm, thời gian đồng hồ là chỉ

số phản ánh rõ ràng nhất trải nghiệm của người dùng cuối.

Từ khoảng thời gian giữa các lần thất bại (failure), chúng ta có thể suy luận ra một số đặc tính liên quan đến độ tin cậy của hệ thống phần mềm. Độ dài hay ngắn của khoảng thời gian này không mang tính tuyệt đối mà phụ thuộc vào bối cảnh sử dụng cũng như kỳ vọng của người dùng. Ví dụ, nếu một hệ điều hành bị crash chỉ một lần trong vòng một năm, phần lớn người dùng sẽ vẫn đánh giá hệ thống đó là đáng tin cậy. Tuy nhiên, nếu cũng với tần suất một lần thất bại mỗi năm, nhưng xảy ra trên một hệ thống kiểm soát không lưu tại sân bay – nơi yêu cầu độ chính xác và liên tục rất cao – thì hệ thống đó có thể bị xem là hoàn toàn không đáng tin cậy. Điều này cho thấy rằng độ tin cậy không chỉ được xác định dựa trên tần suất lỗi, mà còn phụ thuộc vào tính chất nhiệm vụ và mức độ quan trọng của phần mềm trong bối cảnh ứng dụng thực tế.

Khi một thất bại (failure) xảy ra, hệ thống cần một khoảng thời gian để được sửa chữa và khôi phục lại trạng thái hoạt động bình thường. Trung bình của tất cả các khoảng thời gian sửa chữa này được gọi là MTTR (Mean Time To Repair) — thời gian trung bình để khắc phục sự cố.

Giả sử rằng trong suốt thời gian sửa chữa, hệ thống hoàn toàn ngừng hoạt động và không thể sử dụng được. Khi việc sửa chữa hoàn tất, hệ thống quay trở lại trạng thái bình thường và tiếp tục hoạt động cho đến khi xảy ra thất bại tiếp theo. Thời gian trung bình mà hệ thống hoạt động không lỗi trước khi gặp một failure mới được gọi là MTTF (Mean Time To Failure).

Tổng thời gian trung bình giữa hai lần xuất hiện liên tiếp của failure — bao gồm cả thời gian hoạt động bình thường (MTTF) và thời gian sửa chữa (MTTR) — được gọi là MTBF (Mean Time Between Failures). Công thức liên hệ giữa các đại lượng này là:

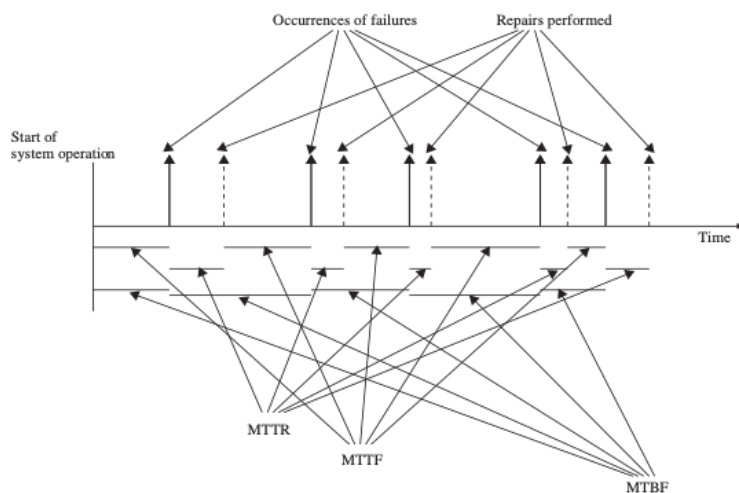
$$MTBF = MTTF + MTTR$$

Chỉ số MTBF thường được sử dụng để đo lường độ tin cậy tổng thể của hệ thống trong các môi trường yêu cầu tính sẵn sàng cao.

Mối quan hệ giữa các khoảng thời gian được thể hiện ở hình vẽ sau:

6.2 Mô hình toán học cho độ tin cậy phần mềm

Một cách tiếp cận phổ biến trong việc xây dựng mô hình toán học cho độ tin cậy phần mềm là theo dõi và đếm số lượng lỗi (failures) đã được quan sát trong quá trình vận hành hệ thống. Dữ liệu này sau đó được sử dụng để xây dựng biểu đồ theo thời gian, qua đó phản ánh xu hướng thay đổi về độ tin cậy của phần mềm.



Hình 6.1: Mối quan hệ giữa MTTR, MTTF, MTBF

Cụ thể, biểu đồ thể hiện tổng số lỗi tích lũy theo thời gian có thể cung cấp nhiều thông tin giá trị. Nếu đường biểu diễn lỗi tích lũy có xu hướng tăng dần đều hoặc tăng nhanh, điều đó cho thấy vẫn còn nhiều sai sót (faults) tiềm ẩn trong hệ thống và hệ thống chưa đạt đến mức độ ổn định mong muốn. Ngược lại, khi biểu đồ dần ổn định hoặc phẳng lại, điều này có thể cho thấy rằng phần lớn các lỗi đã được phát hiện và khắc phục, độ tin cậy của hệ thống đang được cải thiện.

Hơn nữa, tốc độ tăng của biểu đồ — tức là độ dốc của đường lỗi tích lũy — phản ánh tốc độ xuất hiện lỗi trong quá trình sử dụng thực tế. Một tốc độ tăng nhanh có thể là dấu hiệu cảnh báo về chất lượng phần mềm thấp hoặc quy trình phát triển chưa hiệu quả. Do đó, việc phân tích xu hướng biểu đồ lỗi tích lũy là một công cụ quan trọng trong việc đánh giá và dự đoán độ tin cậy phần mềm theo thời gian.

Đây là một phương pháp phổ biến trong đo lường và phân tích độ tin cậy phần mềm (software reliability):

Một trong những phương pháp phổ biến trong đo lường và phân tích độ tin cậy phần mềm là sử dụng biểu đồ số lỗi tích lũy (cumulative failure graph). Đây là một công cụ trực quan hữu ích giúp theo dõi và đánh giá chất lượng hệ thống phần mềm trong suốt quá trình vận hành hoặc kiểm thử.

Trong biểu đồ này:

- Trục tung biểu diễn tổng số lỗi đã được phát hiện kể từ thời điểm bắt đầu quan sát.
- Trục hoành có thể biểu diễn thời gian thực thi phần mềm (execution time)

hoặc thời gian lịch (calendar time), tùy thuộc vào ngữ cảnh sử dụng và cách thu thập dữ liệu.

Biểu đồ số lỗi tích lũy cho phép người phát triển hoặc kiểm thử dễ dàng nhận biết tốc độ phát hiện lỗi, xu hướng ổn định của hệ thống, và đánh giá liệu phần mềm đang tiến gần đến mức độ tin cậy mong muốn hay chưa. Nếu đồ thị có xu hướng chững lại theo thời gian, điều đó cho thấy số lượng lỗi còn lại trong hệ thống có thể đang giảm dần — một tín hiệu tích cực về độ ổn định của phần mềm.

Ý nghĩa của biểu đồ: Biểu đồ số lỗi tích lũy không chỉ là công cụ trực quan để theo dõi quá trình phát hiện lỗi, mà còn mang lại nhiều thông tin quan trọng về mức độ ổn định và độ tin cậy của hệ thống phần mềm trong từng giai đoạn.

- Nếu biểu đồ có **độ dốc lớn** (tăng nhanh theo thời gian), điều đó cho thấy nhiều lỗi được phát hiện trong một khoảng thời gian ngắn. Đây thường là dấu hiệu cho thấy hệ thống vẫn còn chứa nhiều lỗi tiềm ẩn, chưa được phát hiện hết, hoặc phần mềm chưa được kiểm thử đầy đủ.
- Ngược lại, nếu biểu đồ có **độ dốc nhỏ** (tăng chậm hoặc gần như nằm ngang), điều này phản ánh rằng số lỗi phát sinh đang giảm dần theo thời gian. Điều đó có thể được hiểu là hệ thống đang trở nên ổn định hơn, hoặc các lỗi chính đã được phát hiện và xử lý. Trong nhiều trường hợp, đây là chỉ dấu cho thấy phần mềm đã đạt đến mức độ kiểm thử đầy đủ và có độ tin cậy cao hơn.

Phân tích xu hướng của biểu đồ theo thời gian giúp đánh giá tiến độ và hiệu quả của quá trình kiểm thử, đồng thời hỗ trợ dự đoán thời điểm thích hợp để triển khai hoặc phát hành sản phẩm.

6.3 Các nhân tố ảnh hưởng đến độ tin cậy phần mềm

Độ tin cậy của một hệ thống phần mềm phụ thuộc vào nhiều yếu tố, trong đó đáng chú ý là số lượng lỗi tiềm ẩn trong hệ thống và khả năng của các lập trình viên (developers) trong việc phát hiện và xử lý những lỗi đó. Một trong những nhân tố quan trọng có ảnh hưởng trực tiếp là:

1) Kích thước và độ phức tạp của mã nguồn

Kích thước và độ phức tạp của mã nguồn (source code) có mối liên hệ chặt chẽ với xác suất phát sinh lỗi trong hệ thống. Một hệ thống có số lượng dòng lệnh (lines

of code – LOC) lớn thường tương ứng với độ phức tạp cao hơn, dẫn đến khả năng tiềm ẩn nhiều lỗi hơn so với các hệ thống nhỏ.

Trong các hệ thống lớn, nơi tồn tại hàng trăm nghìn dòng mã, các mô-đun thường phải tương tác với nhau ở mức độ cao. Sự phức tạp trong tương tác giữa các mô-đun này làm tăng rủi ro phát sinh lỗi logic, lỗi truyền dữ liệu hoặc lỗi phụ thuộc trạng thái. Bên cạnh đó, khi một đoạn mã chứa nhiều câu lệnh điều kiện, vòng lặp, và phân nhánh (if-else, switch, v.v.), mức độ phức tạp sẽ gia tăng, làm cho việc kiểm thử toàn diện trở nên khó khăn hơn, từ đó tăng khả năng bỏ sót lỗi.

Ngoài ra, do hạn chế về chi phí và thời gian trong quá trình phát triển phần mềm, việc kiểm thử toàn bộ hệ thống — đặc biệt là các hệ thống lớn — thường không thể thực hiện đầy đủ. Điều này dẫn đến khả năng tồn tại lỗi ngay từ giai đoạn phát triển, kiểm thử và cả sau khi triển khai. Hơn nữa, trong quá trình sửa lỗi (debugging), đặc biệt với các hệ thống lớn và phức tạp, việc sửa một lỗi có thể vô tình tạo ra lỗi mới (regression), làm cho quá trình duy trì độ tin cậy trở nên thách thức hơn.

2) Các đặc tính của chu trình phát triển

Trong những năm gần đây, nhiều mô hình và chu trình phát triển phần mềm hiện đại đã được đề xuất và áp dụng, đi kèm với sự ra đời của các kỹ thuật và công cụ hỗ trợ mạnh mẽ cho từng giai đoạn của vòng đời phát triển phần mềm. Các công nghệ này giúp cải thiện chất lượng sản phẩm và góp phần giảm thiểu số lượng lỗi tiềm ẩn trong hệ thống.

Ví dụ, các phương pháp hình thức (formal methods) như UML (Unified Modeling Language), SDL (Specification and Description Language) được sử dụng để mô hình hóa và xác định chính xác yêu cầu đối với các hệ thống phức tạp, đặc biệt là những hệ thống thời gian thực hoặc có ràng buộc cao về độ chính xác và hiệu năng. Việc áp dụng các phương pháp hình thức ngay từ giai đoạn đầu giúp phát hiện và loại bỏ những sai sót trong phân tích yêu cầu và thiết kế, giảm thiểu nguy cơ phát sinh lỗi ở các giai đoạn sau.

Ngoài ra, các kỹ thuật kiểm tra và đánh giá thiết kế, mã nguồn như *code walk-through*, *code inspection* đã được phát triển nhằm hỗ trợ việc phát hiện sớm các khiếm khuyết trong thiết kế và lỗi trong cài đặt. Những kỹ thuật này có vai trò đặc biệt quan trọng trong môi trường phát triển cộng tác, nơi nhiều lập trình viên cùng làm việc trên một cơ sở mã nguồn chung.

Bên cạnh đó, các công cụ kiểm thử phần mềm hiện đại cũng được phát triển và áp dụng cho từng giai đoạn của quy trình kiểm thử như: kiểm thử đơn vị, kiểm thử

tích hợp, kiểm thử hệ thống, kiểm thử hồi quy và kiểm thử tự động. Những công cụ này giúp tăng tính hiệu quả và khả năng kiểm soát chất lượng trong suốt vòng đời phần mềm.

Tổng thể, việc phát triển phần mềm dưới sự hỗ trợ của các kỹ thuật và công cụ tiên tiến sẽ góp phần nâng cao mức độ kiểm soát trong từng giai đoạn phát triển, từ đó giúp giảm thiểu đáng kể lỗi tiềm ẩn và nâng cao độ tin cậy của hệ thống phần mềm.

3) Trình độ, kinh nghiệm và đào tạo nhân sự

Mặc dù ngành công nghệ thông tin đã phát triển mạnh mẽ trong hơn hai thập kỷ qua, nhưng việc đào tạo bài bản về kỹ nghệ phần mềm (software engineering) cho sinh viên đại học mới chỉ được triển khai rộng rãi trong khoảng 10 năm trở lại đây. Do đó, lực lượng lao động trong lĩnh vực này vẫn còn thiếu kinh nghiệm thực tiễn, đặc biệt là trong các pha kiểm thử và triển khai các hệ thống phần mềm quy mô lớn.

Thiếu kinh nghiệm và kỹ năng trong các quy trình phần mềm — như phân tích yêu cầu, thiết kế hệ thống, kiểm thử phần mềm, và vận hành sau triển khai — dễ dẫn đến việc bỏ sót lỗi hoặc xử lý không đúng các tình huống bất thường. Điều này làm tăng khả năng tồn tại lỗi trong hệ thống và ảnh hưởng trực tiếp đến độ tin cậy của phần mềm. Việc đầu tư vào đào tạo chuyên sâu và nâng cao năng lực nhân sự là một yếu tố then chốt để cải thiện chất lượng sản phẩm phần mềm.

4) Môi trường vận hành hệ thống

Độ tin cậy của phần mềm không chỉ phụ thuộc vào chất lượng thiết kế và cài đặt, mà còn chịu ảnh hưởng lớn từ môi trường vận hành thực tế — nơi người dùng cuối tương tác với hệ thống. Một vấn đề phổ biến trong quá trình kiểm thử là sự khác biệt giữa môi trường kiểm thử (test environment) và môi trường triển khai thực tế (production environment). Khi các kỹ sư kiểm thử không thể mô phỏng chính xác điều kiện và cách sử dụng thực tế của người dùng, khả năng phát hiện ra các lỗi giống như người dùng cuối gặp phải là rất thấp.

Vì vậy, để đảm bảo kiểm thử có hiệu quả, đội ngũ kiểm thử cần phải nắm bắt đầy đủ đặc điểm của môi trường vận hành, bao gồm: hệ điều hành, phần cứng, dữ liệu đầu vào thực tế, hành vi người dùng, và các yếu tố mạng. Việc thiếu thông tin hoặc không có khả năng mô phỏng đầy đủ môi trường thực tế sẽ dẫn đến việc bỏ sót lỗi, khiến chúng chỉ được phát hiện khi hệ thống đã được bàn giao cho khách

hàng — lúc mà chi phí sửa chữa và ảnh hưởng đến uy tín là cao nhất.

Như vậy, độ tin cậy của phần mềm có thể được xem như một hàm phụ thuộc vào nhiều yếu tố đầu vào khác nhau — bao gồm kích thước và độ phức tạp của hệ thống, quy trình phát triển, kỹ năng và kinh nghiệm của đội ngũ phát triển, cũng như môi trường vận hành thực tế. Từ quan điểm mô hình hóa và mô phỏng hệ thống, việc xây dựng một mô hình toán học với các tham số phản ánh đầy đủ các yếu tố trên sẽ rất hữu ích trong việc đánh giá và cải thiện chất lượng phần mềm.

Tuy nhiên, trên thực tế, việc xây dựng một mô hình toàn diện như vậy là vô cùng khó khăn do mối quan hệ phức tạp và phi tuyến giữa các yếu tố, cũng như sự không chắc chắn và khó định lượng của nhiều biến liên quan. Do đó, hầu hết các mô hình độ tin cậy phần mềm trong thực tế đều chỉ dựa trên một số lượng nhỏ các tham số có thể đo lường được. Mặc dù còn nhiều hạn chế, những mô hình đơn giản này vẫn mang lại giá trị thực tiễn đáng kể bằng cách cung cấp cái nhìn sâu sắc về chất lượng phần mềm và xu hướng cải thiện theo thời gian.

Một ví dụ điển hình là việc sử dụng thời gian kiểm thử làm tham số đầu vào. Giả sử hệ thống được kiểm thử kỹ lưỡng và toàn diện trong pha kiểm thử hệ thống, thì số lượng lỗi được phát hiện sẽ tăng lên đáng kể. Khi các lỗi này được khắc phục, độ tin cậy của hệ thống cũng sẽ được cải thiện tương ứng. Điều này cho thấy rằng thời gian dành cho kiểm thử có ảnh hưởng trực tiếp đến độ tin cậy của hệ thống.

Do đó, có thể xây dựng các mô hình độ tin cậy dựa trên thời gian kiểm thử, trong đó thời gian là biến độc lập và độ tin cậy là biến phụ thuộc. Các mô hình như vậy không chỉ giúp ước lượng mức độ độ tin cậy đạt được tại một thời điểm nhất định, mà còn hỗ trợ trong việc xác định khoảng thời gian kiểm thử cần thiết để đạt đến một ngưỡng độ tin cậy mong muốn. Đây là công cụ quan trọng trong việc ra quyết định liên quan đến việc phát hành, triển khai và duy trì phần mềm.

So sánh các kỹ thuật công nghệ phần mềm: Để phát triển những phần mềm có độ tin cậy cao quy mô lớn với chi phí hiệu quả, rất nhiều kỹ thuật mới được sử dụng ví dụ: mô hình thác nước, mô hình xoáy ốc, mô hình chế thử, mô hình extreme programming hay mô hình Scrum. Dưới góc nhìn của kiểm thử, rất nhiều hệ thống công cụ và kỹ thuật mới được phát triển cho các pha khác nhau của kiểm thử hệ thống. Ở những pha sớm hơn của giai đoạn phát triển cũng có rất nhiều kỹ thuật được sử dụng để mô hình hoá yêu cầu và đặc tả phần mềm như sử dụng biểu đồ UML, lược đồ quan hệ các biểu đồ dữ liệu,...

Khi một kỹ thuật mới được giới thiệu và thử nghiệm rất cần đo đạc độ hiệu quả của nó dựa trên một số câu hỏi được đặt ra:

- Chi phí để áp dụng công nghệ mới đó?

- Việc áp dụng công nghệ đó ảnh hưởng thế nào đến lịch phát triển của hệ thống?
- Tác dụng của công nghệ đó lên độ tin cậy của hệ thống?

Khái niệm độ tin cậy phần mềm có thể được sử dụng để đánh giá các kỹ thuật công nghệ được sử dụng dưới góc độ của tính hữu dụng của những công nghệ đó đối với việc nâng cao chất lượng của phần mềm.

Ví dụ: hai công nghệ M1 và M2, M1 được sử dụng để phát triển hệ thống S1, M2 cho S2 để cài đặt cùng một loại ứng dụng. Việc so sánh độ tin cậy của hai hệ thống S1 và S2 sẽ cho phép xác định tính hữu dụng của hai công nghệ này

2) Đo lường tiến độ kiểm thử hệ thống

Đo lường tiến độ kiểm thử hệ thống là trung tâm đối với việc quản lý dự án. Câu hỏi quan trọng nhất trong quá trình quản lý dự án là gì? Sau bao lâu thì dự án được hoàn thành? Trả lời cho câu hỏi này cho chúng ta xác định được tiến độ thực hiện có match với plan đã đặt ra ban đầu hay không. Việc quản lý tiến độ kiểm thử hệ thống đóng vai trò rất quan trọng vì nó là giai đoạn tiêu tốn nhiều thời gian và chi phí. Có hai metrics có thể được sử dụng để đo lường nó

- test case đã được thực hiện
- những test case đã passed cho các chức năng có độ ưu tiên cao

Khái niệm về độ tin cậy của phần mềm có thể được sử dụng để đánh giá tiến độ kiểm thử hệ thống. Ví dụ, chúng ta có thể đo mật độ thất bại của một hệ thống tại một thời điểm để xác định độ tin cậy hiện thời của hệ thống, so sánh với độ tin cậy cho phép sau khi hệ thống được released cho phép chúng ta đánh giá được tiến độ hiện tại của dự án, nếu lớn chứng tỏ còn rất nhiều việc phải làm, nếu nhỏ rõ ràng chúng ta đang tiến gần đến mục tiêu đặt ra và dự án sớm kết thúc.

3) Điều khiển sự vận hành hệ thống

Độ tin cậy của phần mềm thường giảm trong giai đoạn bảo trì. Vì sao? Vì càng nhiều thay đổi được thực hiện trên hệ thống, càng nhiều lỗi tiềm ẩn phát sinh dẫn tới giảm độ tin cậy của phần mềm. Ví dụ, bắt đầu giai đoạn kiểm thử hệ thống, một hệ thống có k faults trên 1000 lines of code, nếu thêm vào N lines, tỉ lệ phát sinh lỗi sẽ là $Nk/1000$, tăng lên. Việc này làm giảm độ tin cậy của hệ thống và nó sẽ yêu cầu thời gian dài hơn để kiểm thử hệ thống và thời gian dành để sửa các lỗi phát sinh nhằm nâng cao độ tin cậy của hệ thống. Do vậy người quản lý dự án có thể đặt ra một con số cụ thể về sự thay đổi của mức độ tin cậy của hệ thống đối với

mỗi hoạt động bảo trì. Do đó, lượng công việc đặt ra trong giai đoạn bảo trì có thể được xác định dựa trên mức độ tin cậy của hệ thống đặt ra trong giai đoạn đó.

4) Cái nhìn tốt hơn về chu trình phát triển phần mềm

Đánh giá độ tin cậy của hệ thống có thể giúp người quản lý và các nhà phát triển đánh giá được chu trình phát triển phần mềm mà họ đang sử dụng và đưa ra các quyết định cụ thể. Ví dụ: bằng cách quan sát mật độ thất bại ngay từ giai đoạn đầu kiểm thử hệ thống, người quản lý có thể xác định được thời gian và mức độ công việc cần phải bỏ ra trong giai đoạn kiểm thử để tăng cường độ tin cậy của hệ thống lên một con số chấp nhận được

6.4 Hồ sơ vận hành - Operational Profile

Hồ sơ vận hành (Operational Profile) hoặc Hồ sơ sử dụng (Usage Profile) Hồ sơ vận hành là một mô hình mô tả cách thức người dùng thực sự tương tác và sử dụng hệ thống phần mềm trong môi trường thực tế. Mỗi loại hành vi sử dụng sẽ tương ứng với một tập các hành động (operations). Khái niệm này đặc biệt quan trọng trong việc ước lượng và đánh giá độ tin cậy phần mềm, bởi vì độ tin cậy không chỉ phụ thuộc vào chất lượng nội tại của hệ thống, mà còn phụ thuộc mạnh mẽ vào cách hệ thống được sử dụng trong thực tế. Một hệ thống có thể vận hành ổn định trong một nhóm người dùng này nhưng lại dễ sinh lỗi trong nhóm khác, nếu hành vi sử dụng khác biệt đáng kể.

Hồ sơ vận hành (Operational Profile) được xây dựng với mục tiêu chính là phản ánh chính xác hành vi sử dụng thực tế của người dùng đối với hệ thống phần mềm. Việc thiết lập hồ sơ này mang lại nhiều lợi ích quan trọng trong quá trình kiểm thử, đánh giá chất lượng và bảo trì phần mềm. Cụ thể, hồ sơ vận hành phục vụ các mục đích sau:

Thứ nhất, nó giúp hiểu rõ cách thức người dùng thực sự tương tác với hệ thống trong các tình huống thực tế, từ đó cung cấp một cơ sở vững chắc để thiết kế các ca kiểm thử có tính đại diện cao.

Thứ hai, bằng cách mô hình hóa xác suất xảy ra của từng hành vi sử dụng, hồ sơ vận hành cho phép ước lượng chính xác hơn các chỉ số chất lượng quan trọng như độ tin cậy (reliability) và hiệu suất (performance) của hệ thống trong môi trường thực tế thay vì chỉ trong môi trường kiểm thử lý tưởng.

Thứ ba, hồ sơ vận hành hỗ trợ việc ưu tiên kiểm thử và bảo trì các chức năng hoặc phần của hệ thống được sử dụng thường xuyên hơn. Điều này giúp tối ưu hóa tài nguyên kiểm thử, tập trung vào các khu vực có rủi ro cao, và từ đó nâng cao

hiệu quả đảm bảo chất lượng phần mềm.

Hồ sơ vận hành được cấu thành từ hai thành phần chính:

- Thứ nhất, đó là tập các thao tác (operations) có thể được thực hiện trên hệ thống. Đây là danh sách các hành động hoặc chức năng mà người dùng thực sự sử dụng trong quá trình tương tác với phần mềm, chẳng hạn như “đăng nhập”, “tìm kiếm sản phẩm”, “thanh toán”, “in hóa đơn”, v.v. Tập thao tác này nên được xác định một cách đầy đủ và chính xác để phản ánh đúng thực tế sử dụng của hệ thống.
- Thứ hai, với mỗi thao tác trong tập trên, cần xác định xác suất xuất hiện (probability of occurrence) tương ứng. Đây là xác suất thống kê phản ánh tần suất sử dụng của từng thao tác trong môi trường thực tế. Thông tin này có thể được thu thập từ dữ liệu log, khảo sát người dùng, hoặc mô phỏng hành vi sử dụng.

Sự kết hợp giữa danh sách các thao tác và xác suất xuất hiện của chúng tạo thành hồ sơ vận hành đầy đủ. Hồ sơ này đóng vai trò là cơ sở quan trọng để thiết kế kiểm thử theo thống kê, ước lượng độ tin cậy phần mềm, và tối ưu hóa kế hoạch bảo trì, kiểm thử theo mức độ ưu tiên sử dụng.

Ví dụ minh họa: Giả sử có ba thao tác A, B và C với xác suất xuất hiện lần lượt là 50%, 30% và 2%. Hồ sơ vận hành sẽ được biểu diễn như sau: (A, 0.5), (B, 0.3), (C, 0.02)

Các cách biểu diễn hồ sơ vận hành:

Hồ sơ vận hành có thể được biểu diễn dưới nhiều hình thức khác nhau nhằm phục vụ cho việc phân tích, trực quan hóa và áp dụng trong kiểm thử. Hai cách biểu diễn phổ biến nhất là:

- Biểu diễn dạng bảng (Tabular representation): Đây là cách trình bày đơn giản và trực tiếp nhất, trong đó mỗi hàng biểu thị một thao tác (operation) có thể được thực hiện trên hệ thống, kèm theo xác suất xuất hiện tương ứng. Cách biểu diễn này phù hợp để sử dụng trong phân tích định lượng, kiểm thử theo xác suất, hoặc tích hợp vào mô hình toán học về độ tin cậy.
- Biểu diễn dạng đồ họa (Graphical representation): Các biểu đồ như biểu đồ thanh (bar chart) hoặc biểu đồ tròn (pie chart) được sử dụng để trực quan hóa tần suất hoặc xác suất xuất hiện của các thao tác trong hồ sơ vận hành. Cách biểu diễn này giúp người xem dễ dàng nhận biết những thao tác được sử

dụng thường xuyên nhất, từ đó hỗ trợ việc ưu tiên kiểm thử và đánh giá mức độ rủi ro.

Việc lựa chọn hình thức biểu diễn phù hợp phụ thuộc vào mục đích sử dụng, đối tượng người đọc và giai đoạn áp dụng (phân tích, kiểm thử, báo cáo...).

Biểu diễn dạng bảng

Dạng bảng với 3 cột Ví dụ: Hệ thống thông tin thư viện

- Cột 1: Tên các thao tác
- Cột 2: Tần suất sử dụng thao tác
- Cột 3: Xác suất sử dụng thao tác

Operation	Operations per Hour	Probability
Book checked out	450	0.45
Book returned in time	324	0.324
Book renewed	81	0.081
Book returned late	36	0.036
Book reported lost	9	0.009
⋮	⋮	⋮
Total	1000	1.0

Hình 6.2: Ví dụ hồ sơ vận hành của hệ thống thông tin thư viện

Biểu diễn dạng đồ họa

Cấu trúc cây (tree structure) là một phương pháp trực quan để biểu diễn hồ sơ vận hành phức tạp, nơi mỗi thao tác được mô tả thông qua nhiều thuộc tính cụ thể.

Nút (Nodes): Mỗi nút trong cây thể hiện một thuộc tính của thao tác, ví dụ: loại người dùng, loại tài liệu, hành động cụ thể.

Nhánh (Branches): Từ mỗi nút, các nhánh thể hiện các giá trị có thể có của thuộc tính đó (ví dụ: Sinh viên, Giảng viên), kèm theo xác suất xảy ra của mỗi giá trị.

Mục đích của cách biểu diễn này là giúp phân tích sâu hơn hành vi người dùng và tính toán chính xác hơn độ tin cậy của hệ thống bằng cách mô hình hóa chi tiết cách các thao tác được thực hiện theo ngữ cảnh khác nhau.

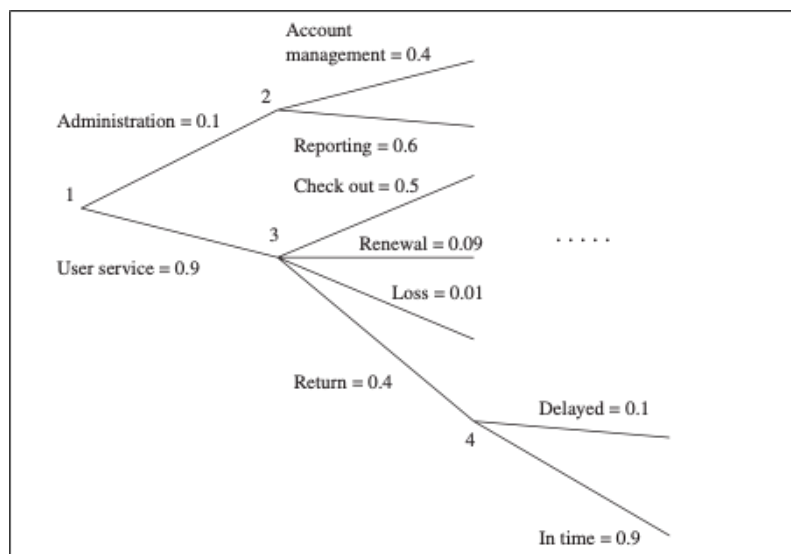
Ví dụ: 4 nodes: 4 attributes of operations

- Node 1: biểu diễn thuộc tính scope của operation với hai giá trị là administration và user service. Phạm vi của một operation chỉ operation đó là dành cho

quản trị viên để quản lý thông tin của hệ thống hay dành cho cung cấp dịch vụ cho người dùng. Giả sử rằng xác suất của administration là 0,1 và của user service là 0.9

- Node 2: biểu diễn một thao tác thuộc về administration trong khi node 3 biểu diễn thao tác của người dùng. Có 2 giá trị tương ứng với thao tác admin, là quản lý account và báo cáo với các xác suất xuất hiện tương ứng là 0.4 và 0.6. Tương tự có 4 thuộc tính của user operation là check out, renewal, loss và return với xác suất tương ứng là 0.5, 0.09, 0.01, 0.4.
- Node 4 biểu diễn thao tác trả sách với 2 thuộc tính là delayed và in-time với xác suất tương ứng là 0.1 và 0.9

Biểu diễn bằng bảng có thể dễ dàng suy ra từ dạng đồ thị bằng cách xem xét tất cả các nhánh của đồ thị và nhân các xác suất xuất hiện trên từng đường, ví dụ xác suất để trả sách muộn là 0.036 trong bảng bằng cách nhân 3 xác suất: $0.9 \times 0.4 \times 0.1 = 0.036$



Hình 6.3: Ví dụ hồ sơ vận hành dạng đồ họa của hệ thống thông tin thư viện

Chương 7

Quy trình kiểm thử phần mềm

7.1	Vòng đời kiểm thử phần mềm (Software Testing Lifecycle – STLC)	142
7.2	Lập kế hoạch và tổ chức kiểm thử	145
7.3	Kiểm thử hệ thống - System testing	149
7.4	Kiểm thử chấp nhận (Acceptance Testing)	157
	Danh mục từ khóa	161

7.1 Vòng đời kiểm thử phần mềm (Software Testing Lifecycle – STLC)

Vòng đời kiểm thử phần mềm (STLC) là một quy trình có cấu trúc gồm nhiều giai đoạn nhằm đảm bảo chất lượng phần mềm thông qua việc lập kế hoạch, thiết kế, thực hiện và đánh giá kiểm thử. Mỗi giai đoạn đều có tiêu chí đầu vào, đầu ra, tài liệu và hoạt động cụ thể.

Các mô hình phát triển phần mềm phổ biến

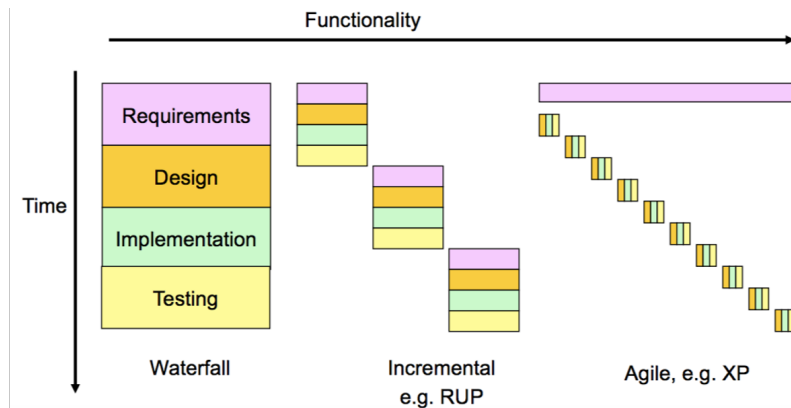
Mỗi mô hình phát triển phần mềm mang lại những ưu điểm và hạn chế riêng, ảnh hưởng trực tiếp đến chất lượng, tiến độ và độ tin cậy của hệ thống.

Mô hình thác nước (Waterfall) là mô hình truyền thống, trong đó các giai đoạn phát triển như phân tích yêu cầu, thiết kế, cài đặt, kiểm thử và bảo trì được thực hiện tuần tự. Mô hình này giúp minh bạch hóa quy trình với các bước rõ ràng và dễ quản lý. Tuy nhiên, điểm yếu lớn nhất là tính không linh hoạt và rủi ro cao, vì hoạt động kiểm thử chỉ được tiến hành ở giai đoạn cuối cùng. Điều này khiến cho

việc phát hiện lỗi trở nên muộn màng và tốn kém nếu cần sửa đổi ở các giai đoạn trước đó.

Mô hình tăng trưởng (Incremental) khắc phục một phần hạn chế trên bằng cách cho phép phát triển nhiều phần (increment) độc lập của hệ thống song song. Mỗi phần được phân tích, thiết kế, cài đặt và kiểm thử như một đơn vị riêng biệt. Mô hình này giúp hệ thống có thể được sử dụng sớm và từng bước được hoàn thiện. Tuy nhiên, nhược điểm là yêu cầu vẫn cần phải được xác định khá rõ ràng ngay từ đầu để đảm bảo tính đồng nhất và tích hợp suôn sẻ giữa các phần.

Mô hình Agile ngày càng trở nên phổ biến nhờ tính linh hoạt và khả năng thích ứng cao với thay đổi. Trong mô hình này, yêu cầu phần mềm có thể được thu thập, điều chỉnh và hoàn thiện xuyên suốt vòng đời dự án. Các hoạt động phân tích, thiết kế, lập trình và kiểm thử được tiến hành song song theo từng vòng lặp nhỏ (iteration hoặc sprint), giúp đảm bảo mỗi yêu cầu nhỏ được phát triển và kiểm chứng một cách nhanh chóng. Agile đặc biệt phù hợp với các dự án có yêu cầu thay đổi thường xuyên hoặc không thể xác định đầy đủ ngay từ đầu.

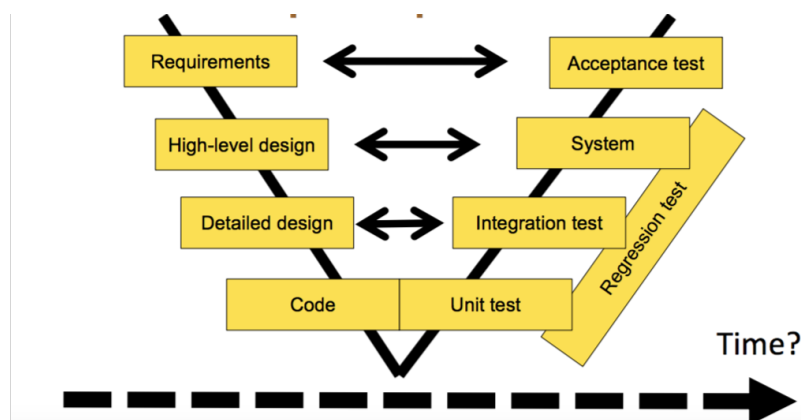


Hình 7.1: Các mô hình phát triển phần mềm

Trong quy trình phát triển phần mềm, các pha kiểm thử sẽ cần được xen kẽ để đảm bảo mỗi một giai đoạn phân tích, thiết kế và cài đặt đều được kiểm chứng rõ ràng.

Trong mô hình chữ V, mỗi giai đoạn phát triển phần mềm (bên trái của chữ V) đều có một giai đoạn kiểm thử tương ứng (bên phải của chữ V). Điều này đảm bảo rằng từng bước trong quá trình phát triển đều được kiểm chứng độc lập và có thể truy vết.

Mô hình chữ V (V-Model) là một phiên bản được cải tiến từ mô hình Waterfall truyền thống, trong đó các hoạt động kiểm thử không được thực hiện sau cùng, mà



Hình 7.2: Mô hình chữ V

được thiết kế và liên kết trực tiếp với từng giai đoạn phát triển phần mềm tương ứng. V-Model nhấn mạnh tính xác minh (verification) và xác thực (validation) thông qua một quy trình có cấu trúc rõ ràng.

Một đặc điểm nổi bật của V-Model là các ca kiểm thử được xây dựng song song ngay tại thời điểm diễn ra các hoạt động mà chúng dùng để xác minh.

Ví dụ:

- Kiểm thử chấp nhận (Acceptance Tests) được xây dựng khi hệ thống đang trong giai đoạn phân tích yêu cầu.
- Kiểm thử hệ thống (System Tests) được thiết kế song song với đặc tả hệ thống.
- Kiểm thử tích hợp (Integration Tests) được tạo ra khi thiết kế kiến trúc được hoàn thành.
- Kiểm thử đơn vị (Unit Tests) được xây dựng từ thiết kế chi tiết.

Điều này giúp đảm bảo rằng mọi yêu cầu đều có cơ chế kiểm chứng tương ứng và rõ ràng ngay từ đầu.

Nếu một hoạt động không vượt qua được bài kiểm thử đã thiết kế, quá trình phát triển cần được quay lại và lặp lại từ chính giai đoạn tương ứng để khắc phục lỗi. Điều này thể hiện tính lặp có kiểm soát (controlled iteration) trong V-Model.

Với cách tiếp cận này, mô hình chữ V cung cấp một phương pháp phát triển phần mềm có cấu trúc chặt chẽ, giúp cải thiện tính truy vết giữa yêu cầu và kiểm thử, đồng thời nâng cao khả năng phát hiện lỗi sớm và tăng độ tin cậy của hệ thống.

7.2 Lập kế hoạch và tổ chức kiểm thử

Việc kiểm thử phần mềm cần được lên kế hoạch từ sớm, cụ thể là ngay trong giai đoạn xác định yêu cầu và đặc tả hệ thống. Việc lập kế hoạch sớm giúp đảm bảo rằng quá trình kiểm thử sẽ được tích hợp đầy đủ vào vòng đời phát triển phần mềm, thay vì chỉ thực hiện như một bước sau cùng.

Một số hoạt động quan trọng trong quá trình lập kế hoạch và tổ chức kiểm thử bao gồm:

- **Phân bổ nguồn lực:** Cần xác định và phân bổ các nguồn lực cần thiết cho kiểm thử, bao gồm cả nhân lực (kỹ sư kiểm thử, lập trình viên) và tài nguyên máy tính (môi trường kiểm thử, thiết bị phần cứng, v.v.).
- **Phát triển công cụ và dữ liệu kiểm thử:** Chuẩn bị các công cụ hỗ trợ kiểm thử như test driver, test stub, và cơ sở dữ liệu dùng cho dữ liệu kiểm thử. Việc chuẩn bị đầy đủ giúp kiểm thử được thực hiện hiệu quả và có thể tự động hóa khi cần thiết.
- **Tính khả kiểm thử (Testability):** Đây là một trong những yêu cầu quan trọng đối với mọi hệ thống phần mềm. Một hệ thống được thiết kế tốt cần phải hỗ trợ cho việc kiểm thử dễ dàng, ví dụ như có các giao diện rõ ràng, trạng thái có thể quan sát được và kết quả có thể xác minh được.
- **Phát triển kế hoạch kiểm thử (Test Plan):** Kế hoạch kiểm thử là tài liệu mô tả chi tiết cách thức kiểm thử sẽ được thực hiện. Nó bao gồm phạm vi kiểm thử, chiến lược, lịch trình, công cụ, tiêu chí hoàn thành, và cách quản lý rủi ro trong quá trình kiểm thử.
- **Tính truy vết (Traceability):** Mọi hoạt động kiểm thử, bao gồm các ca kiểm thử và kết quả kiểm thử, cần phải được truy vết ngược về các yêu cầu và đặc tả ban đầu. Điều này đảm bảo rằng tất cả các yêu cầu đã được kiểm chứng đầy đủ, và không có yêu cầu nào bị bỏ sót.

Việc lập kế hoạch kiểm thử bài bản không chỉ đảm bảo hiệu quả của hoạt động kiểm thử, mà còn góp phần quan trọng vào việc nâng cao độ tin cậy và chất lượng tổng thể của phần mềm.

Tài liệu kiểm thử (Testing Documentation)

Tài liệu kiểm thử là một phần quan trọng trong quy trình kiểm thử phần mềm, đảm bảo các hoạt động kiểm thử được lập kế hoạch, thực hiện và theo dõi một cách

có hệ thống và truy vết được. Các tài liệu kiểm thử chính bao gồm:

- Yêu cầu kiểm thử và đặc tả kiểm thử (Testing Requirements and Test Specifications): Được phát triển trong giai đoạn phân tích yêu cầu và đặc tả hệ thống. Chúng mô tả các điều kiện cần kiểm thử để xác minh rằng hệ thống đáp ứng đúng các yêu cầu chức năng và phi chức năng.
- Kế hoạch kiểm thử (Test Plan): Là tài liệu định hướng toàn bộ quy trình kiểm thử. Nó mô tả trình tự các hoạt động kiểm thử, phạm vi kiểm thử, chiến lược, tài nguyên, công cụ, lịch trình, và phần mềm hỗ trợ kiểm thử sẽ được xây dựng (ví dụ như test driver, stub hoặc các công cụ tự động).
- Thủ tục kiểm thử (Test Procedure): Quy định chi tiết trình tự tích hợp hệ thống và cách thức kiểm thử từng module. Tài liệu này cũng bao gồm các bước thực hiện kiểm thử đơn vị (unit test), kiểm thử tích hợp, cũng như mô tả dữ liệu kiểm thử được sử dụng trong từng trường hợp.
- Ghi nhật ký và lưu trữ kết quả kiểm thử (Test Logging and Archiving): Việc ghi nhận các kiểm thử đã thực hiện và lưu trữ kết quả là phần không thể thiếu trong quá trình kiểm thử. Nó hỗ trợ việc phân tích hậu kiểm, truy vết nguyên nhân lỗi và đảm bảo tính minh bạch, nhất quán trong đánh giá chất lượng phần mềm.

Việc duy trì bộ tài liệu kiểm thử đầy đủ, chính xác và cập nhật thường xuyên không chỉ giúp cải thiện hiệu quả kiểm thử, mà còn góp phần nâng cao khả năng bảo trì, mở rộng và kiểm soát chất lượng trong các dự án phần mềm có quy mô lớn hoặc yêu cầu nghiêm ngặt.

Các thành phần chính của kế hoạch kiểm thử (Test Plan Components)

Một kế hoạch kiểm thử (Test Plan) là tài liệu cốt lõi trong quá trình kiểm thử phần mềm, nhằm mô tả toàn bộ chiến lược, phạm vi, phương pháp, nguồn lực và thời gian cần thiết để kiểm thử hệ thống. Kế hoạch này bao gồm nhiều thành phần then chốt như sau:

- Xác lập mục tiêu kiểm thử (Establish Test Objectives): Xác định rõ mục tiêu chính của quá trình kiểm thử là gì — ví dụ: xác minh tính đúng đắn chức năng, kiểm tra hiệu năng, đảm bảo khả năng tương thích, hoặc phát hiện các lỗi nghiêm trọng trước khi bàn giao.

- **Thiết kế ca kiểm thử (Designing Test Cases):** Dựa trên yêu cầu hệ thống và đặc tả phần mềm, các ca kiểm thử được thiết kế để bao phủ toàn bộ các điều kiện, tình huống đầu vào và hành vi mong đợi của phần mềm.
- **Viết chi tiết ca kiểm thử (Writing Test Cases):** Các ca kiểm thử được viết một cách chi tiết, bao gồm: mục tiêu, dữ liệu đầu vào, các bước thực hiện, điều kiện tiên đề, và kết quả mong đợi. Ca kiểm thử càng rõ ràng thì khả năng thực hiện kiểm thử càng hiệu quả.
- **Kiểm tra ca kiểm thử (Testing the Test Cases):** Các ca kiểm thử cần được rà soát, xác minh và (nếu có thể) thử nghiệm trên môi trường thử nghiệm nhỏ để đảm bảo tính chính xác và phù hợp với hệ thống thực tế.
- **Thực thi kiểm thử (Executing Tests):** Thực hiện các ca kiểm thử đã thiết kế trên phần mềm thực tế, ghi nhận kết quả, và phát hiện các sai lệch giữa kết quả thực tế và kỳ vọng.
- **Đánh giá kết quả kiểm thử (Evaluate Test Results):** Phân tích kết quả thực thi để xác định phần mềm có đáp ứng được các yêu cầu đã đề ra không. Những lỗi tìm được sẽ được báo cáo và xử lý. Đồng thời, đánh giá mức độ hoàn thành của kiểm thử dựa trên tiêu chí đã thiết lập.

Tổ chức kiểm thử phần mềm (Test Organisation)

Một tổ chức kiểm thử hiệu quả đóng vai trò quan trọng trong việc đảm bảo chất lượng sản phẩm phần mềm. Việc tổ chức bài bản các hoạt động, nguồn lực và vai trò trong kiểm thử mang lại nhiều lợi ích cho toàn bộ dự án phần mềm. Cụ thể, tổ chức kiểm thử góp phần:

- **Hỗ trợ ra quyết định (Support Decision Making):** Cung cấp dữ liệu, kết quả kiểm thử và đánh giá rủi ro giúp ban quản lý đưa ra các quyết định quan trọng như: phát hành phần mềm, dừng triển khai, hoặc điều chỉnh kế hoạch phát triển.
- **Tăng cường làm việc nhóm (Enhance Teamwork):** Việc phân định rõ vai trò, trách nhiệm trong quá trình kiểm thử giúp các thành viên phối hợp hiệu quả, giảm chồng chéo, tăng năng suất và tính cộng tác.
- **Đảm bảo tính độc lập (Independency):** Tổ chức kiểm thử độc lập (so với nhóm phát triển) giúp giảm thiểu thiên lệch và đảm bảo kiểm thử khách quan, trung lập — yếu tố rất quan trọng trong việc đánh giá chất lượng phần mềm.

- Cân bằng chất lượng kiểm thử (Balance Testing Quality): Giúp kiểm soát chất lượng kiểm thử thông qua quy trình chuẩn hóa, quy định các tiêu chí chấp nhận, phương pháp đánh giá và công cụ hỗ trợ phù hợp.
- Hỗ trợ quản lý kiểm thử (Assist Test Management): Tổ chức tốt giúp quản lý các hoạt động kiểm thử dễ dàng hơn, từ việc lập kế hoạch, phân bổ nguồn lực, theo dõi tiến độ đến đánh giá hiệu quả.
- Làm chủ công nghệ kiểm thử (Ownership of Test Technology): Thiết lập, quản lý và cập nhật các công cụ kiểm thử, môi trường kiểm thử và quy trình kiểm thử để phục vụ các mục tiêu dự án và nâng cao hiệu quả kiểm thử tổng thể.
- Tối ưu hóa nguồn lực (Resources Utilization): Quản lý và sử dụng hiệu quả các tài nguyên sẵn có — từ nhân sự, phần mềm, phần cứng cho đến dữ liệu kiểm thử, giúp tiết kiệm chi phí và thời gian.
- Phát triển lộ trình nghề nghiệp (Career Path): Một tổ chức kiểm thử tốt sẽ có lộ trình phát triển nghề nghiệp rõ ràng cho các kiểm thử viên, bao gồm đào tạo kỹ năng, nâng cao chuyên môn và định hướng thăng tiến.

7 Cách Tiếp Cận Tổ Chức Kiểm Thử Phần Mềm

Trong một tổ chức phần mềm, việc tổ chức hoạt động kiểm thử có thể dựa trên nhiều yếu tố: vai trò cá nhân, trách nhiệm đơn vị, cấu trúc tổ chức, và trung tâm công nghệ hỗ trợ kiểm thử. Dưới đây là những yếu tố thường gặp:

1. Trách nhiệm cá nhân (Each person's responsibility). Trong mô hình này, mọi thành viên trong nhóm phát triển phần mềm (bao gồm cả lập trình viên, nhà phân tích, QA,...) đều chịu trách nhiệm cho chất lượng và kiểm thử phần mềm mà họ tham gia phát triển. Đây là triết lý phổ biến trong Agile hoặc DevOps, thúc đẩy kiểm thử sớm (shift-left testing) và tinh thần trách nhiệm chung (collective ownership).
2. Trách nhiệm của từng đơn vị (Each unit's responsibility). Mỗi bộ phận chức năng trong tổ chức (ví dụ: frontend team, backend team, integration team) đều có trách nhiệm kiểm thử phần mà họ phát triển. Kiểm thử có thể do chính đội phát triển thực hiện hoặc phối hợp cùng tester chuyên trách trong nhóm.
3. Tài nguyên chuyên trách (Dedicated resource). Cung cấp nhân sự kiểm thử riêng biệt (tester, QA engineer) làm việc toàn thời gian cho việc thiết kế, thực hiện và báo cáo kiểm thử. Điều này giúp đảm bảo chuyên môn và tập trung

vào chất lượng, đặc biệt quan trọng trong các hệ thống có độ phức tạp hoặc rủi ro cao.

4. Tổ chức kiểm thử trong QA (Test Organisation in QA). Mô hình kiểm thử được đặt dưới quyền quản lý của phòng/ban QA độc lập, thường hoạt động song song với bộ phận phát triển. QA phụ trách từ việc lên kế hoạch kiểm thử, quản lý ca kiểm thử đến báo cáo chất lượng. Tính độc lập giúp đảm bảo khách quan và rõ ràng về trách nhiệm.
5. Tổ chức kiểm thử trong phát triển (Test Organisation in Development). Kiểm thử được tổ chức bên trong nhóm phát triển phần mềm. Developer tự thực hiện kiểm thử đơn vị, trong khi tester nội bộ có thể kiểm tra tích hợp hoặc hệ thống. Cách này giúp rút ngắn thời gian vòng đời, phù hợp với Agile/CI-CD nhưng cần đảm bảo năng lực kiểm thử chuyên sâu.
6. Tổ chức kiểm thử tập trung (Centralized Test Organisation). Một nhóm kiểm thử chung cho toàn công ty hoặc toàn dự án lớn. Họ quản lý chuẩn kiểm thử, công cụ, và hỗ trợ kỹ thuật kiểm thử cho các nhóm phát triển. Mô hình này giúp chuẩn hóa, chia sẻ tài nguyên và đảm bảo tính nhất quán trong kiểm thử.
7. Trung tâm công nghệ kiểm thử (Test Technology Centre). Đây là đơn vị hỗ trợ công nghệ kiểm thử cho toàn bộ tổ chức. Họ phát triển framework kiểm thử, quản lý công cụ tự động hóa, xây dựng môi trường kiểm thử, đào tạo nội bộ, và đảm bảo sự cập nhật kỹ thuật kiểm thử tiên tiến. Mục tiêu là hạ tầng hóa và tối ưu hoá kiểm thử.

7.3 Kiểm thử hệ thống - System testing

Kiểm thử hệ thống là một hoạt động kiểm thử ở cấp độ cao, nhằm đánh giá toàn bộ hệ thống phần mềm hoạt động đúng theo yêu cầu đã được chỉ định. Tuy nhiên, kiểm thử hệ thống rất đa dạng và không đồng nhất – nội dung cụ thể cần kiểm thử sẽ phụ thuộc vào đặc thù của từng ứng dụng và lĩnh vực triển khai. Do sự phức tạp và phạm vi rộng, kiểm thử hệ thống có thể bao gồm nhiều loại kiểm thử khác nhau, mỗi loại có những ưu điểm và hạn chế riêng.

Việc xây dựng môi trường kiểm thử mô phỏng đầy đủ cả phần mềm và các thành phần vật lý liên quan (ví dụ: thiết bị đầu cuối, hệ thống điều khiển, phần cứng nhúng...) đôi khi có thể tốn nhiều chi phí hơn cả việc phát triển phần mềm chính. Do đó, việc lập kế hoạch kỹ lưỡng, ưu tiên các kiểm thử quan trọng, và tái

sử dụng môi trường kiểm thử là những yếu tố cần thiết để kiểm thử hệ thống đạt hiệu quả mà vẫn tiết kiệm nguồn lực.

Kiểm thử khả năng (Capacity Testing) Capacity Testing là một dạng kiểm thử phi chức năng nhằm đánh giá khả năng của hệ thống trong việc xử lý khối lượng lớn dữ liệu hoặc số lượng người dùng đồng thời, để từ đó xác định giới hạn vận hành của hệ thống cũng như hành vi của hệ thống khi vượt quá giới hạn đó.

Kiểm thử khả năng thường được áp dụng trong các hệ thống mà việc xử lý dữ liệu ở quy mô lớn là một yêu cầu trọng yếu, ví dụ như hệ thống lưu trữ hồ sơ y tế quốc gia, hệ thống ngân hàng điện tử, hoặc nền tảng thương mại điện tử với lượng người dùng truy cập cao. Trong những trường hợp như vậy, kiểm thử dung lượng giúp xác minh rằng hệ thống có thể đáp ứng đúng yêu cầu sử dụng thực tế, chẳng hạn một hệ thống hồ sơ y tế cần đảm bảo khả năng xử lý dữ liệu của hàng chục triệu người dùng.

Về mặt kỹ thuật, việc thực hiện kiểm thử khả năng thường bao gồm xây dựng một công cụ giả lập (test harness) để sinh ra một lượng lớn dữ liệu mô phỏng, hoặc sử dụng dữ liệu lịch sử nếu có. Các chỉ số quan trọng được theo dõi bao gồm: thời gian phản hồi, mức sử dụng bộ nhớ, trạng thái của hệ thống khi đạt đến hoặc vượt quá giới hạn tài nguyên. Điều này cho phép các kỹ sư đánh giá hiệu suất của hệ thống dưới điều kiện tải cực hạn.

Kiểm thử khả năng có một số ưu điểm nổi bật, như giúp gia tăng độ tin cậy của hệ thống bằng cách xác minh rằng hệ thống có thể hoạt động ổn định dưới tải lớn, từ đó giúp phát hiện và xử lý các vấn đề tiềm ẩn trước khi triển khai thực tế. Tuy nhiên, kỹ thuật này cũng tồn tại một số hạn chế. Việc tạo ra dữ liệu mô phỏng có thể không phản ánh chính xác hành vi sử dụng thật của người dùng, đồng thời quá trình thiết lập và thực thi các kịch bản kiểm thử có thể rất phức tạp và tốn nhiều thời gian, tài nguyên.

Kiểm thử chịu tải cực hạn (Stress Testing) Stress Testing là một kỹ thuật kiểm thử phi chức năng được sử dụng để đánh giá khả năng của hệ thống trong việc hoạt động ổn định dưới các điều kiện bất lợi như tải đột biến, môi trường khắc nghiệt hoặc khi các tài nguyên hệ thống đạt đến giới hạn. Kỹ thuật này đặc biệt quan trọng trong các hệ thống yêu cầu phản hồi theo thời gian thực hoặc hệ thống nhúng, chẳng hạn như hệ thống điều khiển phanh ABS trong ô tô, các thiết bị y tế, hoặc hệ thống điều khiển trong ngành hàng không. Trong những trường hợp này, hệ thống được kỳ vọng phải hoạt động ổn định nhất khi đối mặt với những điều kiện khó khăn nhất – điều này khiến kiểm thử chịu tải trở thành một phần không thể thiếu trong quy trình đảm bảo chất lượng.

Về mặt kỹ thuật, kiểm thử chịu tải thường được thực hiện bằng cách mô phỏng những tình huống cực đoan như gửi đồng loạt hàng loạt thông điệp từ cảm biến, tái tạo điều kiện môi trường lạnh, ẩm, nhiễu điện từ hoặc rung động cơ học. Một mô hình thử nghiệm (test rig) có thể được thiết kế để tích hợp và kiểm tra đồng thời cả phần cứng, phần mềm và các yếu tố môi trường. Trong trường hợp cần thực hiện kiểm thử khi hệ thống đã được triển khai (in-service), có thể sử dụng chính hệ thống thật để đảm bảo độ chính xác và sát với điều kiện vận hành thực tế.

Mục tiêu chính của stress testing là xác minh rằng hệ thống có thể tiếp tục vận hành đúng và an toàn ngay cả khi bị đặt vào các tình huống vượt ngoài phạm vi hoạt động thông thường. Đây là điều đặc biệt quan trọng đối với các hệ thống nhúng và thời gian thực, vốn được người dùng tin cậy trong những tình huống khẩn cấp. Kiểm thử kiểu này mang lại giá trị lớn khi giúp phát hiện ra những lỗi tiềm ẩn chỉ xuất hiện trong điều kiện bất thường – điều mà các phương pháp kiểm thử thông thường khó có thể phát hiện.

Tuy nhiên, stress testing cũng gặp một số hạn chế. Việc xây dựng một môi trường kiểm thử đủ đại diện cho các điều kiện thực tế có thể gặp nhiều khó khăn, hoặc thậm chí là không thể. Ví dụ, trong hệ thống ô tô, môi trường sóng radio thực tế thường rất nhiều và phức tạp, nhưng việc tái tạo chính xác môi trường này trong phòng thí nghiệm có thể không khả thi. Ngoài ra, việc kiểm thử có thể bỏ sót một số yếu tố thực tế nếu các thành phần trong môi trường bị mô phỏng không đầy đủ.

Kiểm thử tính dùng được (Usability Testing) là một kỹ thuật kiểm thử phi chức năng quan trọng, được áp dụng khi hệ thống có giao diện người dùng đáng kể và tính thân thiện, dễ sử dụng đóng vai trò then chốt trong việc đảm bảo hiệu quả vận hành. Loại kiểm thử này đặc biệt cần thiết trong các hệ thống mà sai sót của người dùng có thể dẫn đến hậu quả nghiêm trọng, hoặc khi nhà phát triển muốn xây dựng sản phẩm có trải nghiệm người dùng vượt trội hơn so với các đối thủ cạnh tranh.

Phương pháp kiểm thử tính dùng được có thể thực hiện theo nhiều cách khác nhau. Trong các hệ thống nhúng hoặc phần mềm gắn với thiết bị vật lý, có thể xây dựng một mô phỏng hệ thống (simulator) để quan sát cách người dùng tương tác trong điều kiện tương tự thực tế. Trong những trường hợp khác, đặc biệt là với phần mềm có giao diện đồ họa, một nhóm người dùng có thể được mời thử nghiệm hệ thống trong một môi trường có kiểm soát, nơi hành vi của họ được ghi nhận và phân tích. Mục đích của kiểm thử là xác định những điểm gây nhầm lẫn, khó hiểu, không thuận tiện, hoặc tạo ra gánh nặng nhận thức quá lớn cho người dùng.

Kiểm thử tính dùng được mang lại nhiều lợi ích đáng kể, đặc biệt trong những

bối cảnh ứng dụng rõ ràng và yêu cầu tương tác cao. Kỹ thuật này có thể cung cấp các phản hồi chất lượng cao từ người dùng thực tế và thường được hỗ trợ bởi các lý thuyết nền tảng như ước lượng tải nhận thức (cognitive load) hay quy tắc thiết kế giao diện người dùng. Nhờ đó, các nhà phát triển có thể điều chỉnh giao diện và luồng tương tác nhằm tối ưu hóa trải nghiệm người dùng.

Tuy nhiên, kỹ thuật này cũng gặp một số hạn chế. Một số yêu cầu về tính dùng được rất khó được biểu đạt rõ ràng dưới dạng tiêu chí kiểm thử cụ thể, dẫn đến khó khăn trong việc thiết kế ca kiểm thử phù hợp. Hơn nữa, ngay cả khi kiểm thử được tiến hành với quy mô lớn, việc xử lý và rút ra kết luận từ dữ liệu thu thập được không phải lúc nào cũng rõ ràng – đặc biệt nếu thiếu tiêu chuẩn định lượng cụ thể hoặc phương pháp phân tích phù hợp. Điều này khiến việc chuyển đổi dữ liệu người dùng thành hành động cải tiến thiết kế trở nên khó khăn và tốn nhiều công sức.

Kiểm thử bảo mật (Security Testing) là một bước kiểm thử thiết yếu đối với các hệ thống có khả năng bị truy cập từ bên ngoài hoặc có những chức năng cần được bảo vệ khỏi sự gián đoạn hoặc truy cập trái phép. Mục tiêu chính của kiểm thử bảo mật là xác định và loại bỏ các điểm yếu trong hệ thống có thể bị khai thác bởi người dùng độc hại (malicious users), từ đó đảm bảo rằng hệ thống hoạt động liên tục, đúng chức năng và không bị xâm phạm bởi các cuộc tấn công từ bên ngoài.

Các phương pháp kiểm thử bảo mật rất đa dạng. Một cách tiếp cận phổ biến là sử dụng các bảng xếp hạng hoặc danh sách lỗi bảo mật điển hình để rà soát mã nguồn, ví dụ như “SANS Top 25 Most Dangerous Software Errors” – danh sách những lỗi lập trình liên quan đến bảo mật thường gặp và nguy hiểm nhất. Ngoài ra, một phương pháp hiệu quả khác là tổ chức một nhóm kiểm thử chuyên trách (thường gọi là red team hoặc ethical hacker team) với mục tiêu tìm cách tấn công, đột nhập hoặc phá vỡ hệ thống để phát hiện lỗ hổng bảo mật tiềm ẩn mà các phương pháp phân tích truyền thống có thể bỏ sót.

Mục đích của kiểm thử bảo mật không chỉ là để bảo vệ hệ thống khỏi các cuộc tấn công mạng, mà còn là để duy trì uy tín và sự tin cậy từ phía người dùng. Đối với một số hệ thống thiết yếu như hệ thống viễn thông, ngân hàng, hoặc cơ sở dữ liệu y tế, việc đảm bảo hoạt động ổn định và bảo mật là yêu cầu bắt buộc, bởi chỉ cần một lỗ hổng nhỏ cũng có thể dẫn đến hậu quả nghiêm trọng về tài chính, pháp lý hoặc danh tiếng.

Ưu điểm của kiểm thử bảo mật nằm ở chỗ đây là cách tốt nhất hiện nay để xác minh mức độ an toàn của một hệ thống, đặc biệt khi được kết hợp với thiết kế an toàn từ đầu (secure-by-design) và việc sử dụng các thành phần phần mềm đã được kiểm chứng về bảo mật. Tuy nhiên, kỹ thuật này cũng có những hạn chế nhất định.

Việc dựa vào danh sách lỗi hoặc checklist chỉ cho phép phát hiện các kiểu tấn công đã được biết đến trong quá khứ và không thể bao quát các phương thức tấn công mới, sáng tạo hơn. Do đó, việc kết hợp cả kiểm thử tĩnh (dựa vào checklist) và kiểm thử động (thử đột nhập thực tế) sẽ mang lại cái nhìn toàn diện hơn về tính bảo mật của hệ thống.

Kiểm thử hiệu năng (Performance Testing) là một kỹ thuật kiểm thử phi chức năng được sử dụng để đánh giá mức độ đáp ứng của hệ thống so với các mục tiêu hiệu suất đã được xác định trước, thường là theo cam kết trong các thỏa thuận mức dịch vụ (Service Level Agreement – SLA). Ví dụ, một nhà cung cấp dịch vụ Internet (ISP) có thể cam kết cung cấp tốc độ tải xuống tối thiểu là 2Mb/s; hoặc hệ thống thanh toán VISA có thể đảm bảo thời gian phản hồi cho mỗi giao dịch không vượt quá 5 giây. Trong các trường hợp này, kiểm thử hiệu năng là công cụ quan trọng giúp xác minh hệ thống có đáp ứng được các yêu cầu về hiệu suất hay không.

Có hai cách tiếp cận chính để thực hiện kiểm thử hiệu năng: thứ nhất là mô hình hóa hoặc mô phỏng hành vi hệ thống trong môi trường kiểm thử, sử dụng các công cụ như trình mô phỏng tải (load simulator); thứ hai là thực hiện kiểm thử trực tiếp trên hệ thống thật hoặc trong môi trường mô phỏng gần giống thực tế. Cả hai phương pháp đều có mục tiêu là đo lường các chỉ số hiệu suất chính như thời gian phản hồi, thông lượng, mức sử dụng CPU/bộ nhớ và độ ổn định dưới tải.

Mục tiêu của kiểm thử hiệu năng không chỉ là đảm bảo hệ thống hoạt động đúng theo cam kết, mà còn nhằm xác định các điểm nghẽn hiệu năng (performance bottlenecks), giúp nhà phát triển tối ưu hóa hệ thống. Điều này đặc biệt quan trọng với các hệ thống mà hiệu suất ảnh hưởng trực tiếp đến trải nghiệm người dùng và tính cạnh tranh của sản phẩm – chẳng hạn như các nền tảng giao dịch tài chính hoặc dịch vụ trực tuyến tốc độ cao.

Ưu điểm của kiểm thử hiệu năng là nó cung cấp bằng chứng xác thực và định lượng về khả năng đáp ứng của hệ thống, đồng thời giúp phát hiện sớm các vấn đề về hiệu suất trước khi hệ thống được triển khai thực tế. Ngoài ra, các kỹ thuật mô hình hóa hiệu năng còn có thể giúp dự đoán hành vi hệ thống trong các tình huống tải cao, từ đó hỗ trợ ra quyết định về mở rộng hoặc cải tiến kiến trúc. Tuy nhiên, một trong những hạn chế lớn nhất của kiểm thử hiệu năng là độ đại diện của các kịch bản kiểm thử. Nếu môi trường kiểm thử không sát với thực tế hoặc không mô phỏng đúng hành vi người dùng thực, kết quả thu được có thể không phản ánh đúng khả năng của hệ thống khi triển khai ngoài thực địa.

Kiểm thử độ tin cậy (Reliability Testing) là một kỹ thuật kiểm thử nhằm

đánh giá khả năng hệ thống có thể vận hành liên tục mà không gặp lỗi trong một khoảng thời gian xác định. Loại kiểm thử này đặc biệt quan trọng trong các hệ thống đòi hỏi độ an toàn hoặc tính liên tục cao như phần mềm điều khiển nhà máy điện hạt nhân, thiết bị y tế, hệ thống viễn thông hay hàng không. Ví dụ, một hệ thống điều khiển trong nhà máy điện hạt nhân có thể được yêu cầu không xảy ra quá một lỗi trong 10.000 giờ vận hành, để đảm bảo an toàn ở mức tuyệt đối.

Để thực hiện kiểm thử độ tin cậy, cần thiết lập một tập dữ liệu kiểm thử đại diện cho điều kiện sử dụng thực tế của người dùng, đồng thời thu thập một lượng dữ liệu đủ lớn để đưa ra các tuyên bố thống kê có giá trị về độ tin cậy. Các kỹ thuật như mô hình cấu trúc hệ thống (system structured modelling) có thể được sử dụng để chứng minh mối liên hệ giữa tỷ lệ lỗi của toàn hệ thống với tỷ lệ lỗi của các thành phần riêng lẻ. Điều này đặc biệt hữu ích trong việc lập luận rằng một hệ thống với các thành phần có độ tin cậy cao sẽ có xác suất lỗi tổng thể rất thấp.

Mục đích chính của kiểm thử độ tin cậy là nhằm cung cấp bằng chứng định lượng cho các tuyên bố về độ tin cậy, thường là để đáp ứng yêu cầu của cơ quan quản lý hoặc khách hàng trong các ngành công nghiệp nhạy cảm. Ngoài ra, doanh nghiệp cũng có thể sử dụng kết quả kiểm thử để cạnh tranh trên thị trường, đặc biệt nếu muốn bắt kịp hoặc vượt qua mức độ tin cậy mà các đối thủ dẫn đầu đang cung cấp.

Ưu điểm lớn của kỹ thuật này là nếu tập dữ liệu kiểm thử được thiết kế đúng và có tính đại diện cao, thì các dự đoán về độ tin cậy của hệ thống có thể đạt độ chính xác cao, từ đó hỗ trợ ra quyết định trong thiết kế, bảo trì và triển khai sản phẩm. Tuy nhiên, điểm yếu của kiểm thử độ tin cậy nằm ở chỗ để đưa ra các tuyên bố đáng tin cậy về các hệ thống có độ tin cậy cao, người kiểm thử cần một khối lượng dữ liệu cực lớn. Điều này vừa tốn kém thời gian, vừa dễ dẫn đến sự lạc quan không thực tế nếu dữ liệu thu được không đủ đa dạng hoặc không phản ánh đúng các điều kiện vận hành trong thế giới thực.

Kiểm thử tuân thủ (Compliance Testing) là một kỹ thuật kiểm thử được sử dụng khi phần mềm cần được triển khai trong một thị trường có quy định hoặc tiêu chuẩn bắt buộc, và việc bán hoặc phân phối sản phẩm phụ thuộc vào khả năng chứng minh rằng phần mềm đáp ứng các tiêu chuẩn đó. Ví dụ, nếu một công ty phát triển trình biên dịch ngôn ngữ lập trình C, họ cần chứng minh rằng sản phẩm của mình có khả năng biên dịch đúng các chương trình C theo tiêu chuẩn ANSI C. Kiểm thử tuân thủ đóng vai trò quan trọng trong việc chứng minh sự phù hợp của hệ thống với các quy chuẩn kỹ thuật, luật định hoặc tiêu chuẩn công nghiệp.

Phương pháp kiểm thử tuân thủ thường dựa trên một bộ dữ liệu kiểm thử đã

được chuẩn hóa, đại diện cho một tập hợp hành vi được mong đợi từ hệ thống. Ví dụ, trong trường hợp trình biên dịch C, bộ kiểm thử có thể bao gồm hàng trăm chương trình C mẫu cùng với kết quả kỳ vọng sau khi biên dịch và thực thi. Các tổ chức phát triển phần mềm sử dụng bộ kiểm thử chuẩn này để so sánh đầu ra thực tế với đầu ra tiêu chuẩn, từ đó xác định mức độ tuân thủ.

Mục tiêu của kiểm thử tuân thủ không chỉ là để thỏa mãn yêu cầu pháp lý hoặc kỹ thuật, mà còn nhằm xác định các khu vực còn yếu trong hệ thống, giúp tổ chức xây dựng các ca kiểm thử bổ sung để cải thiện chất lượng tổng thể. Ngoài ra, vì các bài kiểm thử tuân thủ thường được phát triển và sử dụng chung giữa nhiều tổ chức trong ngành, chi phí để phát triển và duy trì bộ kiểm thử được chia sẻ, điều này làm giảm chi phí kiểm thử đơn lẻ cho từng tổ chức mà vẫn đảm bảo độ bao phủ tốt.

Tuy nhiên, một nhược điểm lớn của kiểm thử tuân thủ là các nhà phát triển phần mềm thường có xu hướng "hướng tới kiểm thử" – nghĩa là họ tối ưu hóa phần mềm chỉ để vượt qua bộ bài kiểm thử tuân thủ, thay vì đảm bảo chất lượng trên toàn bộ tập các tình huống sử dụng thực tế. Điều này dẫn đến rủi ro là phần mềm tuy vượt qua được bài kiểm thử nhưng lại hoạt động kém trong các trường hợp thực tế không nằm trong tập kiểm thử tiêu chuẩn. Do đó, mặc dù kiểm thử tuân thủ là cần thiết, nó nên được bổ sung bởi các kỹ thuật kiểm thử bổ sung khác như kiểm thử người dùng thực tế, kiểm thử hiệu năng, và kiểm thử phi chức năng để đảm bảo chất lượng toàn diện.

Kiểm thử tính sẵn sàng (Availability Testing) là một kỹ thuật kiểm thử phi chức năng nhằm đánh giá khả năng hệ thống có thể duy trì hoạt động liên tục và khôi phục nhanh chóng sau khi xảy ra sự cố. Kỹ thuật này đặc biệt quan trọng trong bối cảnh các hệ thống cung cấp dịch vụ quan trọng hoặc theo thời gian thực, nơi thời gian chết (downtime) phải được tối thiểu hóa để đảm bảo chất lượng dịch vụ, ví dụ như dịch vụ cấp cứu (999) hoặc hệ thống ngân hàng trực tuyến. Trong các môi trường này, tính sẵn sàng thường được xem là một chỉ số hiệu suất chính (Key Performance Indicator – KPI) và có ảnh hưởng trực tiếp đến uy tín và nghĩa vụ pháp lý của nhà cung cấp dịch vụ.

Về mặt phương pháp, kiểm thử tính sẵn sàng có nhiều điểm tương đồng với kiểm thử độ tin cậy (reliability testing). Tuy nhiên, thay vì chỉ tập trung vào việc hệ thống thất bại bao lâu một lần, kiểm thử tính sẵn sàng nhấn mạnh vào việc đo lường thời gian phục hồi sau thất bại. Trong một số trường hợp, các lỗi có thể được chủ động "gieo" (seeded) hoặc mô phỏng lỗi phần cứng/phần mềm để quan sát khả năng tự động khôi phục, thời gian phát hiện lỗi và thời gian đưa hệ thống quay trở lại trạng thái ổn định. Việc này giúp đánh giá khả năng duy trì hoạt động của hệ

thống khi có sự cố bất ngờ xảy ra.

Mục đích chính của kiểm thử tính sẵn sàng là đảm bảo rằng các dịch vụ quan trọng luôn sẵn sàng cho người dùng, đồng thời có khả năng hồi phục nhanh chóng sau gián đoạn. Đây là yếu tố sống còn đối với các hệ thống phục vụ công chúng, nơi mà mỗi giây ngừng hoạt động có thể gây ra hậu quả nghiêm trọng về xã hội hoặc kinh tế.

Về mặt lợi ích, kiểm thử tính sẵn sàng cũng chia sẻ những điểm mạnh tương tự như kiểm thử độ tin cậy, đặc biệt trong việc cung cấp bằng chứng định lượng cho khả năng vận hành bền vững của hệ thống. Tuy nhiên, kỹ thuật này cũng gặp phải những thách thức tương tự. Một trong số đó là sự khác biệt giữa môi trường kiểm thử và môi trường thực tế. Trong thực tế, quá trình khôi phục có thể diễn ra nhanh hơn nhờ vào kinh nghiệm tích lũy của đội ngũ vận hành, sự hỗ trợ kỹ thuật tốt hơn hoặc hệ thống giám sát thông minh. Điều này khiến cho kết quả kiểm thử trong môi trường phòng lab có thể không hoàn toàn phản ánh chính xác thời gian phục hồi thực tế khi vận hành ngoài thực địa.

Kiểm thử tài liệu hướng dẫn (Documentation Testing) là một hoạt động kiểm thử nhằm đảm bảo tính nhất quán giữa phần mềm và các tài liệu đi kèm như hướng dẫn sử dụng, tài liệu cài đặt, hoặc tài liệu đào tạo. Trong nhiều hệ thống, đặc biệt là các phần mềm thương mại hoặc công cụ dành cho lập trình viên, việc tài liệu không phản ánh đúng hành vi thực tế của hệ thống có thể gây nhầm lẫn lớn cho người dùng, làm giảm hiệu quả sử dụng và gây tổn hại đến uy tín của nhà phát triển. Do đó, kiểm thử tài liệu là một phần không thể thiếu trong quá trình kiểm thử tổng thể, đặc biệt trong các đợt phát hành chính thức (release).

Phương pháp thực hiện kiểm thử tài liệu có thể bao gồm việc duy trì một tập kiểm thử (test set) chuyên dùng để xác minh rằng các mô tả, ví dụ và quy trình được trình bày trong tài liệu là chính xác so với hành vi thực tế của phần mềm. Trong một số trường hợp, các hệ thống tích hợp sẵn các bài kiểm thử trong tài liệu (documentation-based tests hoặc doctests), và việc chạy thành công các bài kiểm thử này là một bước bắt buộc trong quy trình phát hành phần mềm mới. Một cách tiếp cận đơn giản hơn là mời người dùng hoặc kỹ sư kiểm thử thực hiện theo hướng dẫn trong tài liệu (ví dụ như làm theo tutorial) để phát hiện các lỗi sai lệch hoặc không đồng nhất.

Lý do chính cho kiểm thử tài liệu là để bảo vệ trải nghiệm người dùng. Nếu phần mềm hoạt động khác với những gì được mô tả trong tài liệu, người dùng có thể cảm thấy bối rối, không hài lòng, thậm chí từ bỏ việc sử dụng sản phẩm. Tính nhất quán giữa hệ thống và tài liệu hướng dẫn do đó đóng vai trò quan trọng trong việc đảm

bảo chất lượng phần mềm.

Ưu điểm của kỹ thuật này là nó giúp phát hiện và loại bỏ những mâu thuẫn giữa tài liệu và phần mềm thực tế, từ đó tăng tính chuyên nghiệp và tin cậy cho sản phẩm. Tuy nhiên, một điểm hạn chế là việc kiểm thử tài liệu thường tập trung vào các ví dụ minh họa (examples), quy trình thao tác cụ thể, mà ít chú trọng đến việc đánh giá tính nhất quán hoặc logic của các phần nội dung mang tính mô tả hoặc diễn giải (narrative consistency). Do đó, để kiểm thử toàn diện tài liệu, cần kết hợp giữa kiểm thử tự động (doctests) và đánh giá thủ công có hệ thống từ phía chuyên gia kỹ thuật và người dùng thực tế.

7.4 Kiểm thử chấp nhận (Acceptance Testing)

Kiểm thử chấp nhận (Acceptance Testing) là giai đoạn cuối cùng trong quy trình kiểm thử phần mềm, được thực hiện bởi khách hàng hoặc người đại diện khách hàng tại chính môi trường triển khai thực tế của họ. Giai đoạn này mang tính độc lập hoàn toàn so với tổ chức phát triển phần mềm, nhằm đảm bảo rằng sản phẩm được kiểm tra một cách khách quan, theo đúng các tiêu chí đã thống nhất trong hợp đồng hoặc đặc tả yêu cầu.

Mục tiêu chính của kiểm thử chấp nhận là để khách hàng xác định xem hệ thống có đáp ứng đầy đủ các yêu cầu chức năng và phi chức năng đã đề ra hay không. Đây là cột mốc quan trọng đánh dấu sự chuyển giao từ giai đoạn phát triển sang giai đoạn sử dụng chính thức, đồng thời thể hiện mức độ tin tưởng của khách hàng đối với sản phẩm phần mềm đã bàn giao.

Kết quả của kiểm thử chấp nhận đóng vai trò quyết định trong việc khách hàng có đồng ý chấp nhận hệ thống hay yêu cầu sửa đổi, cải tiến thêm. Vì vậy, các kịch bản kiểm thử trong pha này cần được thiết kế bám sát với các quy trình nghiệp vụ thực tế và do chính người sử dụng cuối cùng xác nhận hoặc tham gia thực hiện. Việc vượt qua kiểm thử chấp nhận là điều kiện tiên quyết để hệ thống chính thức được đưa vào vận hành trong môi trường sản xuất.

Trong thực tiễn phát triển phần mềm, kiểm thử chấp nhận (Acceptance Testing) có thể được phân thành nhiều loại khác nhau, tùy thuộc vào đối tượng thực hiện và mục tiêu cụ thể của hoạt động kiểm thử. Hai dạng phổ biến nhất là Kiểm thử chấp nhận người dùng (User Acceptance Testing – UAT) và Kiểm thử chấp nhận nghiệp vụ (Business Acceptance Testing – BAT).

Kiểm thử chấp nhận người dùng (User Acceptance Testing) là hình thức kiểm thử được thực hiện bởi khách hàng hoặc người đại diện của họ, nhằm xác minh rằng

hệ thống đáp ứng đúng các tiêu chí chấp nhận đã được quy định trong hợp đồng hoặc đặc tả yêu cầu. Mặc dù việc lập kế hoạch và thực hiện các bài kiểm thử không nhất thiết phải do chính khách hàng trực tiếp thực hiện, nhưng họ vẫn giữ vai trò chủ đạo trong việc xác nhận kết quả. Trong nhiều trường hợp, khách hàng có thể thuê một bên thứ ba độc lập để thực hiện kiểm thử, đảm bảo tính khách quan và chuyên nghiệp trong đánh giá sản phẩm.

Kiểm thử chấp nhận nghiệp vụ (Business Acceptance Testing) lại thường được tiến hành bên trong tổ chức phát triển phần mềm. Ở đây, nhóm phát triển chịu trách nhiệm phân tích hợp đồng hoặc tài liệu yêu cầu để thiết kế và thực hiện các ca kiểm thử phù hợp. Mục tiêu là bảo đảm hệ thống phù hợp với các quy trình nghiệp vụ của khách hàng, đồng thời chuẩn bị cho việc bàn giao và chấp nhận chính thức. Hình thức kiểm thử này đóng vai trò như một bước xác minh cuối cùng từ phía nhà phát triển trước khi hệ thống được chuyển sang kiểm thử chấp nhận người dùng.

Việc kết hợp cả hai hình thức UAT và BAT giúp đảm bảo rằng hệ thống không chỉ đúng về mặt kỹ thuật mà còn phù hợp với nhu cầu nghiệp vụ và kỳ vọng thực tế của khách hàng.

Tiêu chí chấp nhận (Acceptance Criteria) là tập hợp các điều kiện hoặc yêu cầu được sử dụng để xác định xem một hệ thống phần mềm có được chấp nhận chính thức bởi khách hàng hay không. Các tiêu chí này thường được xây dựng dựa trên đặc tả yêu cầu và được sử dụng như cơ sở để đánh giá kết quả của kiểm thử chấp nhận. Một số tiêu chí phổ biến bao gồm khả năng sử dụng, hiệu năng, độ tin cậy và tính sẵn sàng của hệ thống.

Khả năng sử dụng (Usability) là yếu tố then chốt, đặc biệt với các hệ thống có giao diện người dùng trực tiếp. Các câu hỏi như hệ thống có dễ sử dụng không?, người dùng mất bao lâu để học cách vận hành hệ thống? là những tiêu chí phổ biến để đánh giá tính khả dụng. Hệ thống cần thân thiện với người dùng cuối và dễ tiếp cận, đặc biệt là với người dùng không chuyên về kỹ thuật.

Hiệu năng (Performance) cũng là một tiêu chí quan trọng, yêu cầu xác định rõ các đặc điểm hiệu suất mong muốn như tốc độ xử lý, thời gian phản hồi hoặc dung lượng dữ liệu mà hệ thống có thể xử lý trong một đơn vị thời gian. Việc đo lường hiệu năng chỉ có ý nghĩa nếu được so sánh với các ngưỡng hoặc mục tiêu cụ thể đã được định nghĩa trước trong tài liệu yêu cầu hoặc hợp đồng.

Độ tin cậy và tính sẵn sàng (Reliability and Availability) là yếu tố đặc biệt quan trọng với các hệ thống nghiệp vụ hoặc dịch vụ quan trọng. Một số câu hỏi cần xem xét bao gồm: Tỷ lệ lỗi hiện tại của hệ thống là bao nhiêu?, Nếu kiểm thử tiếp tục được thực hiện trong thời gian dài, tỷ lệ lỗi sẽ thay đổi như thế nào?, Có bao nhiêu

lỗi còn tiềm ẩn trong hệ thống?, và Cần thực hiện thêm bao nhiêu lần kiểm thử để đạt được một tỷ lệ lỗi chấp nhận được? Việc trả lời các câu hỏi này cho phép định lượng độ tin cậy của phần mềm, từ đó đưa ra quyết định chính xác về việc chấp nhận hệ thống.

Tài liệu tham khảo

